Università di Pisa

Dipartimento di Informatica
Corso di Laurea Magistrale in Informatica

Compressed Representations of Set Collections

Candidato
Luca Lombardo

Relatore
Roberto Grossi

Anno Accademico 2025/2026

Contents

Contents ii

1 Introduction 1
1.1 Why compress set collections? . . . 1
1.2 Set collections . . . 4
1.3 Contribution and roadmap . . . 6

2 Succinct Dictionaries 7
2.1 Rank and Select . . . 7
2.2 Sparse Bitvectors . . . 15
2.3 Wavelet trees . . . 20

3 Succinct Representations of Trees 26
3.1 Encoding problem . . . 26
3.2 Depth-first encodings . . . 31
3.3 Fully functional encodings . . . 36
3.4 Labeled trees and α-operations . . . 40

4 Tree Extraction and Labeled Trees 43
4.1 Tree extraction . . . 43
4.2 α-operations on labeled trees . . . 46
4.3 Path queries . . . 49

5 Compressibility Measures for Set Collections 57
5.1 Setup and baseline . . . 57
5.2 Containment and insertion . . . 58
5.3 Symmetric-difference compressibility . . . 62
5.4 Atom-based information-theoretic bound . . . 69
5.5 Set-Union Matching . . . 73

Appendix 77

A Lattice closure framework for set collection encoding 78
A.1 Formulation and structure . . . 78
A.2 Variant landscape, adjacent literature, and open problems . . . 83

Bibliography 87

List of Figures

2.1 A bitvector $B[1..20]$. The prefix $B[1..15]$ (shaded red) contains nine ones, so $\text{rank}_1(B, 15) = 9$. The seventh one (circled blue) lies at position 12, so $\text{select}_1(B, 7) = 12$. 8

2.2 The two-level rank directory of Jacobson's construction. Superblocks of size Z store absolute ranks r_i; blocks of size z inside each superblock store ranks $r_{i,j}$ relative to the enclosing superblock. The bottom row represents the four-Russians lookup table, indexed by the z-bit intra-block pattern and an in-block offset, returning the rank inside the pattern. 9

2.3 Elias-Fano encoding of the set $\{5, 7, 8, 15, 32\}$ with universe $n = 36$ and $m = 5$, so $w = 6$ and $z = \lfloor \lg 5 \rfloor = 2$. Each element is split into a high part of z bits and a low part of $w - z = 4$ bits. The array L is the concatenation of the low parts in order; the bitvector H unary-codes the successive increments of the high parts $(0, 0, 0, 0, 2)$. To answer $\text{select}_1(j)$, locate the j-th one in H at position $p = \text{select}_1(H, j)$, recover the high part $q_j = p - j$, and concatenate it with the low part $L[j]$. . . 17

2.4 Wavelet tree for the string $S =$ `wookies_wield_wicked_weapons_with_wisdom$` over an alphabet of 17 symbols. Each internal node carries a sub-alphabet interval; its bitmap encodes, for every symbol of the implicit subsequence, which side of the midpoint split it falls on. The subsequences are drawn beside each node for illustration; the structure stores only the topology and the bitmaps. 21

2.5 Trace of $\text{rank}_{\texttt{e}}(S, 13) = 2$ on the wavelet tree of Figure 2.4. The descent follows the root-to-leaf path of the target symbol `e`, remapping the query prefix length through one binary rank at each of four levels. The intermediate values $13 \to 6 \to 5 \to 3 \to 2$ trace the query index from the root to the leaf [`e`, `e`]. Each remapping is a rank_0 or rank_1 on the local bitmap, selected by whether `e` falls in the left or right half of the current alphabet interval. 22

3.1 LOUDS encoding of an ordinal tree on seven nodes. The super-root $\perp$ is added so that every node, including the original root, has a unique parent and contributes a unary degree codeword. The augmented tree is traversed in BFS order, the BFS indices $1, \ldots, 8$ are shown next to each node, and the degree of each node in turn is written in unary as 1^d0, concatenated to form the bitmap B. Grouping braces above B mark the codewords contributed by the super-root and by nodes a, b, c, d, e, f, g in BFS order. 29

3.2 Balanced parenthesis encoding of the same seven-node ordinal tree used in Figure 3.1. A preorder DFS visit writes an open parenthesis on entry to a node and a close on exit. Each dashed arc joins the matching pair of one node, labelled by the node it represents. The substring between the two parentheses of a node is exactly the encoding of its subtree, of length $2 \cdot |T_v|$. 32

3.3 DFUDS encoding of the same seven-node ordinal tree of Figure 3.2. Each node v contributes a block of d_v open parentheses followed by one close, written in DFS preorder; a leading open parenthesis (block $\perp$) is prepended so that the resulting string of length $2n = 14$ is balanced. The colored blocks above the sequence highlight the per-node contributions of a, b, e, c, d, f, g in preorder. The position p_v of the first parenthesis of each block is the address by which the node is referenced. 35

3.4 A two-level tree covering of an ordinal tree. Solid rectangles enclose minitrees produced by the covering procedure with parameter M; the recursive application of the same procedure with parameter $M' < M$ partitions each minitree into the microtrees enclosed by dashed rectangles. Two minitrees intersect at most at their common roots, and the same property holds within a minitree for its microtrees. The two minitrees μ_1, μ_2 each have size in $[M, 3M - 4]$ except for an undersized fragment containing the root of the whole tree; their microtrees obey the same bound for the parameter M'. 37

4.1 X-extraction on a twelve-node ordinal tree, reproduced from Gagie, He, and Navarro [9]. On the left, T with $X = \{A, B, D, E, F, H, I, J, K, O\}$ drawn as the shaded nodes; on the right, the extracted tree T_X. The deletion of each unshaded node promotes its children into the sibling list of its parent, and the natural bijection between X and $V(T_X)$ sends every shaded node to the position it occupies on the right. 44
4.2 From left to right, an ordinal tree T on five nodes with labels in $\{1, 2, 3\}$; the three extracted trees T_1, T_2, T_3 built on the X_α-extractions described in the text, with 0/1 markers drawn as empty and shaded circles; and the merged tree $\mathcal{T}$ obtained by appending the T_α as subtrees of a fresh root $\mathcal{R}$. The α-operations on T are answered by the 0/1 α-operations on the single tree $\mathcal{T}$, which is the role of the building block $\mathcal{D}_3$ of Lemma 4.2.1. 47
4.3 Hierarchical binary partition of a labeled ordinal tree for root-to-node path queries, reproduced from Gagie, He, and Navarro [9]. Starting from the root range, each intermediate tree $\mathcal{T}_{a,b}$ is 0/1-labeled by the midpoint of its range. Bold arrows lead to the child trees $\mathcal{T}_{a,m}$ on marker 0 and $\mathcal{T}_{m+1,b}$ on marker 1. The binary alphabet at every level makes each level transition cost $O(1)$ via a single rank or select on the marker sequence. 51

5.1 Reproduced from Gagie, He, and Navarro [9]. Left: an insertion graph for a collection $\mathcal{S} = \{S_1, \ldots, S_9\}$, with edge weights written as slanted values. The edge weights sum to $I(\mathcal{S}) = 20$. Right: the corresponding insertion tree obtained by expanding each edge $(v(p'(S)), v(S))$ of weight $w = |S \setminus p'(S)|$ into a chain of w labelled edges. The tree has $1 + I(\mathcal{S}) = 21$ nodes. Each set S_i labels the chain endpoint $v(S_i)$. 62
5.2 Reproduced from Gagie, He, and Navarro [9]. Left: a minimum-weight symdiff graph on $\mathcal{S} = \{S_1, \ldots, S_9\}$ with $\Delta(\mathcal{S}) = 13$. Dashed edges are available in the complete graph K but not used by the MST; they are the edges whose weight exceeds the largest MST edge weight $\ell = 2$. Right: the two indel trees rooted at U (drawn upside down) and at $\emptyset$. Chain labels $+x$ are insertions, $-x$ are deletions relative to the parent. 68
5.3 SUM on four pairwise-disjoint sets of size $u/4$. From F_0 to F_1, two merges pair (S_1, S_2) and (S_3, S_4) into synthetic unions of size $u/2$; the cost drops from $4 \lg \binom{u}{u/4}$ to $2 \lg \binom{u}{u/2} + 2 \lg \binom{u/2}{u/4}$. From F_1 to F_2, a single merge produces U and the cost changes to $\lg \binom{u}{u/2} + 2 \lg \binom{u/2}{u/4}$; the structural gain $\lg \binom{u}{u/2}$ of the new merge outweighs the support saving $\lg \binom{u}{u/2}$ it removes, as proved in Proposition 5.5.1. The minimum is $F_{t^*} = F_1$. 75

A.1 A minimum-cost directed Steiner arborescence in $G_{\mathcal{S}}$ for $\mathcal{S} = \{S_1, S_2, S_3\}$ over $U = \{1, \ldots, 8\}$, with $S_1 = \{1, 2, 3, 4, 5\}$, $S_2 = \{4, 5, 6, 7\}$, $S_3 = \{5, 6, 8\}$. The synthetic node $M = S_2 \cup S_3 = \{4, 5, 6, 7, 8\}$ has size 5. Solid arcs form the arborescence (cost ≈ 17.26 bits); dashed arcs show the two direct $U \to S_2$ and $U \to S_3$ edges that a flat one-level encoding would use instead (cost ≈ 17.74 bits on this collection). 83

Introduction 1

1.1 Why compress set collections?

1.1 Why compress set collections? 1
1.2 Set collections 4
1.3 Contribution and roadmap 6

Every finite object has a shortest description. Given a universal Turing machine $\mathcal{U}$, the *Kolmogorov complexity* $K(x)$ of a binary string x is the length of the shortest program p such that $\mathcal{U}(p) = x$ [1]. This quantity captures the intrinsic information content of x. A string whose shortest description is nearly as long as the string itself is incompressible, while one admitting a substantially shorter program is compressible. Kolmogorov complexity does not depend on the choice of compression scheme, only on the choice of universal machine, and even that dependence vanishes up to an additive constant[1].

[1]: Li et al. (2008), *An introduction to Kolmogorov complexity and its applications*

1: The *invariance theorem* states that for any two universal machines $\mathcal{U}_1$ and $\mathcal{U}_2$, the corresponding complexities satisfy $|K_{\mathcal{U}_1}(x) - K_{\mathcal{U}_2}(x)| \leq c$ for a constant c independent of x.

[1]: Li et al. (2008), *An introduction to Kolmogorov complexity and its applications*

Kolmogorov complexity is, however, not computable [1]. No algorithm can determine, given an arbitrary string, whether a shorter program exists. We cannot even approximate $K(x)$ from above in general, since doing so would require solving the halting problem. Any practical compression scheme must identify specific structural regularities in the data (patterns, redundancies, correlations) and design representations that exploit them.

This observation shapes the way we think about data compression in the context of data structures. A classical compression algorithm treats data as a passive archive, compressing it as tightly as possible for storage and decompressing it in full before use. A *compact data structure*, by contrast, maintains the data in a compressed form that supports direct queries (membership tests, rank computations, element retrieval) without decompression [2]. The representation must use space close to a well-defined lower bound and support the desired operations efficiently. The formal study of compact data structures can be traced to the work of Jacobson [3], who showed that trees and graphs admit representations in space matching information-theoretic lower bounds while supporting navigational queries efficiently. Since then, the field has produced compact representations for bitvectors, sequences, permutations, trees, graphs, and text indexes, each measured against an appropriate entropy baseline.

[2]: Navarro (2016), *Compact data structures: A practical approach*

[3]: Jacobson (1989), *Space-efficient static trees and graphs*

What constitutes the *right* lower bound depends on the data and the queries we wish to support. For a single set S of n elements drawn from a universe U of size u, the number of possible such sets is $\binom{u}{n}$, and any encoding must use at least $\lg \binom{u}{n}$ bits to distinguish all possibilities.[2] This quantity is both the worst-case lower bound and the independent encoding cost for a single set. But when we have not one set but a *collection* of sets, the situation changes. Inter-set relationships may make the collection far more compressible than the sum of its parts. Identifying the right compressibility measure for a collection, and building data structures that achieve it, is the central problem of this thesis.

2: Throughout this thesis, lg denotes $\log_2$.

Data takes the form of a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ of finite subsets of a shared universe U in many application domains. Three examples

illustrate the scale of the problem and the correlations that independent encoding ignores.

Web graphs

A *web graph* is a directed graph in which each node represents a URL and each arc represents a hyperlink [4]. The adjacency list of a node (the set of its successors) is a subset of the vertex set, and the entire graph is naturally viewed as a collection of sets: one per node, drawn from the shared universe of all URLs. At the time of Boldi and Vigna's 2004 paper, the World Wide Web contained an estimated three billion nodes and more than fifty billion arcs, and these figures were only lower bounds, since they came from search engines that index only a fraction of the Web.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

The naive representation of a graph on N nodes stores each adjacency list as a sorted sequence of node identifiers, preceded by the outdegree. For the WebBase graph, a crawl of 118 million pages and one billion links, this representation is prohibitive. Boldi and Vigna identified two key empirical regularities. *Locality* means that most links are navigational, connecting pages within the same site, so that source and target URLs share long common prefixes and, when sorted lexicographically, have indices close to each other. *Similarity* means that pages close in lexicographic order share large segments of their successor lists, because many navigational links point to the same fixed set of pages within a site hierarchy.

Similarity in web graphs means that the *symmetric difference* between neighbouring adjacency lists is small, and this is the regularity that symmetric-difference compression exploits. The WebGraph framework encodes it through *reference compression*, where the adjacency list $S(x)$ of node x is encoded relative to the list $S(y)$ of a nearby reference node y, via a *copy list* indicating which elements of $S(y)$ also appear in $S(x)$, together with the remaining elements encoded using gap compression. On graphs with hundreds of millions of nodes and billions of links, WebGraph achieved compression ratios of a few bits per link [4]. The key observation is that adjacent adjacency lists, viewed as sets of node identifiers, are highly correlated, and their symmetric differences are small relative to the individual sets.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

Colored de Bruijn graphs in pangenomics

A *de Bruijn graph* of order k for a collection of strings is a directed graph whose vertices are the distinct k-mers (substrings of length k) and whose edges connect consecutive k-mers in the input [5]. In computational pangenomics, a *colored* de Bruijn graph augments this structure with color information: each k-mer is associated with a *color set*, the set of identifiers of the reference genomes that contain it. The color sets are subsets of a shared universe $\{1, \ldots, m\}$, where m is the number of reference genomes, and the collection of all color sets is the data structure that must be stored compactly and queried efficiently.

[5]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

The scale of modern pangenomic datasets makes compact representation essential. Alanko et al. evaluated Themisto, a colored k-mer index, on

collections of up to hundreds of thousands of *Salmonella enterica* genomes [5]. At these scales, competing tools either failed to complete within practical time limits or required an order of magnitude more memory.

[5]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

The color sets exhibit strong inter-set correlation. The most common color-set cardinality is 2, and the distribution of color-set sizes follows a power law. Most color sets are small, and a few very large color sets dominate the total space. Neighbouring k-mers in the de Bruijn graph tend to have identical or nearly identical color sets, because a mutation in a single genome affects only a few consecutive k-mers while leaving the rest of the color information unchanged. Almodaresi et al. [6] exploited this graph-topological locality by computing a minimum spanning tree over a sparse similarity graph induced by the de Bruijn graph structure, encoding each color set as a symmetric difference with its neighbour on the tree.[3]

[6]: Almodaresi et al. (2020), *An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search*

3: The containment entropy framework of Alanko et al. [7] was later tested on Themisto's color sets, achieving 0.18 bits per element compared to Themisto's default of 0.32 bits per element.

Each color set is a subset of $\{1, \ldots, m\}$, where the universe is the set of genome identifiers. The collection of all color sets (one per k-mer) must support membership queries (given a genome i and a k-mer x, decide whether x appears in genome i) and retrieval (list all genomes containing a given k-mer) in compressed space.

Inverted indexes in information retrieval

An *inverted index* maps each term in a text collection to its *posting list*, the set of documents in which the term appears. If the collection contains u documents and the vocabulary has m terms, the inverted index is a collection of m sets drawn from the universe $\{1, \ldots, u\}$. Bookstein and Klein [8] observed that posting lists consume large amounts of storage and that inter-list correlations offer an underexploited avenue for compression.

[8]: Bookstein et al. (1991), *Compression of correlated bit-vectors*

Certain words are strongly correlated across documents. If one word appears in a particular document, semantically related words are likely to appear in the same document. Bookstein and Klein proposed to exploit this correlation by compressing collections of bitvectors (the characteristic vectors of the posting lists) using their symmetric differences. If two bitvectors X_1 and X_2 are *close* in Hamming distance, then $X_1 \oplus X_2$ is a sparse bitvector that compresses well. The original X_2 can be recovered from X_1 and $X_1 \oplus X_2$.

The problem has a clean graph-theoretic structure. The bitvectors form the vertices of a complete graph, with edge weights given by Hamming distances. Introducing a zero vector X_0 (corresponding to the empty set) and computing a minimum spanning tree on this augmented graph yields the optimal clustering, solvable in $O(m^2 \log m)$ time on the $m + 1$ vertices. Each root bitvector is stored directly, and each non-root bitvector is stored as the symmetric difference with its parent on the tree. Experiments on the Hebrew Bible showed that this clustering nearly doubled the compression savings for some sets of bitmaps.

This 1991 paper is the earliest treatment, in the literature on set collections, of the principle that underlies all subsequent work on symmetric-difference compression. The idea was rediscovered independently by Adler and Mitzenmacher for web graph compression, by Bharat et al.

for web link information, by Buchsbaum et al. for massive tabular data, and by Björklund and Lingas for Boolean matrix multiplication [9]. Despite more than thirty years of applied work along these lines, a formal compressibility measure for set collections, with provable bounds on the achievable space and query time, was proposed only recently. In each case, the application-specific implementations restricted the graph structure for efficiency (Boldi and Vigna [4] to a window of W consecutive nodes, Almodaresi et al. to edges between neighbouring k-mers in the de Bruijn graph), sacrificing global optimality for practical speed.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

The question this thesis addresses

All three domains represent data as a collection of sets over a shared universe. Encoding each set independently costs $H_{wc}(\mathcal{S}) = \sum_i \lg \binom{u}{|S_i|}$ bits, ignoring the correlations between sets that the examples above make evident. What compressibility measure should we use for a collection of sets, and which queries can we support on the compressed representation?

The answer depends on which structural regularities we choose to exploit. If some sets are subsets of others, we can encode a smaller set cheaply relative to a larger one. Alanko, Bille, Gørtz, Navarro, and Puglisi [7] call the resulting measure the *containment entropy*. If sets that are neither subsets nor supersets of each other nevertheless have small symmetric differences, we can encode one in terms of the other via insertions and deletions. Gagie, He, and Navarro [9] define the *symdiff compressibility* to capture this. Both measures improve upon $H_{wc}(\mathcal{S})$ under the right conditions, and both yield compressed representations supporting the five primitives of membership, access, rank, predecessor, and successor (the building blocks of higher-level tasks such as set intersection and union) with logarithmic time bounds. Both also have limitations that leave room for further improvement.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

This thesis proposes a third compressibility measure, based on augmenting the collection with synthetic sets (unions of pairs) that serve as common ancestors in a hierarchy. The resulting measure, $L_{\mathrm{SUM}}(\mathcal{S})$, is never worse than $H_{wc}(\mathcal{S})$, and the hierarchy supports the same five operations with logarithmic query times. The construction, which we call *Set-Union Matching*, runs in polynomial time.

1.2 Set collections

The three domains in Section 1.1 share a common abstraction. Data takes the form of many sets over a shared universe, and the question of how to represent them compactly reduces to understanding what it means for a collection of sets to be compressible. We now state the problem precisely.

Definition 1.2.1 (Set collection) *A set collection is a tuple $(\mathcal{S}, U)$, where U is a finite, totally ordered set called the universe, and $\mathcal{S} = \{S_1, \ldots, S_m\}$ is a family of m finite subsets of U. We write $u = |U|$ for the size of the universe and $n = \sum_{i=1}^{m} |S_i|$ for the total size of the collection. We assume*

$U = [1..u] = \{1, 2, \ldots, u\}$ *without loss of generality, since any totally ordered set of size u can be mapped to this interval by a rank function computable in $O(1)$ time after $O(n \lg u)$-time preprocessing*[4].

4: If the universe is not $[1..u]$, one collects the n elements, sorts them, and assigns integers in $[1..u]$ to the u distinct values. Queries and answers translate in $O(1)$ time via a lookup table.

Each set S_i inherits the total order from U, so its elements can be listed as $S_i[1] < S_i[2] < \cdots < S_i[|S_i|]$. The total order is what makes rank, predecessor, and successor well-defined; without it, only membership and access would make sense. The integer n counts elements with multiplicity across sets; an element belonging to k sets contributes k to the sum. In practice $n \ll m \cdot u$, because most sets are small relative to the universe. In the web graph example from Section 1.1, a node with ten outgoing links contributes a set of size ten to the collection, even though the universe contains hundreds of millions of URLs.

A representation of $\mathcal{S}$ must support queries that allow us to recover information about any individual set S_i without decompressing the entire collection. Gagie, He, and Navarro [9] state the same five operations in their Section 1, and Alanko et al. [7] work with the same operations in a bitvector formulation that is equivalent after identifying each set with its characteristic vector.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

Definition 1.2.2 (Operations on set collections) *Given a set collection $(\mathcal{S}, U)$ with $U = [1..u]$, we define five operations on each $S \in \mathcal{S}$:*

- member(S, x)*: returns true if $x \in S$ and false otherwise.*
- access(S, i)*: given $i \in [1..|S|]$, returns the i-th smallest element of S, that is, $S[i]$.*
- rank(S, x)*: returns $|\{y \in S : y \leq x\}|$, the number of elements of S at most x.*
- predecessor(S, x)*: returns $\max\{y \in S : y \leq x\}$, or $-\infty$ if no such element exists.*
- successor(S, x)*: returns $\min\{y \in S : y \geq x\}$, or $+\infty$ if no such element exists.*

These five operations are the primitives on which higher-level tasks depend. Set intersection, for instance, can be computed by interleaving predecessor and successor queries on both operands, advancing through the sorted elements in tandem.

Predecessor [5] and successor reduce to rank followed by access. A predecessor query performs one rank computation and one access computation; a successor query does the same. The three operations member, rank, and access are therefore the independent primitives. Any representation that supports these three automatically supports all five, at the cost of one additional access per predecessor or successor query.

5: To compute predecessor(S, x), first compute $r = \text{rank}(S, x)$. If $r = 0$, no predecessor exists. Otherwise, return access(S, r). Successor works symmetrically.

We can now ask how many bits are needed to store the collection $\mathcal{S}$. A natural starting point treats each set independently. A set $S_i \subseteq U$ of size $|S_i|$ is one of $\binom{u}{|S_i|}$ possible subsets of U of that size. Any encoding that can distinguish all such subsets requires at least $\lg \binom{u}{|S_i|}$ bits for S_i. This binomial counting argument establishes the baseline against which every compressed representation in this thesis is measured.

Definition 1.2.3 (Independent encoding cost) *The independent encoding cost (or worst-case entropy) of a set collection* $(\mathcal{S}, U)$ *is*

$$H_{wc}(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{u}{|S_i|}.$$

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

Alanko, Bille, Gørtz, Navarro, and Puglisi call this quantity the worst-case entropy [7], writing it as $H(\mathcal{S})$; we adopt $H_{wc}(\mathcal{S})$ to distinguish it from the finer compressibility measures introduced later. The name reflects that $\lg \binom{u}{|S_i|}$ is the number of bits needed to encode a subset of size $|S_i|$ from a universe of size u, without any assumptions about the structure of the specific set. We work in the word RAM model with word size $\omega \geq \lg u$, so each universe element fits in a single machine word, and space is measured in bits throughout. The bound $H_{wc}(\mathcal{S})$ is achievable. Encoding each set independently costs $H_{wc}(\mathcal{S}) + O(n + m \lg n)$ bits in total, where the $O(m \lg n)$ term accounts for index pointers and the $O(n)$ term for per-set overhead,supporting rank and constant-time access to the i-th element of any set. When the sets in $\mathcal{S}$ are unrelated, no encoding can do substantially better.

The three applications in Section 1.1 show, however, that the sets in a collection are rarely unrelated. Adjacent adjacency lists in a web graph share most of their links. Neighbouring k-mers in a de Bruijn graph have nearly identical color sets. Posting lists for semantically related terms overlap heavily. In each case, $H_{wc}(\mathcal{S})$ treats each set as if it were drawn independently from $\binom{u}{|S_i|}$ possibilities, ignoring the correlations that make the collection far more compressible than the sum of its parts. If $S_i \subset S_j$, we can describe S_i as a subset of S_j in $\lg \binom{|S_j|}{|S_i|}$ bits, which is much less than $\lg \binom{u}{|S_i|}$ when $|S_j| \ll u$ [7]. If S_i and S_j have small symmetric difference, we can describe one in terms of the other cheaply [9]. Either way, the encoding cost drops below $H_{wc}(\mathcal{S})$.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Two questions arise naturally. The first is combinatorial. Which compressibility measure captures the inter-set relationships of a given collection, and how much smaller can the encoding be compared to $H_{wc}(\mathcal{S})$? The second is algorithmic. Can we build data structures that achieve the compressed space and still support member, access, rank, predecessor, and successor queries efficiently? Section 1.3 outlines our answers.

1.3 Contribution and roadmap

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Succinct Dictionaries 2

2.1 Rank and Select 7
2.2 Sparse Bitvectors 15
2.3 Wavelet trees 20

To represent many sets we first need to represent one, and representing one reduces, at its heart, to rank and select on a characteristic bitvector. Section 1.2 left us with two open questions, one combinatorial and one algorithmic, and the five operations of Definition 1.2.2 waiting for a data-structural answer. A set $S \subseteq [1..u]$ is specified by its characteristic bitvector $B[1..u]$, with $B[i] = 1$ if and only if $i \in S$. This representation carries no more information than the set itself, and it exposes the combinatorial structure of S at the level of individual positions. Each operation of Definition 1.2.2, evaluated on S, becomes a question about counts and positions of ones in B. Membership asks whether $B[x] = 1$. Access asks for the position of the i-th one. Rank asks for the number of ones in a prefix. Predecessor and successor combine these. The question of how efficiently we can answer these queries on a bitvector of length n, and with how many bits of auxiliary storage, is the starting point of this chapter and the foundation on which every later structure rests.

A single set $S \subseteq [1..u]$ is a binary-alphabet problem. Its characteristic bitvector turns the five operations of Definition 1.2.2 into questions about counts and positions of ones, answered by the primitives rank and select.

2.1 Rank and Select

Throughout this chapter we work in the word RAM model with word size $\omega = \Omega(\lg n)$, so that a machine word holds a bitvector index and the bit-level primitives below admit constant-time arithmetic, bitwise, and shift operations. All space bounds are measured in bits, and lg denotes $\log_2$. Let $B[1..n]$ be a bitvector of length n. The two primitives we study are defined as follows.

Definition 2.1.1 (Rank) *For $b \in \{0, 1\}$ and $1 \le i \le n$, the operation* $\mathrm{rank}_b(B, i)$ *returns the number of occurrences of b in the prefix $B[1..i]$,*

$$\mathrm{rank}_b(B, i) \;=\; |\{j : 1 \le j \le i,\ B[j] = b\}|.$$

By convention $\mathrm{rank}_b(B, 0) = 0$.

Definition 2.1.2 (Select) *For $b \in \{0, 1\}$ and $j \ge 0$, the operation* $\mathrm{select}_b(B, j)$ *returns the position in B of the j-th occurrence of b, that is, the unique index i such that $B[i] = b$ and* $\mathrm{rank}_b(B, i) = j$. *By convention* $\mathrm{select}_b(B, 0) = 0$, *and* $\mathrm{select}_b(B, j) = n + 1$ *whenever* $j > \mathrm{rank}_b(B, n)$.

The two operations are mutual inverses on their domains. For any i with $B[i] = b$ we have $\mathrm{select}_b(B, \mathrm{rank}_b(B, i)) = i$, and for every valid j we have $\mathrm{rank}_b(B, \mathrm{select}_b(B, j)) = j$. The identity $\mathrm{rank}_0(B, i) = i - \mathrm{rank}_1(B, i)$ reduces rank for the two values of b to a single subtraction, so only rank_1 needs direct support.Select lacks an analogous identity; select_0 admits no constant-time formula in terms of select_1, and a separate auxiliary structure is needed.

Figure 2.1 fixes a concrete bitvector of length $n = 20$ with population count 11. The red prefix gives $\mathrm{rank}_1(B, 15) = 9$, and the blue-circled bit gives $\mathrm{select}_1(B, 7) = 12$.

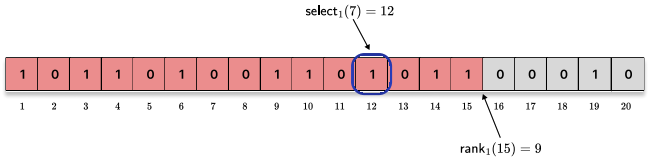

Figure 2.1: A bitvector $B[1..20]$. The prefix $B[1..15]$ (shaded red) contains nine ones, so $\text{rank}_1(B, 15) = 9$. The seventh one (circled blue) lies at position 12, so $\text{select}_1(B, 7) = 12$.

Without auxiliary structures, both queries reduce to a linear scan. Rank scans i bits and counts the ones; select scans from the left, decrementing a counter each time it sees a one, until the counter reaches j. In the worst case, both queries take $\Theta(n)$ time, with no extra storage beyond the n bits of B itself and $O(\lg n)$ bits for a loop counter.

The target $n + o(n)$ bits at $O(1)$ query time is what Jacobson called *succinct* [3, 10]. The term is a quantitative criterion distinguishing data structures that match an information-theoretic lower bound up to a low-order term.

The linear-time solution is acceptable for very short bitvectors, but the set collections we will study in later chapters push n into the billions and demand fast repeated queries. A natural goal is constant query time, and we now ask at what cost in space. Encoding an arbitrary bitvector of length n takes n bits in the worst case, by a trivial counting argument, and B itself already attains that lower bound. The question is how much additional storage, beyond the n bits of B, is needed to support $O(1)$-time rank and select. An auxiliary structure of size $o(n)$ bits, meaning a quantity that grows strictly slower than n, yields a total storage of $n+o(n)$ bits, the bitvector plus a low-order overhead. Jacobson called such a representation *succinct* [10], and this target has guided three decades of subsequent work. The word RAM model assumes $\omega = \Omega(\lg n)$, so a position in B fits in one machine word.The primitives of this section recur throughout the rest of the thesis as black-box building blocks, and every bound depending on them inherits their query time. A linear-time rank would propagate linearly to every higher-level operation, and a logarithmic rank would propagate logarithmically. The $O(1)$-time $o(n)$-bit target is therefore the operating assumption of every structure built above bitvectors. Subsection 2.1.1 through Subsection 2.1.4 show how this target is met, why it is optimal in a precise sense, and what happens when the bitvector is sparse enough that even n bits are wasteful.

[10]: Jacobson (1988), *Succinct Static Data Structures*

2.1.1 Constant-time rank

Jacobson's 1988 PhD thesis and 1989 FOCS paper introduced the term *succinct data structure* [3, 10]. The FOCS 1989 paper [3] gave an $O(\log n)$-time, $o(n)$-bit auxiliary structure for rank and select; the $O(1)$-time rank structure is developed in Jacobson's 1988 PhD thesis [10]. The technique combines a two-level directory with a universal lookup table on short blocks, often called the four-Russians trick.
[10]: Jacobson (1988), *Succinct Static Data Structures*

The first question to settle is whether the linear-time baseline can be improved to $O(1)$ without paying more than $o(n)$ extra bits. Jacobson showed this is possible [10], and the construction has remained the template for every subsequent rank structure. The idea is a two-level directory, combined with a universal lookup table on very short blocks.

Theorem 2.1.1 (Constant-time rank[10]) *For any bitvector $B[1..n]$, there is a data structure of $n + O(n \lg\lg n/\lg n)$ bits that supports $\text{rank}_1(B, i)$ in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim and accessed read-only.*

The construction partitions B into two nested levels, illustrated in Figure 2.2. We choose a *superblock* size $Z = \lfloor \lg^2 n \rfloor$ and a *block* size $z = \lfloor (\lg n)/2 \rfloor$, so that each superblock contains exactly Z/z blocks.For simplicity we assume z divides Z and Z divides n; the general case differs only in the treatment of the final partial superblock, which contributes $O(\lg n)$ bits of overhead.

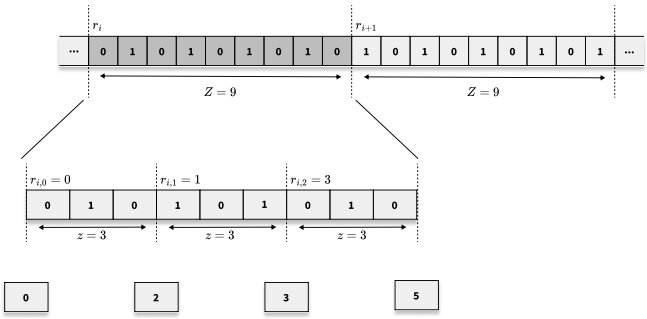

Figure 2.2: The two-level rank directory of Jacobson's construction. Superblocks of size Z store absolute ranks r_i; blocks of size z inside each superblock store ranks $r_{i,j}$ relative to the enclosing superblock. The bottom row represents the four-Russians lookup table, indexed by the z-bit intra-block pattern and an in-block offset, returning the rank inside the pattern.

Two auxiliary arrays store precomputed rank values. The *superblock directory* R_S holds absolute ranks at superblock boundaries. For $k = 0, 1, \ldots, n/Z - 1$ we set

$$R_S[k] \;=\; \mathrm{rank}_1(B, kZ).$$

The *block directory* R_B holds ranks relative to the enclosing superblock. For each block index $\ell = 0, 1, \ldots, n/z - 1$, belonging to superblock $k = \lfloor \ell z / Z \rfloor$, we set

$$R_B[\ell] \;=\; \mathrm{rank}_1(B, \ell z) - R_S[k].$$

A rank query at position i decomposes as the sum of three contributions, namely the superblock prefix rank, the block prefix rank within its superblock, and the rank inside the final block up to the offset $i \bmod z$.

The third contribution, rank within a z-bit block, requires care. A naive table indexed by the z-bit block pattern and the offset would have size $2^z \cdot z \cdot \lg z$ bits. With $z = \lfloor (\lg n)/2 \rfloor$, the factor 2^z becomes $\sqrt{n}$, and the total is $O(\sqrt{n} \cdot \lg n \cdot \lg\lg n)$ bits, which is $o(n)$, and can be made smaller still. The block pattern $B[\ell z + 1..(\ell + 1)z]$ remains in B and is accessed directly through a single word read, since $z \le \omega$. The lookup table only needs to answer, for every z-bit pattern p and every offset $j \in [1..z]$, the precomputed count $\mathrm{rank}_1(p, j)$. The table has $2^z \cdot z$ entries, each fitting in $\lceil \lg(z + 1) \rceil$ bits, giving total size $O(2^z \cdot z \cdot \lg z) = O(\sqrt{n}\ \lg n\ \lg\lg n)$ bits.

Sketch. The superblock directory R_S stores n/Z absolute ranks of $\lceil \lg n \rceil$ bits each, totalling $O(n/\lg n)$ bits. The block directory R_B stores n/z values in $[0, Z]$ at $O(\lg\lg n)$ bits each, totalling $O(n \lg\lg n/\lg n)$ bits. The lookup table has $2^z \cdot z = O(\sqrt{n} \lg n)$ entries of $O(\lg\lg n)$ bits, which is $o(n/\lg n)$. A query reads $R_S[\lfloor i/Z \rfloor]$, $R_B[\lfloor i/z \rfloor]$, the block pattern from B, and one table entry, performing $O(1)$ arithmetic. □

The construction exposes where the $o(n)$ overhead comes from. The block directory R_B contributes the dominant term $\Theta(n \lg\lg n/\lg n)$, because each of $\Theta(n/\lg n)$ blocks carries a directory entry of $\Theta(\lg\lg n)$ bits. The superblock directory is smaller by a $\lg n/\lg\lg n$ factor, since superblock boundaries are sparser. The lookup table is smaller still, contributing only $O(\sqrt{n}\,\mathrm{polylog}\, n)$ bits. The $\lg\lg n/\lg n$ overhead is a direct consequence of the block size $z = \Theta(\lg n)$. A larger z would shrink the block directory and blow up the lookup table, and a smaller z would do the opposite. Subsection 2.1.3 will establish that $\Theta(n \lg\lg n/\lg n)$ is essentially optimal within the class of indexing structures.

The four-Russians technique predates succinct data structures by almost two decades, named after a 1970 paper on Boolean matrix multiplication by Arlazarov, Dinic, Kronrod, and Faradzhev [11]. Jacobson's contribution was to apply the precomputed-table idea to rank.

2.1.2 Constant-time select

Rank and select are mutual inverses, but the two-level directory of Theorem 2.1.1 does not support select directly. The source of the asymmetry is easy to see. The superblock and block directories record ranks at positions that are fixed multiples of Z and z. Given a rank query at position i, the superblock and block indices are $\lfloor i/Z \rfloor$ and $\lfloor i/z \rfloor$, computable from the query with constant-time arithmetic. Given a select query for the j-th one, the position is unknown, and the division that identified the right block for rank is unavailable. Binary-searching R_S to find the superblock containing the j-th one and then binary-searching R_B within that superblock yields a total query time of $O(\lg \lg n)$. Constant time by this route is unavailable.

Clark's 1996 PhD thesis at Waterloo [12] developed the three-level select hierarchy on top of Jacobson's rank machinery. The same construction remains standard in every practical library today.

[12]: Clark (1996), *Compact Pat Trees*

Clark resolved this in his 1996 PhD thesis [12] by partitioning the bitvector according to the number of ones instead of the number of positions. The partition is no longer uniform in position; it is uniform in rank. A *chunk* contains a fixed number K of ones, so its boundaries lie at the positions $\text{select}_1(B, K)$, $\text{select}_1(B, 2K)$, and so on. Locating the j-th one at the top level reduces trivially to computing $\lceil j/K \rceil$. The cost is that chunks now have variable length, and a second idea is needed to handle queries within a chunk. Clark's solution is a three-level hierarchy with a sparse/dense dichotomy at each level.

Theorem 2.1.2 (Constant-time select[12]) *For any bitvector $B[1..n]$, there is a data structure of $n + o(n)$ bits that supports* $\text{select}_1(B, j)$ *in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim and accessed read-only. A symmetric construction on the complemented bitvector $\overline{B}$ supports* select_0 *with the same asymptotic overhead, doubling the constant.*

The three-level partition is described here for select_1; select_0 is handled by replicating the construction on $\overline{B}$.Let $m = \text{rank}_1(B, n)$ be the number of ones in B, and set the top-level block size to $K = \lfloor \lg^2 n \rfloor$.

With $K = \lfloor \lg^2 n \rfloor$, a sparse chunk has length at least $K^2 = \lg^4 n$, while a dense chunk fits in a polylogarithmic window. The threshold balances explicit storage against the recursive layer applied within dense chunks.

At the top level, we divide the positions of ones into groups of K. The chunk starts are recorded in an array P_1 of size $\lceil m/K \rceil$, with

$$P_1[c] \;=\; \text{select}_1(B, cK + 1) \qquad \text{for } c = 0, 1, \ldots, \lceil m/K \rceil - 1.$$

Each entry of P_1 fits in $\lceil \lg n \rceil$ bits, so the total space for P_1 is $O\left(\frac{m}{K} \cdot \lg n\right) = O(n/\lg n) = o(n)$ bits, using $m \le n$. Given a query $\text{select}_1(B, j)$, the chunk index is $c = \lfloor (j-1)/K \rfloor$ and the local rank within the chunk is $j' = ((j-1) \bmod K) + 1$, both computed in $O(1)$ time.

A chunk covers positions $P_1[c]$ through $P_1[c+1] - 1$ of B, so its length $Z_c = P_1[c+1] - P_1[c]$ varies with c. The construction splits into two cases by density. A chunk is *sparse* when $Z_c \ge K^2 = \lg^4 n$, and *dense* otherwise. For a sparse chunk, the positions of its K ones are stored explicitly as an array of K offsets relative to $P_1[c]$, each fitting in $\lceil \lg Z_c \rceil = O(\lg n)$ bits. A sparse chunk has length at least K^2, so disjoint sparse chunks are substrings of B of total length at most n, giving at most n/K^2 of them. The total sparse-offset space is therefore

$$\frac{n}{K^2} \cdot K \cdot O(\lg n) \;=\; \frac{n \lg n}{K} \;=\; O\left(\frac{n}{\lg n}\right) \text{ bits,}$$

which is $o(n)$. Within a dense chunk, explicit storage is too expensive, and we recurse.

Each dense chunk is sub-divided by number of ones, with sub-chunk size $k = \lceil \sqrt{\lg n} \rceil$. Inside dense chunk c, we record the positions of the k-th, $2k$-th, $\ldots$, K-th ones, measured as offsets relative to $P_1[c]$; there are K/k such offsets. Each fits in $\lceil \lg Z_c \rceil = O(\lg \lg n)$ bits, since $Z_c < K^2 = \lg^4 n$ implies $\lg Z_c = O(\lg \lg n)$. Summed over all dense chunks, the space is

$$\sum_{\text{dense } c} \frac{K}{k} \cdot O(\lg \lg n) \;=\; O\left(\frac{n}{\lg n} \cdot \frac{\lg \lg n}{\sqrt{\lg n}} \right),$$

which is $o(n)$. A sub-chunk spans K/k consecutive ones, so its length z_c is variable. The same sparse/dense dichotomy applies at this level. A sub-chunk is sparse when $z_c \geq k^4$ and dense otherwise. Sparse sub-chunks store the positions of their ones explicitly, giving $o(n)$ bits by the same analysis.

Dense sub-chunks satisfy $z_c < k^4 = O(\lg^2 n)$, still comparable to a machine word. The table is indexed by every possible z_c-bit pattern and every rank i'' in $[1..K/k]$, and returns the position of the i''-th one within the pattern. The number of patterns and the number of local rank values both grow polynomially in $\lg n$, so the table size stays $o(n)$. This is the four-Russians technique of Jacobson [10] for rank, extended to select by Clark [12].

A dense sub-chunk has length polynomial in $\sqrt{\lg n}$, asymptotically smaller than $\lg n$, so its bit pattern fits in a fraction of a machine word and can be resolved by a precomputed table.

[10]: Jacobson (1988), *Succinct Static Data Structures*
[12]: Clark (1996), *Compact Pat Trees*

The three levels assemble into a constant-time query. First the chunk index c and local rank j' are computed, and $P_1[c]$ gives the chunk start. For a sparse chunk, the answer is $P_1[c]$ plus the j'-th stored offset. For a dense chunk, the sub-chunk index $\ell = \lfloor (j'-1)/k \rfloor$ and the local rank $j'' = ((j'-1) \bmod k) + 1$ give the sub-chunk start relative to $P_1[c]$ through the sub-chunk offset table. A sparse sub-chunk returns the precomputed offset of the j''-th one; a dense sub-chunk looks up the j''-th one in the dense-pattern table. Each level's branch choice is a single comparison, and the query reads $O(1)$ words.

Sketch. Each storage component is $o(n)$ bits. The top-level array P_1 costs $O(n/\lg n)$; the sparse-chunk offsets cost $O(n/\lg n)$; the dense-chunk recursive offsets and the bottom-level lookup table also contribute $o(n)$. The query performs $O(1)$ probes at each of the three levels. □

The same construction, applied separately to $\overline{B}$, handles select_0.Storing both structures doubles the auxiliary space while keeping it $o(n)$. Combined with the rank structure of Subsection 2.1.1, we obtain a bitvector representation that supports rank_b and select_b for $b \in \{0, 1\}$ in $O(1)$ time, with auxiliary space $o(n)$ bits on top of the verbatim bitvector.

For very small dense sub-chunks, a direct broadword scan using hardware population count may outperform the lookup table on modern processors. The theoretical analysis does not rely on hardware popcount.

2.1.3 Lower bounds

Jacobson and Clark achieve $n + o(n)$ bits for constant-time rank and select, with $\Theta(n \lg \lg n / \lg n)$ for rank. A natural question asks whether these bounds are tight. Can the overhead be pushed below $\Theta(n \lg \lg n / \lg n)$ while keeping $O(1)$ query time, by any clever $O(1)$-time algorithm, no matter how small its index?

Miltersen's indexing model [13] stores the input verbatim and allows only an auxiliary index. The restriction is natural for bitvectors, because the n bits of B already meet the information-theoretic minimum for arbitrary bitvectors, and the only freedom is in how much additional structure the query algorithm may consult.

[13]: Miltersen (2005), *Lower Bounds on the Size of Selection and Rank Indexes*

The setting in which this question has a clean answer is the *indexing* model, introduced by Miltersen [13].

Definition 2.1.3 (Indexing data structure) *An indexing data structure for a problem on inputs $x \in \{0,1\}^n$ consists of the input x stored verbatim in $\lceil n/\omega \rceil$ words, together with an r-bit index $\phi(x)$ stored in $\lceil r/\omega \rceil$ words.*

The query algorithm against such a storage scheme is an adaptive procedure that probes bits of x and $\phi(x)$ and returns an answer; it has query time t if it makes at most t bit-probes in the worst case.The indexing model constrains the representation to store the bitvector B untouched while allowing an extra index. This matches Theorem 2.1.1 and Theorem 2.1.2, where B is accessed read-only and the auxiliary structures R_S, R_B, P_1, and the lookup tables form the index. The redundancy r in this model is precisely the size of the index. Miltersen's 2005 paper gives the following lower bound.

Theorem 2.1.3 (Index size lower bound[13]) *For any indexing data structure supporting* select_1 *on a bitvector of length n in the cell-probe model with word size ω, with r bits of index and query time t, it holds that*

$$3(r+2)(t\omega+1) \;\geq\; n.$$

For rank_1, *it holds that*

$$2(2r+\lg(\omega+1))t\omega \;\geq\; n\lg(\omega+1).$$

In the regime $\omega = \Theta(\lg n)$ and $t = O(1)$, the select bound becomes $r = \Omega(n/\lg n)$ and the rank bound becomes $r = \Omega(n \lg\lg n/\lg n)$. The rank bound matches Theorem 2.1.1 up to a constant factor. The select bound falls short of Theorem 2.1.2 by a factor of $\lg\lg n$. Both are proved by counting arguments. We sketch the select argument, since the strategy is short and the technique recurs.

Sketch. Reduce to $\omega = 1$. Fix inputs $x \in \{0,1\}^n$ of Hamming weight $m = \lfloor (n/3-1)/t \rfloor$. Run the m select queries on x, collecting the mt probed bits into a string $\tau(x)$ after zeroing bits that belong to the index $\phi(x)$ or were already seen. Together, $\phi(x)$ and $\tau(x)$ suffice to reconstruct x by simulation. Counting pairs yields $2^r(mt+1)\binom{mt}{m} \geq \binom{n}{m}$. Since $m \leq n/3$, each ratio factor $(n-i)/(mt-i) \geq 2$, so $\binom{n}{m}/\binom{mt}{m} \geq 2^m$, forcing $r+1 \geq m$. The theorem statement follows from the $\omega = 1$ case by the standard bit-probe to cell-probe conversion. □

Golynski's 2006 argument [14] builds a *choices tree* whose leaves correspond to joint outcomes of probes across multiple queries, and bounds the number of bitvectors consistent with any leaf via a Stirling-based density estimate. The analysis closes the $\lg\lg n$ gap in Miltersen's select bound.

[14]: Golynski (2006), *Optimal Lower Bounds for Rank and Select Indexes*

The matching lower bound for select was established by Golynski [14].

Theorem 2.1.4 (Tight select index bound[14]) *Any indexing data structure for* rank_1 *or* select_1 *on a bitvector of length n, with $O(\lg n)$ bit-probes per query to the bitvector and unlimited probes to the index, has index size $r = \Omega(n \lg\lg n/\lg n)$.*

Once the index is fixed, each query becomes a deterministic probe sequence, so different inputs trace different paths from root to leaf of the choices tree. Counting leaves and bounding the number of bitvectors compatible with each leaf yields the lower bound. The argument fixes

the index and considers all possible probe outcomes across p queries on blocks of size $k = t + \lg n$.

$$\frac{C(x)}{2^{U(x)}} \leq \left(\frac{c}{\sqrt{\lg n}}\right)^{ap}$$

for some constants $a, c > 0$, where $U(x)$ is the number of unprobed bits. The same technique extends to select (§3.2) with the same bound. Summing over all 2^{n+r} leaves, and requiring that every bitvector of length n be compatible with at least one leaf, forces $r = \Omega(n \lg\lg n/\lg n)$. The Stirling calculation is in Golynski's technical report [14], Section 3.1.

[14]: Golynski (2006), *Optimal Lower Bounds for Rank and Select Indexes*

Golynski's paper also gives a density-sensitive lower bound. If the bitvector has m ones and the query algorithm uses t bit-probes to the bitvector, the index must have size at least $(m/t)\lg t$ bits. For $m = \Theta(n)$ and $t = O(\lg n)$, this recovers $\Omega(n \lg\lg n/\lg n)$, and for sparser bitvectors it points toward compressed representations studied in Subsection 2.1.4. Beame and Fich [15] established tight cell-probe bounds for the predecessor problem. Pătraşcu observes that predecessor on a bitvector-encoded set reduces to $\text{select}_1 \circ \text{rank}_1$,so the select lower bounds above indirectly constrain predecessor-supporting structures built from rank and select.

The $(m/t)\lg t$ bound is density-sensitive. For very sparse bitvectors, where $m \ll n$, the overhead can be reduced at the cost of increasing the query time. For dense bitvectors, the bound degenerates into the $\Omega(n \lg\lg n/\lg n)$ result.

[15]: Beame et al. (2002), *Optimal Bounds for the Predecessor Problem and Related Problems*

The lower bound of Theorem 2.1.4 applies to indexing data structures, where B itself is stored untouched. Pătraşcu [16] showed that this restriction is essential. If the bitvector is re-encoded, the redundancy can be pushed far below $\Theta(n \lg\lg n/\lg n)$, at the cost of an exponential trade-off with query time. A related line of work by Grossi, Orlandi, Raman, and Srinivasa Rao [17] studied lowering the redundancy of fully indexable dictionaries through more haste, less waste techniques.

Pătraşcu's 2008 FOCS paper *Succincter* [16] broke the redundancy barrier by combining *spill-over representations*, which pass fractional bits of entropy between layers, with aB-trees, a generalization of the augmented search tree of Jacobson. The construction yields redundancy $O(n/\text{polylog}\, n)$ at the cost of only a constant factor in query time.

[16]: Pătraşcu (2008), *Succincter*

[17]: Grossi et al. (2009), *More Haste, Less Waste: Lowering the Redundancy in Fully Indexable Dictionaries*

Theorem 2.1.5 (Succincter[16]) *On a word RAM with cells of* $\Omega(\lg u)$ *bits, a bitvector of length* u *with* n *ones can be stored in*

$$\lg\binom{u}{n} + \frac{u}{((\lg u)/t)^t} + \widetilde{O}(u^{3/4}) \quad \textit{bits},$$

supporting rank_1 *and* select_1 *queries in* $O(t)$ *time, for any integer* $t \geq 1$.

Setting $t = 2$ in Theorem 2.1.5 gives redundancy $O(u/(\lg u)^2)$, strictly smaller than the Jacobson-Clark bound.

The core idea, developed in Pătraşcu's Section 2 and applied to aB-trees in Section 4.1, is the *spill-over representation*. A value from a set X is stored as M memory bits together with an auxiliary *spill* value in $\{0, 1, \ldots, K-1\}$, where $K \cdot 2^M \geq |X|$, in place of the $\lceil \lg |X| \rceil$ memory bits of a plain encoding.Lemma 3 of Succincter chooses parameters so that $r \leq K \leq 2r$ for a target r, and shows the redundancy of a single level is at most $2/r$ bits. Stacking these representations in t levels of an aB-tree yields redundancy $u/((\lg u)/t)^t$, exponential in t rather than linear. The query time is $O(t)$, because navigating the aB-tree touches t levels. Pătraşcu's Section 4.1 gives the detailed reduction from rank and select to aB-trees, and the spill-over composition is in Section 2. Theorem 2.1.5 shows that the Jacobson-Clark bound is optimal only within the indexing model, and that dropping the verbatim-storage restriction yields exponential savings. The aB-tree template implicitly present in Jacobson's and Clark's constructions can be extracted, abstracted, and composed

A spill-over representation stores M memory bits plus a spill value in $\{0, \ldots, K-1\}$. The spill carries fractional bits of entropy that would otherwise be wasted. Pătraşcu's key step is recursive composition of two spill-over representations, giving exponential redundancy reduction where the naive composition would give only a linear trade-off.

recursively; Pătraşcu's generic transformation from augmented search trees to succinct data structures applies to rank and select as one instance among several.

Practical implementations still overwhelmingly use the Jacobson-Clark scheme, because the indexing structure admits very fast word-level operations and simple code, while spill-over representations require more elaborate arithmetic.

2.1.4 Compressed bitvectors

Every structure so far pays n bits for the bitvector of length n, regardless of how many ones it contains. For bitvectors with few ones, this is wasteful. A set of n elements drawn from a universe of size u is one of $\binom{u}{n}$ possibilities, so the information-theoretic cost of encoding it is

$$\mathcal{B}(u, n) \;=\; \left\lceil \lg \binom{u}{n} \right\rceil \;=\; n \lg(u/n) + O(n)$$

bits for $n \le u/2$,which is much smaller than u when $n \ll u$. Can we represent the bitvector in $\mathcal{B}(u, n) + o(u)$ bits while still supporting $O(1)$ rank and select? A sequence of papers starting with Raman and Rao [18] and culminating in Raman, Raman, and Rao [19] answers yes.

[18]: Raman et al. (1999), *Static Dictionaries Supporting Rank*

[19]: Raman et al. (2007), *Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets*

The 1999 ISAAC paper of Raman and Rao [18] established the template. Their structure combines the classical FKS perfect-hashing dictionary [20] with a *quotient* representation to achieve the following.

Raman and Rao's 1999 ISAAC paper [18] was the first to combine the classical FKS dictionary [20] with quotient information to break the $n\lceil \lg u \rceil$-bit barrier while keeping $O(1)$ rank. Its per-bucket rank trick foreshadows the full RRR construction.

[18]: Raman et al. (1999), *Static Dictionaries Supporting Rank*

[20]: Fredman et al. (1984), *Storing a Sparse Table with $O(1)$ Worst Case Access Time*

Theorem 2.1.6 (Quotient dictionary with $O(1)$ rank[18]) *A subset $S \subseteq [1..u]$ of size n can be stored in $n \lg u + O(\lg \lg u) - \Theta(n)$ bits so that membership and rank queries on elements of S are answered in $O(1)$ time.*

The bound in Theorem 2.1.6 consolidates Theorem 1 (p. 21, space $n \lg u + O(n + \lg \lg u)$) and Corollary 1 (p. 22) of Raman and Rao 1999, where an additional layer of block-wise rank storage shaves the $\Theta(n)$ term. The construction stores each element of S through a composite hash, keeping only a *quotient* that distinguishes it from other elements mapped to the same bucket. Combined with k^{-1} and κ^{-1} inverses of the hash parameters, the quotient together with the bucket index and compressed tables recovers the original element in constant time.A per-bucket rank structure, implemented as a unary bitmap with the standard $o(n)$-bit rank auxiliary structures, gives rank on elements of S in $O(1)$ time. The space bound $n \lg u + O(\lg \lg u) - \Theta(n)$ still exceeds $\mathcal{B}(u, n)$ by $\Theta(n)$ bits in general, because FKS does not by itself achieve information-theoretic optimality. The bound is the first to break the $n\lceil \lg u \rceil$-bit barrier while keeping $O(1)$ rank on members, and Raman and Rao leave open whether the $\Theta(n)$ deficit can be closed.

The question was settled by the 2002 SODA paper and 2007 TALG journal version of Raman, Raman, and Rao [19]. Their construction, known as the RRR structure, achieves space within a lower-order term of the information-theoretic minimum while preserving $O(1)$ rank and select. We describe the class/offset encoding, state the main theorem, and trace the query algorithm.

[19]: Raman et al. (2007), *Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets*

The bitvector $B[1..n]$ is partitioned into $\lceil n/b \rceil$ blocks of length $b = \lfloor (\lg n)/2 \rfloor$, matching the block size used in Jacobson's rank structure of Subsection 2.1.1 to balance the block directory against the lookup table. Each block is replaced by a (class, offset) pair in place of its verbatim content.For each block $B_j = B[(j-1)b+1..jb]$, define its *class* c_j as the population count,

$$c_j = \sum_{i=(j-1)b+1}^{jb} B[i] \in \{0, 1, \ldots, b\},$$

and its *offset* o_j as the rank of the block's pattern within the lexicographic enumeration of the $\binom{b}{c_j}$ b-bit patterns of population count c_j. The pair (c_j, o_j) uniquely determines B_j, and inversely B_j determines (c_j, o_j). The class fits in $\lceil \lg(b+1) \rceil$ bits; the offset fits in $\lceil \lg \binom{b}{c_j} \rceil$ bits, which depends on the class.

Theorem 2.1.7 (Indexable dictionary (RRR)[19]) *A bitvector $B[1..n]$ with m ones can be stored in*

$$\mathcal{B}(n, m) + o(n) + O(\lg \lg n) \quad \textit{bits},$$

where $\mathcal{B}(n, m) = \lceil \lg \binom{n}{m} \rceil$, so that $\mathrm{rank}_b(B, i)$ *and* $\mathrm{select}_b(B, i)$ *for $b \in \{0, 1\}$ are answered in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.*

The proof follows the structure of Section 4 of the 2007 journal version.

Sketch. Partition B into $p = \lceil n/b \rceil$ blocks of length $b = \lfloor (\lg n)/2 \rfloor$, encode each block by its (c_j, o_j) pair, and store the class array C plus the variable-length offset stream. For the $p-1$ full blocks of length b and the boundary block of length at most b, the Vandermonde-type identity controlled by $pb \geq n$ bounds the offset stream by $\mathcal{B}(n, m) + O(n/\lg n)$ bits.Variable-length offsets are accessed through the searchable prefix-sum structure of Theorem 2.1.6 (Lemma 2.3 of RRR 2007) applied to the bit-length array; this costs $o(n)$ bits.Lookup tables for rank and select within a block contribute $o(n)$ bits. Rank assembles a superblock prefix, the class-sequence prefix sum retrieved via the searchable prefix-sum structure, and the within-block lookup.Select uses a segment structure partitioning the universe into segments each containing $v = \lfloor (\lg p)^2 \rfloor$ ones, with a dense/sparse dichotomy at the segment level.Section 4.2 reduces the lower-order term by MSB bucketing, grouping keys by a $\Theta(\sqrt{\lg n})$-bit prefix into $\Theta(n\sqrt{\lg n})$ buckets and representing bucket boundaries compactly;the final bound is $\mathcal{B}(n, m) + o(n) + O(\lg \lg n)$ bits with $O(1)$ rank and select. □

The quantity $\mathcal{B}(n, m)$ connects directly to Definition 1.2.3.

2.2 Sparse Bitvectors

Every structure of Section 2.1 pays at least n bits for a bitvector of length n, regardless of how many ones it contains. When the number of ones m is much smaller than n, this is wasteful. A set of m elements drawn from

a universe of size n is one of $\binom{n}{m}$ possibilities, so the information-theoretic minimum is

$$\mathcal{B}(n,m) \;=\; \lceil \lg \tbinom{n}{m} \rceil \;=\; m \lg(n/m) + O(m) \quad \text{bits},$$

a quantity much smaller than n when $m \ll n$. Theorem 2.1.7 already reaches $\mathcal{B}(n,m)+o(n)+O(\lg\lg n)$ bits with constant-time rank and select, through class/offset encoding over blocks of length $(\lg n)/2$. Sparsity admits a structurally simpler alternative. A sparse representation stores the positions of the m ones directly, in the form of a monotone integer sequence, and exploits the monotonicity to split each position into a cheap high part and an explicit low part. The simpler mechanics trade a slower rank or access for a faster select, and for very sparse inputs yield smaller constants and simpler code than RRR.

Elias introduced a sparse integer-set representation [21], working in parallel with an independent unpublished MIT memorandum of Fano [22]. The construction now carries both names. Neither original formulation gives a word-RAM analysis; a rigorous constant-time treatment appears with Grossi and Vitter [23], whose Lemma 2 we state and prove below.

Sparse representations and RRR meet at the same information-theoretic bound, but they distribute the cost differently across operations, and the regime $m \ll n$ determines which is preferable in practice.

2.2.1 Elias-Fano

Let $B[1..n]$ be a bitvector with exactly m ones, and let $x_1 < x_2 < \cdots < x_m$ be the positions of its ones, so each $x_h \in [1..n]$.The sequence $(x_1, \ldots, x_m)$ is a monotone sequence of integers in a universe of size n, and the bitvector and the sequence are equivalent representations of the same subset. RRR compresses B at the block level, without acknowledging monotonicity. Elias-Fano exploits monotonicity directly. Split each x_h into a low part and a high part,

$$x_h \;=\; q_h \cdot 2^{w-z} + r_h,$$

where $z = \lfloor \lg m \rfloor$, $w = \lceil \lg n \rceil$, the high part q_h occupies the top z bits of x_h, and the low part r_h occupies the remaining $w - z$ bits.The low parts $r_1, r_2, \ldots, r_m$ are stored verbatim as a fixed-width array L, one $(w-z)$-bit field per element. The high parts are stored in a bitvector H obtained by unary-coding the differences of consecutive quotients,

$$H \;=\; 0^{q_1}\,1\,0^{q_2-q_1}\,1 \cdots 0^{q_m-q_{m-1}}\,1.$$

The ones in H mark the m quotients, and the zeros interpolate the gaps between successive quotient values. Since $q_h \in [0, 2^z)$ and $2^z \le m$, there are at most m zeros and exactly m ones, so $|H| \le 2m$.

Figure 2.3 illustrates the encoding on a concrete example. The positions $5, 7, 8, 15, 32$ form a strictly increasing sequence in $[0, 36)$, so $n = 36$, $m = 5$, $w = 6$, and $z = \lfloor \lg 5 \rfloor = 2$. The low parts are the bottom $w - z = 4$ bits of each position and form the array L; the top $z = 2$ bits of each position form the quotient sequence $(0, 0, 0, 0, 2)$, unary-coded into H.

[23]: Grossi et al. (2005), *Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching*

Grossi and Vitter [23] give the first rigorous word-RAM analysis.

Theorem 2.2.1 (Elias-Fano encoding[23]) *Given m integers in sorted order, each containing w bits, with $m < 2^w$, there is a representation of*

$$m\,\big(2 + w - \lfloor \lg m \rfloor\big) \;+\; O\big(m/\lg\lg m\big) \quad \textit{bits}$$

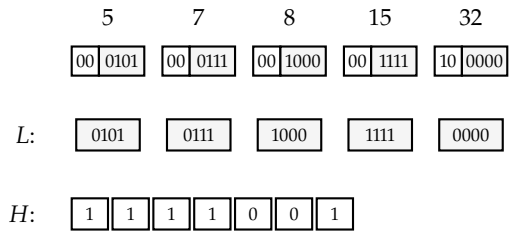

Figure 2.3: Elias-Fano encoding of the set $\{5, 7, 8, 15, 32\}$ with universe $n = 36$ and $m = 5$, so $w = 6$ and $z = \lfloor \lg 5 \rfloor = 2$. Each element is split into a high part of z bits and a low part of $w - z = 4$ bits. The array L is the concatenation of the low parts in order; the bitvector H unary-codes the successive increments of the high parts $(0, 0, 0, 0, 2)$. To answer $\text{select}_1(j)$, locate the j-th one in H at position $p = \text{select}_1(H, j)$, recover the high part $q_j = p - j$, and concatenate it with the low part $L[j]$.

that retrieves the h-th integer in $O(1)$ time on the word RAM with word size $\omega = \Omega(w)$. For the characteristic bitvector of a subset $S \subseteq [1..n]$ of size m, with $w = \lceil \lg n \rceil$, the bound is $m(\lceil \lg n \rceil - \lfloor \lg m \rfloor) + 2m + O(m/\lg \lg m) \leq m\lceil \lg(n/m) \rceil + 3m + O(m/\lg \lg m)$ bits, and the h-th retrieval answers $\text{select}_1(B, h)$.

Proof. We follow the construction of Grossi and Vitter [23]. Take $z = \lfloor \lg m \rfloor$ and split each x_h into the quotient $q_h \in [0, 2^z)$ and the remainder $r_h \in [0, 2^{w-z})$, as above. Encode L as the concatenation $r_1 r_2 \cdots r_m$ of fixed-width $(w - z)$-bit fields, totalling $|L| = m(w - z)$ bits. Encode H as the unary concatenation of the differences $q_1, q_2 - q_1, \ldots, q_m - q_{m-1}$, which has m ones (one per integer) and at most $2^z - 1 \leq m$ zeros (one per occupied cell of the quotient universe), so $|H| \leq 2m$ bits.

[23]: Grossi et al. (2005), *Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching*

Space. The two arrays together occupy $|L| + |H| \leq m(w - z) + 2m = m(2 + w - \lfloor \lg m \rfloor)$ bits. To support select_1 on H in constant time, we augment H with a select structure of $O(|H|/\lg \lg |H|) = O(m/\lg \lg m)$ bits, an overhead strictly smaller than the $O(|H| \lg \lg |H|/\lg |H|)$ budget of Theorem 2.1.2.The total space is $m(2 + w - \lfloor \lg m \rfloor) + O(m/\lg \lg m)$ bits.

Retrieval. To recover x_h, we must compute its quotient q_h and its remainder r_h in constant time. The quotient is stored implicitly in H through the position of its h-th one. Indeed, the h-th one of H lies at position $q_h + h$, because the unary code for the differences places the h-th one after exactly q_h zeros cumulatively. Therefore

$$q_h = \text{select}_1(H, h) - h,$$

computable in $O(1)$ time by the auxiliary select structure. The remainder is read directly from L through the fixed-width access $r_h = L[h]$. The answer is $x_h = q_h \cdot 2^{w-z} + r_h$, a constant-time composition of two word-level arithmetic operations. □

The space bound of Theorem 2.2.1 consolidates the two intuitions behind Elias-Fano. The remainder array L costs $m(w - z)$ bits, within an additive m-bit slack of $m\lceil \lg(n/m) \rceil$. The high-part bitvector H costs at most $2m$ bits, the classical constant-overhead observation of Elias and Fano; a monotone sequence of m integers in $[0, 2^z)$ with $2^z \leq m$ is cheaply encoded in unary, because the combined zeros-and-ones budget is bounded by the count of ones plus the span of the quotient range. The leading term $m \lg(n/m) + O(m)$ approaches $\mathcal{B}(n, m) = \lg \binom{n}{m}$ within a linear additive term, since $\lg \binom{n}{m} = m \lg(n/m) + m \lg e + O(m^2/n)$ for $m \ll n$.

The tighter $O(m/\lg \lg m)$ select overhead on H is smaller than the generic $\Theta(m \lg \lg m/\lg m)$ of Theorem 2.1.2; Grossi and Vitter invoke it through references [15, 42, 60] of their Lemma 2 without further elaboration.

The argument that at most m bits flip in H between any consecutive pair of integers is the same counting principle behind gap encoding in inverted indexes [8]. Elias-Fano commits the low bits to a fixed-width format and the high bits to unary, which always costs $2m$ bits regardless of gap distribution, decoupling compression from statistical assumptions about gap skew.

The retrieval algorithm in the proof of Theorem 2.2.1 answers select_1, namely the position of the h-th one of B. A single word-RAM call to Theorem 2.1.2, applied to H, reduces the query on B to an $O(1)$ composition with a fixed-width lookup in L. Rank is not symmetric. Given a position $i \in [1..n]$, computing $\text{rank}_1(B, i)$ requires identifying the largest h such that $x_h \leq i$, a binary search over the sorted sequence. The standard implementation binary-searches $\text{select}_1(B, \cdot)$, trading $O(\lg m)$ time for $O(1)$ space overhead.Access reduces to rank by $\text{access}(B, i) = [\text{rank}_1(B, i) > \text{rank}_1(B, i-1)]$, inheriting the same $O(\lg m)$ cost. This is the operational asymmetry that distinguishes Elias-Fano from Theorem 2.1.7. The latter buys $O(1)$ rank and select simultaneously, at the cost of a two-level segment structure and variable-length offsets. Elias-Fano buys $O(1)$ select at the cost of logarithmic rank and access.

2.2.2 Rank and access

[24]: Okanohara et al. (2007), *Practical entropy-compressed rank/select dictionary*

Theorem 2.2.1 supports select_1 in $O(1)$ time but leaves rank_1 and access at $\Theta(\lg m)$, the cost of a binary search over $\text{select}_1(B, \cdot)$. Okanohara and Sadakane [24] lower these costs to $O(1 + \lg(n/m))$ at the same asymptotic space, by coupling Theorem 2.2.1 with a second-layer select structure on the high-part bitvector H. The bit profile of H, with m ones and at most m zeros (total length $\leq 2m$), admits efficient select_0.

Theorem 2.2.2 (sdarray[24]) *A bitvector $B[1..n]$ with m ones can be represented in*

$$m\lceil \lg(n/m) \rceil \;+\; 2m \;+\; o(m) \quad \textit{bits,}$$

supporting $\text{select}_1(B, j)$ *in* $O(1)$ *time,* $\text{rank}_1(B, i)$ *and* $\text{access}(B, i)$ *in* $O(1 + \lg(n/m))$ *time, and* $\text{select}_0(B, j)$ *in* $O(\lg n)$ *time, on the word RAM with word size* $\omega = \Omega(\lg n)$.

Vigna's 2013 paper [25] engineers Elias-Fano for large-scale inverted indexes in MG4J. Two additions to the base structure matter in practice. Forward pointers track the position reached after kq unary reads, giving average constant-time sequential access. Skip pointers installed on the negated unary code support *skip queries* (select_1 to a value rather than to a rank), in average constant time under a linear lower bound on the number of zeros.

The representation is a direct extension of Theorem 2.2.1. The remainder array L and the high-part bitvector H are stored as before, at a combined cost of $m\lceil \lg(n/m) \rceil + 2m$ bits. select_1 on B is inherited unchanged, since the identity $\text{select}_1(B, j) = (\text{select}_1(H, j) - j) \cdot 2^{w-z} + L[j]$ reduces the query to one select on H and one fixed-width read. The novel component is select_0 on H, needed to support rank and access on B.

The sarray structure uses darray, the dense-set counterpart of Okanohara and Sadakane, as a subroutine on the high-part bitvector H. darray is stated in [24] for a bitvector of length N with parameters $L = O(\lg^2 N)$, $L_2 = O(\lg^4 N)$, $L_3 = O(\lg N)$; applied to H of length at most $2m$, the parameters become $L = O(\lg^2 m)$, $L_2 = O(\lg^4 m)$, $L_3 = O(\lg m)$.Partition the m ones of H into blocks of L ones each. A block whose length exceeds L_2 stores the positions of its ones explicitly; a shorter block records only every L_3-th position and resolves the remainder by sequential scan. The total auxiliary space is $o(m)$ bits, and select_0 on H runs in $O(\lg^4 m / \lg n)$ time in the worst case, which is $O(1)$ for any realistic n, m under the word-RAM assumption $\omega = \Omega(\lg n)$.

rank_1 and access on B then reduce to select_0 on H together with a local search on L. Given a query position $i \in [1..n]$, let $q = \lfloor i/2^{w-z} \rfloor$ and $r = i \bmod 2^{w-z}$. The number of ones of B with quotient strictly less than q equals the number of ones of H strictly before its q-th zero, retrievable as $\text{select}_0(H, q) - q$. The remaining ones, those with quotient exactly q and

remainder in $[0, r]$, form a contiguous range of L of at most $2^{w-z} \leq n/m$ elements, resolved by binary search on the $\lg(n/m)$-bit remainders in time $O(\lg(n/m))$.The total cost of rank is $O(1 + \lg(n/m))$, and access follows the same route. The last operation, select_0 on B, is supported by binary search on select_1, running in $O(\lg n)$ time.

Theorem 2.2.2 and Theorem 2.1.7 meet at the same information-theoretic bound $\mathcal{B}(n, m) = \lg \binom{n}{m}$, and distribute the cost differently across operations. Theorem 2.1.7 achieves $\mathcal{B}(n, m) + o(n) + O(\lg \lg n)$ bits with $O(1)$ rank and select for both values, a uniform guarantee across all densities.Theorem 2.2.2 achieves $m\lceil \lg(n/m) \rceil + 2m + o(m)$ bits with $O(1)$ select_1, $O(1 + \lg(n/m))$ rank and access, and $O(\lg n)$ select_0.

The difference lies in the layout. RRR carries the machinery of class/offset encoding over every block of the bitvector regardless of density, with variable-length offsets and a searchable prefix-sum structure. sdarray avoids this machinery by committing to a single global split of each position into high and low parts, and recovers the operations from two select structures on H. When $m \ll n$, the constants hidden in sdarray's $o(m)$ are small, the leading term $m\lceil \lg(n/m) \rceil + 2m$ is within $O(m)$ bits of the RRR bound, and the $\lg(n/m)$ factor in rank is small. Neither representation dominates the other. The density m/n and the operation mix determine which is preferable.

The Elias-Fano $2m$-bit overhead on H is tight against the gap distribution only up to a constant. When the gaps between consecutive ones are highly clustered, H contains large runs of zeros that encode redundant information, and the gap distribution itself admits coding well below $2m$ bits. Sadakane and Grossi [26] closed this gap with *entropy-bound indexable dictionaries*, a construction that achieves the following bound.

Each gap $g = s_i - s_{i-1}$ needs $1 + \lfloor \lg g \rfloor$ bits of binary representation. When the gap distribution concentrates on small values, $\text{gap}(\hat{S})$ falls well below the $2m$ bits spent by Elias-Fano on the unary H, and the Sadakane-Grossi bound beats Elias-Fano by more than a constant factor.

[26]: Sadakane et al. (2006), *Squeezing Succinct Data Structures into Entropy Bounds*

$$\min\{nH_k,\ \text{gap}(\hat{S})\} \ + \ O(n \lg \lg n / \lg n) \quad \text{bits},$$

Here H_k is the k-th order empirical entropy of B, and $\text{gap}(\hat{S}) = \sum_{i=1}^{m}(1 + \lfloor \lg(s_i - s_{i-1}) \rfloor)$ sums the minimum binary lengths of the gaps between consecutive ones, with $s_0 = 0$.

The construction is structurally different from sdarray. Sadakane and Grossi replace the verbatim bitvector B with a compressed representation from which any $O(\lg n)$-bit substring decodes in constant time, and build rank/select on top of the compressed form. The nH_k argument of the min is tight on dense inputs with symbol-level regularity; the $\text{gap}(\hat{S})$ argument is tight on sparse inputs with clustered gaps. The min adapts to whichever term is smaller on the given input, without committing at encoding time.

Like Theorem 2.1.5, the Sadakane-Grossi construction steps outside the indexing model of Subsection 2.1.3 and avoids the lower bounds of Theorem 2.1.3 and Theorem 2.1.4.

The bound is never worse than the RRR or sdarray space, and on clustered inputs it is strictly smaller by a large factor; rank and select remain $O(1)$ under the word-RAM assumption.The construction carries a more involved encoding and decoding pipeline than sdarray, so RRR and sdarray still dominate in practice. The result confirms that neither $m\lceil \lg(n/m) \rceil$ nor $\mathcal{B}(n, m)$ is the last word on the sparse-bitvector problem.

The quantity $\lg \binom{n}{m} = \mathcal{B}(n, m)$ is the independent-encoding cost H_{wc} of Definition 1.2.3, evaluated on a single set. Applying Theorem 2.2.2 (or Theorem 2.1.7) to each characteristic bitvector of a collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ yields a representation of total space $H_{wc}(\mathcal{S}) + O(n + m \lg n)$

bits, the set-by-set baseline.The leap from one set to many, where inter-set structure can drive the cost below H_{wc}, is the problem of Chapter 5.

2.3 Wavelet trees

Rank and select on a bitvector handle a binary alphabet. A sequence $S[1..n]$ over an alphabet $\Sigma = [1..\sigma]$ with $\sigma > 2$ admits no characteristic bitvector of length n, and encoding each symbol in a fixed $\lceil \lg \sigma \rceil$-bit field costs $n\lceil \lg \sigma \rceil$ bits with no support for rank_c or select_c beyond a linear scan. A naive reduction to the binary case, with one bitvector B_c per symbol $c \in \Sigma$ indicating the positions of c, costs $n\sigma$ bits, unacceptable for any non-trivial alphabet.The wavelet tree of Grossi, Gupta, and Vitter [27] reduces rank and select on S to rank and select on a hierarchy of bitmaps defined by a binary partition of the alphabet, at total cost $n\lceil \lg \sigma \rceil + o(n \lg \sigma)$ bits and $O(\lg \sigma)$ time per operation. Compressing those bitmaps to their zero-order entropy drops the leading term to $nH_0(S)$, matching the empirical entropy of the sequence.

[27]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

Definition 2.3.1 (Sequence operations) *Let $S[1..n]$ be a sequence over the alphabet $[1..\sigma]$. For $c \in [1..\sigma]$, $1 \le i \le n$, and $j \ge 1$, we define three operations on S:*

- $\text{access}(S, i)$*: returns the symbol $S[i]$.*
- $\text{rank}_c(S, i)$*: returns $|\{k : 1 \le k \le i,\ S[k] = c\}|$, the number of occurrences of c in the prefix $S[1..i]$, with $\text{rank}_c(S, 0) = 0$ by convention.*
- $\text{select}_c(S, j)$*: returns the unique position k such that $S[k] = c$ and $\text{rank}_c(S, k) = j$, or $\perp$ if fewer than j occurrences of c appear in S.*

For $\sigma = 2$ these three operations reduce to access, rank_b and select_b on the characteristic bitvector, recovering Definition 2.1.1 and Definition 2.1.2.

2.3.1 Structure and navigation

The wavelet tree is a balanced binary tree of height $\lceil \lg \sigma \rceil$.Each node v carries a sub-alphabet interval $[a_v, b_v] \subseteq [1..\sigma]$ and an implicit subsequence S_v, defined as the subsequence of S consisting of those symbols that belong to $[a_v, b_v]$, kept in their original order.The subsequences S_v are not stored; only their shape is recorded, through a single bit per position. The midpoint $m_v = \lfloor (a_v + b_v)/2 \rfloor$ splits $[a_v, b_v]$ into a left half $[a_v, m_v]$ and a right half $[m_v + 1, b_v]$, and the node stores a bitmap $B_v[1..|S_v|]$ with

$$B_v[i] = \begin{cases} 0 & \text{if the } i\text{-th symbol of } S_v \text{ lies in } [a_v, m_v], \\ 1 & \text{if it lies in } [m_v + 1, b_v]. \end{cases}$$

The left child of v is a wavelet tree for $S_0 = S_v\langle B_v = 0\rangle$ over $[a_v, m_v]$, and the right child is a wavelet tree for $S_1 = S_v\langle B_v = 1\rangle$ over $[m_v + 1, b_v]$.A leaf corresponds to a singleton alphabet interval $[c, c]$ and stores no bitmap; its label is implicit in the root-to-leaf path.The root covers the entire alphabet $[1..\sigma]$, with $S_{\text{root}} = S$ and B_{root} of length n, as in Figure 2.4.

The space accounting is the first lever that makes the structure useful. At every level ℓ of the tree, the node subsequences $\{S_v\}_{v \text{ at level } \ell}$ partition the

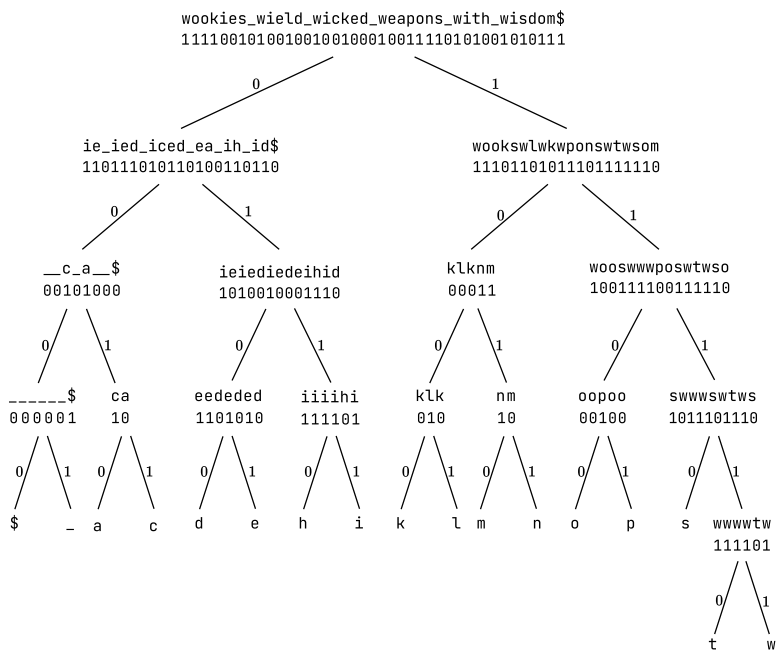

Figure 2.4: Wavelet tree for the string S = `wookies_wield_wicked_weapons_with_wisdom$` over an alphabet of 17 symbols. Each internal node carries a sub-alphabet interval; its bitmap encodes, for every symbol of the implicit subsequence, which side of the midpoint split it falls on. The subsequences are drawn beside each node for illustration; the structure stores only the topology and the bitmaps.

positions of S, because each symbol of S belongs to exactly one alphabet interval at every level.Therefore $\sum_{v \text{ at level } \ell} |B_v| \leq n$, with equality as long as no node at level ℓ is a leaf; deeper levels may store strictly fewer than n bits once some subtrees have already collapsed onto single-symbol leaves. Summing over the $\lceil \lg \sigma \rceil$ levels, the total length of the bitmaps is at most $n \lceil \lg \sigma \rceil$ bits.Attaching to each bitmap the constant-time rank and select auxiliaries of Theorem 2.1.1 and Theorem 2.1.2 costs $o(|B_v|)$ bits per bitmap. Summed level by level the overhead is $o(n \lg \sigma)$.The tree topology, encoded with explicit child pointers, takes $O(\sigma \lg n)$ additional bits, since the tree has σ leaves and $\sigma - 1$ internal nodes each storing pointers and bitmap offsets of $O(\lg n)$ bits.The total space of the pointered wavelet tree is therefore $n \lceil \lg \sigma \rceil + o(n \lg \sigma) + O(\sigma \lg n)$ bits. The tree is built in $O(n \lg \sigma)$ time by linear-time processing at each internal node, where the node reads its subsequence S_v, writes its bitmap B_v, and partitions S_v into the two children's subsequences. Since the subsequences at any level partition the positions of S, the work at each level sums to $O(n)$, and summing over the $\lceil \lg \sigma \rceil$ levels yields the claimed bound.

The wavelet tree supports three operations in $O(\lg \sigma)$ time each, by a traversal of height $\lceil \lg \sigma \rceil$ with a constant amount of word-RAM work per level. We describe them in prose, following the presentation of Navarro [28].

[28]: Navarro (2014), *Wavelet Trees for All*

Access reads the i-th symbol of S by a top-down traversal starting at the root. At the current node v, consult $B_v[i]$. If $B_v[i] = 0$, the i-th symbol of S_v lies in $[a_v, m_v]$ and appears in the left child's subsequence S_0; its position within S_0 is the number of zeros of B_v up to position i, namely $\text{rank}_0(B_v, i)$. Descend to the left child with index updated to this value. If $B_v[i] = 1$, descend symmetrically to the right child with index $\text{rank}_1(B_v, i)$.The traversal terminates at a leaf $[c, c]$, which identifies the symbol $S[i] = c$. The cost is $\lceil \lg \sigma \rceil$ constant-time rank queries plus constant-time navigation, giving $O(\lg \sigma)$ time overall.

Symbol rank computes $\text{rank}_c(S, i)$, the number of occurrences of c in

$S[1..i]$, by a top-down traversal along the fixed root-to-leaf path of c. At node v the direction of descent depends on c, not on the bitmap. When $c \leq m_v$, the occurrences of c among $S[1..i]$ contribute to the zeros of $B_v[1..i]$; update i to $\text{rank}_0(B_v, i)$ and descend to the left child. When $c > m_v$, update i to $\text{rank}_1(B_v, i)$ and descend to the right child.At the leaf $[c, c]$, the final index is $\text{rank}_c(S, i)$ for the original query. The path has length $\lceil \lg \sigma \rceil$ and each step performs one binary rank, yielding $O(\lg \sigma)$ time; Figure 2.5 traces a concrete instance of this descent.

Symbol select computes $\text{select}_c(S, j)$, the position of the j-th occurrence of c in S, by a bottom-up traversal starting at the leaf $[c, c]$ with $j_0 = j$. Moving from the current node v' to its parent v, the index is mapped through a binary select on B_v. If v' is the left child of v, the j-th occurrence of c inside $S_{v'}$ corresponds to the j-th zero of B_v, so update j to $\text{select}_0(B_v, j)$. If v' is the right child, update j to $\text{select}_1(B_v, j)$.When the traversal reaches the root, the final index is the answer $\text{select}_c(S, j)$ in S. The total cost is $\lceil \lg \sigma \rceil$ constant-time selects, giving $O(\lg \sigma)$ time.

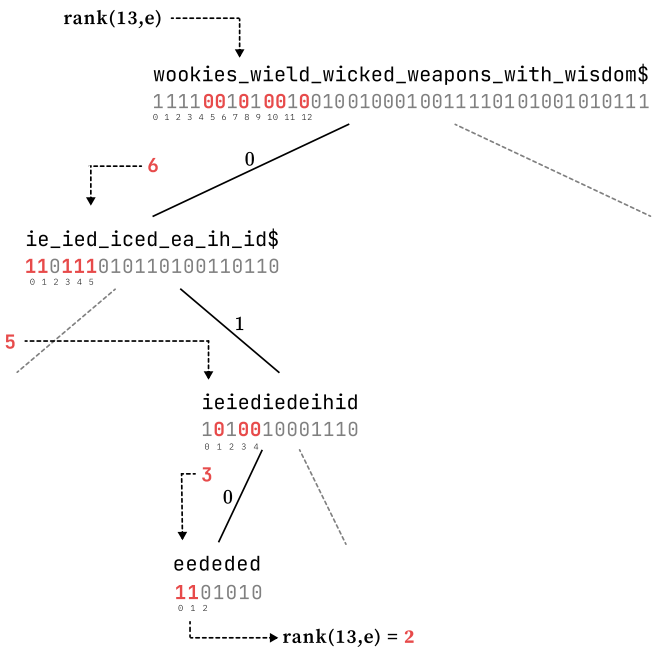

Figure 2.5: Trace of $\text{rank}_{\mathtt{e}}(S, 13) = 2$ on the wavelet tree of Figure 2.4. The descent follows the root-to-leaf path of the target symbol e, remapping the query prefix length through one binary rank at each of four levels. The intermediate values $13 \to 6 \to 5 \to 3 \to 2$ trace the query index from the root to the leaf [e, e]. Each remapping is a rank_0 or rank_1 on the local bitmap, selected by whether e falls in the left or right half of the current alphabet interval.

The analysis and the three operations combine into the base theorem of Grossi, Gupta, and Vitter.

Theorem 2.3.1 (Wavelet tree[27]) *A sequence $S[1..n]$ over an alphabet $[1..\sigma]$ can be represented in*

$$n\lceil \lg \sigma \rceil \;+\; o(n \lg \sigma) \;+\; O(\sigma \lg n) \quad \textit{bits}$$

so that $\text{access}(S, i)$, $\text{rank}_c(S, i)$, *and* $\text{select}_c(S, j)$ *are supported in* $O(\lg \sigma)$ *time on the word RAM with word size* $\omega = \Omega(\lg n)$.

Sketch. The tree is the balanced binary partition of $[1..\sigma]$ with bitmaps $\{B_v\}$ as defined above. At every level, the bitmaps partition the n positions of S, so their combined length is n bits per level and $n\lceil \lg \sigma \rceil$ bits in total. Constant-time rank and select auxiliaries on each B_v from Theorem 2.1.1 and Theorem 2.1.2 contribute $o(|B_v|)$ bits per bitmap and $o(n \lg \sigma)$ bits summed over all levels. The tree topology, with σ leaves and child

pointers of $O(\lg n)$ bits, contributes $O(\sigma \lg n)$ bits. The three operations are carried out by the top-down access and rank traversals and the bottom-up select traversal described above; each performs one binary rank or select per level and visits $\lceil \lg \sigma \rceil$ levels, for $O(\lg \sigma)$ time. □

The topology term $O(\sigma \lg n)$ is asymptotically smaller than $n\lceil \lg \sigma \rceil$ only when $\sigma = o(n)$. For alphabets growing with the input, the pointer overhead can dominate the bitmaps themselves, and the representation is no longer succinct. Navarro [28] surveys a pointerless layout that avoids the topology cost at no asymptotic penalty in query time.

The $O(\sigma \lg n)$ overhead comes entirely from the explicit tree topology and the per-node offsets into the global bit storage. For alphabets of size $\sigma = \Theta(n)$ this term dominates and can overwhelm the $n\lceil \lg \sigma \rceil$ leading term, motivating a topology-free layout.

[28]: Navarro (2014), *Wavelet Trees for All*

The idea reshapes the tree so that, at every level ℓ, the bitmaps of the 2^ℓ nodes are concatenated into a single long bitmap of length n, and the $\lceil \lg \sigma \rceil$ level bitmaps are concatenated into a single global string $B[1..n\lceil \lg \sigma \rceil]$.Level ℓ occupies positions $n(\ell-1)+1$ through $n\ell$ of B. The shape of the tree is altered so that all leaves are aligned to the left, which lets every level be filled to length exactly n without leaving gaps.A node v at level ℓ is identified by its bitmap's range $[l, r] \subseteq [n(\ell-1)+1, n\ell]$ within the level. The range of its left child at level $\ell+1$ starts at $n+l$ and ends at $n+l+\mathrm{rank}_0(B, l, r)-1$, where $\mathrm{rank}_0(B, l, r) = \mathrm{rank}_0(B, r) - \mathrm{rank}_0(B, l-1)$ counts the zeros of B_v; the right child starts immediately after, extending for $\mathrm{rank}_1(B, l, r)$ positions.Navigation from a node to its children requires one rank query per descent, so the downward traversal costs $O(\lg \sigma)$ rank queries, matching the pointered version.Upward traversal is handled by descending from the root to the target leaf to record every node's range, then unwinding the recursion; the asymptotic time is preserved.The topology is entirely encoded by B itself together with the level offsets, and the space drops to $n\lceil \lg \sigma \rceil + o(n \lg \sigma)$ bits, with no explicit $O(\sigma \lg n)$ term.

2.3.2 Entropy compression

The leading term $n\lceil \lg \sigma \rceil$ of Theorem 2.3.1 is tight against the *worst-case* encoding cost of a sequence $S[1..n]$ over $[1..\sigma]$, but ignores symbol-frequency skew. Write $n_c = |\{k : S[k] = c\}|$ for the number of occurrences of symbol c in S; then the zero-order empirical entropy of S is

$$H_0(S) \;=\; \sum_{c \in [1..\sigma]} \frac{n_c}{n} \lg \frac{n}{n_c} \;\le\; \lg \sigma,$$

with the inequality strict whenever the distribution of symbols is unbalanced.A compressor that treats every symbol uniformly cannot beat $n \lg \sigma$ bits, and the plain wavelet tree already matches this baseline level by level. To improve on the baseline, we must exploit the unbalanced frequencies of the bitmaps themselves, which inherit the skew of the source. Ferragina, Mäkinen, Manzini, and Navarro [29] take this route by replacing each bitmap B_v with an entropy-compressed indexable dictionary via Theorem 2.1.7. An identity of Grossi, Gupta, and Vitter [27] tells us that the per-node zero-order entropies assemble into the zero-order entropy of S, and the $nH_0(S)$ bound follows at once.

[29]: Ferragina et al. (2007), *Compressed Representations of Sequences and Full-Text Indexes*

[27]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

The identity is best seen on a small alphabet before being stated in general. Take $\sigma = 4$, so that the wavelet tree has three internal nodes: the root, its left child v_ℓ, and its right child v_r.Let B_{root} contain n_0 zeros (symbols in

$\{1, 2\}$) and n_1 ones (symbols in $\{3, 4\}$); then the zero-order cost of the root bitmap is $n_0 \lg(n/n_0) + n_1 \lg(n/n_1)$.At the left child v_ℓ, the bitmap B_{v_ℓ} has length n_0; it contains n_{00} zeros (occurrences of symbol 1) and n_{01} ones (occurrences of symbol 2), so $n_{00} + n_{01} = n_0$.Its zero-order cost is $n_{00} \lg(n_0/n_{00}) + n_{01} \lg(n_0/n_{01})$. Symmetrically, B_{v_r} has length n_1 with $n_{10} + n_{11} = n_1$, and zero-order cost $n_{10} \lg(n_1/n_{10}) + n_{11} \lg(n_1/n_{11})$.

Summing the three contributions,

$$\sum_{v \text{ internal}} |B_v| \cdot H_0(B_v) = n_0 \lg \tfrac{n}{n_0} + n_1 \lg \tfrac{n}{n_1} + n_{00} \lg \tfrac{n_0}{n_{00}} + n_{01} \lg \tfrac{n_0}{n_{01}} + n_{10} \lg \tfrac{n_1}{n_{10}} + n_{11} \lg \tfrac{n_1}{n_{11}}.$$

Expanding the children's contributions via $\lg(n_0/n_{0j}) = \lg n_0 - \lg n_{0j}$, the cross-term $\lg n_0$ appears with coefficient $n_{00} + n_{01} = n_0$, matching exactly the $n_0 \lg n_0$ piece hidden inside $n_0 \lg(n/n_0) = n_0 \lg n - n_0 \lg n_0$. The same cancellation removes $\lg n_1$.What remains after the cancellation is

$$n_0 \lg n - n_{00} \lg n_{00} - n_{01} \lg n_{01} + n_1 \lg n - n_{10} \lg n_{10} - n_{11} \lg n_{11} = \sum_{c=1}^{4} n_c \lg \frac{n}{n_c} = nH_0(S),$$

where we used $n_0 + n_1 = n$ and relabelled $n_{00}, n_{01}, n_{10}, n_{11}$ as n_1, n_2, n_3, n_4 following the leaf assignment.

The identity depends only on the bottom-up composition of conditional probabilities through the binary partition; the shape of the binary wavelet tree is immaterial. A skewed or Huffman-shaped layout yields the same $nH_0(S)$ as the balanced one.

The cancellation is not an artefact of $\sigma = 4$. For any alphabet size and any binary wavelet tree shape, a bottom-up induction on the tree gives

$$\sum_{v \text{ internal}} |B_v| \cdot H_0(B_v) = nH_0(S).$$

Summing the per-node zero-order costs reproduces the zero-order cost of the whole sequence, and the wavelet tree exposes this decomposition explicitly.

Theorem 2.3.2 (Compressed wavelet tree[29]) *A sequence $S[1..n]$ over an alphabet $[1..\sigma]$ can be represented in*

$$nH_0(S) + o(n \lg \sigma) \quad \textit{bits}$$

so that $\mathrm{access}(S, i)$, $\mathrm{rank}_c(S, i)$, *and* $\mathrm{select}_c(S, j)$ *are supported in $O(\lg \sigma)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.*

Sketch. Take the pointerless wavelet tree of Theorem 2.3.1 and replace each bitmap B_v with its RRR representation from Theorem 2.1.7, which stores B_v in $|B_v|H_0(B_v) + o(|B_v|) + O(\lg \lg |B_v|)$ bits and answers rank_b and select_b on B_v in $O(1)$ time.The per-node entropy summands assemble via the identity into $nH_0(S)$ bits.The $o(|B_v|)$ redundancies, summed along each of the $\lceil \lg \sigma \rceil$ levels where $\sum_v |B_v| = n$, contribute $o(n)$ bits per level and $o(n \lg \sigma)$ bits in total.The access, rank, and select traversals of Theorem 2.3.1 visit $\lceil \lg \sigma \rceil$ levels and perform one constant-time binary rank or select per level against the RRR bitmap; the total time is $O(\lg \sigma)$. □

The structure traces back to Chazelle's functional approach to multidimensional searching [30], an $n \times n$ grid representation for 2D range queries that, read retrospectively, is a wavelet tree on a permutation [28]. Grossi, Gupta, and Vitter in 2003 crystallised the idea into a tool for sequences over finite alphabets.

The wavelet tree carries two complementary readings that together explain why it belongs in our toolkit. Read as a sequence representation,

it answers access, rank_c, and select_c in $O(\lg \sigma)$ time within $nH_0(S) + o(n \lg \sigma)$ bits. Read as a reordering device, the leaves, traversed left to right, present the symbols of S in stable sorted order. The j-th position inside the leaf $[c, c]$ corresponds to the j-th occurrence of c in S, and the bottom-up select traversal of Theorem 2.3.1 converts this sorted position into the original position in S via one select_0 or select_1 per level. The sequence S and its stable sorting are both directly readable from the same bitmaps, related to each other by a $\lceil \lg \sigma \rceil$-level chain of rank/select operations.

The mechanism underlying both readings is a hierarchical binary partition of the alphabet, instantiated level by level as a bitmap on which rank and select answer the primitive queries. This same template, a hierarchical binary partition supported by rank on the induced bitmaps, generalizes from sequences to labeled ordinal trees in a later chapter of this thesis, where the partition is applied to the alphabet of node labels and rank on bitmaps becomes the primitive for queries on labeled trees. The wavelet tree belongs in our toolkit as the prototypical instance of this template on strings.

3 Succinct Representations of Trees

3.1 Encoding problem 26
3.2 Depth-first encodings . . . 31
3.3 Fully functional encodings 36
3.4 Labeled trees and α-operations 40

Pointer-based encodings of an n-node ordinal tree pay $\Theta(n \lg n)$ bits, a factor $\Theta(\lg n)$ above the information-theoretic minimum. Closing the gap is the analog for trees of the $n + o(n)$ goal achieved for bitvectors in Section 2.1.

A bitvector exposes the structure of a single set, and the primitives of Chapter 2 answer membership, rank, and select on it within a low-order term of the information-theoretic optimum. A set collection introduces a second layer of structure, namely the relationships between sets, and the natural object to capture those relationships is a tree whose nodes index the sets and whose edges express containment, similarity, or any other relation we wish to exploit. To carry this idea through, we must first know how to represent the tree itself in succinct space, and how to navigate it in time competitive with a pointer representation. The present chapter develops the answer.

3.1 Encoding problem

A pointer encoding of an ordinal tree of n nodes pays $O(n \lg n)$ bits, since each node carries pointers whose width matches the address space. The number of distinct ordinal trees on n nodes is far smaller than the $n^{\Theta(n)}$ structures expressible with pointers, and the information-theoretic minimum is asymptotically $2n$ bits per tree. The gap between $\Theta(n \lg n)$ and $2n$ is the same factor of $\Theta(\lg n)$ that separated linear-time scanning from the $n + o(n)$ rank-and-select machinery of Section 2.1. The present section sets the lower bound, fixes the operations a tree representation must support, and develops the first encoding that meets the lower bound up to a low-order term.

3.1.1 Catalan lower bound

[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

An ordinal tree is a rooted tree whose children at every node are linearly ordered, so that the children of a node v form a sequence $c_1, c_2, \ldots, c_{\deg(v)}$ rather than a set. Munro and Raman [31] call the same combinatorial object a *rooted ordered tree*, and the term *plane tree* also appears in the combinatorial literature. We write *ordinal tree* throughout. The unique node with no parent is the *root*. A node with no children is a *leaf*; otherwise it is *internal*. The *depth* of a node is its distance from the root, with the root at depth 0. The *subtree* rooted at v, written T_v, is the induced subtree on v and all its descendants, and its size is $|T_v|$. The *degree* of v is the number of its children. A *preorder* traversal visits each node before its children, recursively from left to right; a *postorder* traversal visits each subtree before its root. Both traversals produce a permutation of the n nodes, and we will take preorder as the canonical numbering when nodes are addressed by integers in $[1..n]$.

A representation of an ordinal tree is judged by the operations it supports. The list below comes from the navigational vocabulary surveyed in the modern tree-encoding literature, and is more demanding than the three-operation set {`left-child`, `right-child`, `null`} that Jacobson

considered for binary trees [3, 10]. Munro and Raman extend the goal to general ordinal trees and add subtree size to the basic vocabulary [31]. The full set fixes the target for the rest of the chapter.

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*
[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

Definition 3.1.1 (Operations on ordinal trees) *Given an ordinal tree T on n nodes addressed by integers in $[1..n]$, a representation should support, for every node $x \in T$, the following operations.*

- `parent`(x), *the parent of x (undefined at the root).*
- `first_child`(x), *the leftmost child of x (undefined at a leaf).*
- `next_sibling`(x), *the sibling of x immediately to its right (undefined at the rightmost child).*
- `child`(x, i), *the i-th child of x in left-to-right order.*
- `degree`(x), *the number of children of x.*
- `subtree_size`$(x) = |T_x|$, *the size of the subtree rooted at x.*
- `depth`(x), *the depth of x measured from the root at 0.*
- `ancestor`(x, d), *the ancestor of x at depth d (with* `ancestor`$(x,$ `depth`$(x)) = x$*).*
- `LCA`(x, y), *the lowest common ancestor of x and y.*
- `level_ancestor`(x, d), *the ancestor of x at d levels above x.*
- `preorder_rank`(x) *and* `preorder_select`(i), *mapping nodes to their preorder positions and back.*

The number of ordinal trees on n nodes is a Catalan number, by the standard bijection that maps each ordinal tree on n nodes to a binary tree on $n-1$ nodes (the rotation correspondence recalled by Munro and Raman [31] in their treatment of rooted ordered trees). Writing

[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

$$C_n \;=\; \frac{1}{n+1}\binom{2n}{n}$$

for the n-th Catalan number, the count is C_{n-1}, so any encoding that distinguishes all ordinal trees on n nodes uses at least $\lg C_{n-1}$ bits. The Stirling estimate $\binom{2n}{n} = 2^{2n}/\sqrt{\pi n} \cdot (1 + O(1/n))$ gives

$$\lg C_{n-1} \;=\; 2n - \tfrac{3}{2}\lg n - \tfrac{1}{2}\lg \pi + O(1/n) \;=\; 2n - O(\lg n),$$

so the information-theoretic minimum is $2n$ bits up to a low-order $O(\lg n)$ correction. Jacobson states the bound in the equivalent looser form $2n + o(n)$ [3, 10] and observes that a pointer representation, paying $O(n \lg n)$ bits per tree, exceeds the optimum by a factor of $\Theta(\lg n)$.

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*

Theorem 3.1.1 (Catalan lower bound [3, 10]) *Any encoding of ordinal trees on n nodes that distinguishes every such tree uses at least $\lg C_{n-1} = 2n - O(\lg n)$ bits in the worst case, where $C_k = \frac{1}{k+1}\binom{2k}{k}$ is the k-th Catalan number.*

Sketch. The Catalan number C_{n-1} counts ordinal trees on n nodes by the rotation bijection with binary trees on $n-1$ nodes, equivalently by the bijection with balanced parenthesis sequences of length $2(n-1)$. Any encoding scheme is a function from the C_{n-1} trees into $\{0,1\}^*$, and an injective such function on a domain of size C_{n-1} requires codewords of length at least $\lg C_{n-1}$ for some tree. The closed form for $\lg C_{n-1}$ follows from Stirling. □

The lower bound fixes the target. A representation that uses $2n + o(n)$ bits while supporting every operation of Definition 3.1.1 in $O(1)$ time on the word RAM is the analogue, for ordinal trees, of the $n + o(n)$ goal pursued for bitvectors in Section 2.1. Jacobson [3, 10] calls a representation in this regime *asymptotically optimal*, since the ratio of bits used to bits required tends to 1 as n grows, and Munro and Raman [31] formulate the same target as matching the information-theoretic minimum to within a lower-order term.

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*
[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

The combinatorial existence of a $2n$-bit code is easy. Encoding the parenthesis sequence directly attains $2n$ bits per tree, and the inverse map from sequences to trees is the unique reading of a balanced string as a preorder DFS. The hard problem is to support the operations of Definition 3.1.1 in $O(1)$ time, or even in time competitive with the $O(\lg n)$ bit-probes a pointer representation pays. Jacobson observed [3, 10] that the recursive encoding $\mathrm{rep}(T) = 0\,\mathrm{rep}(\mathrm{left}(T))\,1\,\mathrm{rep}(\mathrm{right}(T))$ achieves $2n$ bits for binary trees, but locating the right child of a node requires a linear scan to balance parentheses. The encoding is space-optimal yet time-disastrous. Munro and Raman pose the same dichotomy for general ordinal trees [31], and they identify the inspection of the size of the subtree at the current node as essential. Their Theorem 2 then resolves the dichotomy at the level of $2n + o(n)$ bits with $O(1)$ parent and subtree size, which we will return to once the simpler encodings are in place.

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*

[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

A preorder traversal of T produces a sequence of $2n$ parentheses by writing an open bracket when a node is first visited and a close bracket when its subtree is exhausted. A postorder traversal interchanges the two events, but yields the same balanced sequence read backwards with parentheses swapped. Either traversal sequence reconstructs T uniquely once one fixes the convention. Neither sequence supports a single operation of Definition 3.1.1 in better than linear time without auxiliary structure. The $O(1)$-time rank and select machinery of Theorem 2.1.1 and Theorem 2.1.2 carries the burden, and every encoding studied below organizes the bits so that each tree operation reduces to a constant number of rank, select, or related queries on a bitvector or balanced-parenthesis sequence.

3.1.2 LOUDS as a first encoding

The most direct route from the lower bound of Theorem 3.1.1 to a working encoding writes the degree of every node, in unary, listed in level order. Jacobson [3, 10] introduced the construction in his 1989 FOCS paper as the *level-order unary degree sequence*; the now-standard acronym LOUDS came into use later in the literature.

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*

The construction proceeds in three steps. First, augment T with a synthetic *super-root*, a new node of degree one whose only child is the original root. The super-root has no analog in T and exists solely to maintain a clean correspondence between bit positions and nodes. Second, in level order, that is, in BFS order starting from the super-root, list the degree of each node. Third, encode each integer degree d in unary as $\underbrace{1\cdots1}_{d}\,0$, and concatenate the codewords in BFS order. The result is a bitvector B

in which each 1-bit announces a child and the 0-bits separate the degree blocks of consecutive nodes.

Figure 3.1 shows the construction on a tree of seven nodes, augmented to eight by the super-root.

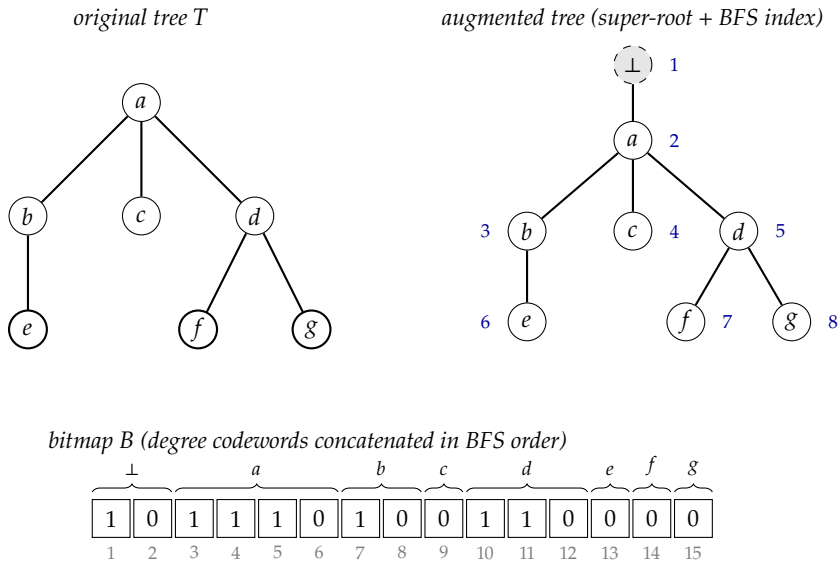

Figure 3.1: LOUDS encoding of an ordinal tree on seven nodes. The super-root $\perp$ is added so that every node, including the original root, has a unique parent and contributes a unary degree codeword. The augmented tree is traversed in BFS order, the BFS indices $1, \ldots, 8$ are shown next to each node, and the degree of each node in turn is written in unary as 1^d0, concatenated to form the bitmap B. Grouping braces above B mark the codewords contributed by the super-root and by nodes a, b, c, d, e, f, g in BFS order.

Counting bits is straightforward. Each non-root node contributes exactly one 1-bit, since it is the child of some other node, so the 1-count equals the number of children summed over all parents, which equals the number of non-root nodes. Each node contributes exactly one 0-bit, the terminator of its own degree codeword. With the super-root added, the augmented tree has $n + 1$ nodes, of which n are non-root, so B contains n ones and $n + 1$ zeros, for a total length of $2n + 1$.

Theorem 3.1.2 (LOUDS [10]) *An ordinal tree of n nodes can be represented in $2n + 1 + o(n)$ bits so that the operations* `parent`, `first_child`, *and* `next_sibling` *run in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The bitvector B is stored verbatim with the rank/select structures of Theorem 2.1.1 and Theorem 2.1.2 on top. Jacobson's 1989 FOCS paper [3] states the construction in a bit-access model with $O(\lg n)$ per operation; the $O(1)$ word-RAM bound is the contribution of his PhD thesis [10] via constant-time rank and select.*

[3]: Jacobson (1989), *Space-efficient static trees and graphs*

[10]: Jacobson (1988), *Succinct Static Data Structures*

Proof. The bit count is counted above. The $o(n)$ overhead is the rank-and-select index of Theorem 2.1.1 and Theorem 2.1.2 mounted on B, which gives $O(1)$ time for rank_b and select_b for $b \in \{0, 1\}$. Two structural properties of the encoding deliver the three operations.

Jacobson identifies the k-th BFS node with the position of its corresponding 1-bit in B. We work instead with the equivalent 0-bit convention, in which a node is identified with the position of its own terminator; this swap leaves the operations unchanged and produces formulas that are direct translations of Jacobson's box. Two structural properties of the encoding follow.

Property 1. The k-th node in BFS order corresponds to the k-th 0-bit of B. Indeed, each node contributes a single 0-bit, the terminator of its degree

codeword, and the codewords are concatenated in BFS order. Identify node $x \in [1..n+1]$ with the position of its terminator in B, so x is the index $\text{select}_0(B, x)$.

Property 2. The children of the k-th BFS node are the nodes whose terminating 0s appear in B immediately after the k-th 1-bit. The k-th node's degree codeword starts at $\text{select}_0(B, k-1)+1$ and ends at $\text{select}_0(B, k)$, with $\deg(k)$ ones followed by one zero. The 1-bit indexed h in B marks the parent-child edge to the $(h+1)$-th BFS node, since the BFS ordering on the children of the augmented tree visits them in the same order as their terminators appear, and each 1-bit corresponds to one such edge.

The three operations follow.

Operation derivations. For `parent`(x), the node x corresponds to the x-th 0 at position $p = \text{select}_0(B, x)$, and the 1-bit immediately preceding p in the cluster of 1s ending at $p-1$ is the edge from x's parent. The number of 1-bits up to position p counts the parent-child edges drawn so far, which is exactly the BFS index of the parent, so `parent`$(x) = \text{rank}_1(B, \text{select}_0(B, x))$.

For `first_child`(x), the parent-child edges out of x are the 1-bits between $\text{select}_0(B, x-1)+1$ and $\text{select}_0(B, x)-1$, in the run of 1s preceding x's terminator. Equivalently, the children of x correspond to the BFS-indexed 1-bits in this run. The first such 1-bit is at position $\text{select}_1(B, \text{rank}_1(B, \text{select}_0(B, x-1))+1)$. The first child is then the node terminating immediately after, and equivalently is identified by the rank of the 1-bit it terminates. Setting $y = \text{rank}_1(B, \text{select}_0(B, x-1))+1$, the first child of x is the $(y+1)$-th BFS node, that is, $y+1$ if we adopt the convention that the BFS index of the node whose parent edge is the h-th 1-bit is $h+1$. The composition of one rank and one select is $O(1)$.

For `next_sibling`(x), examine the bit at position $\text{select}_0(B, x)+1$. If it is a 1, then x has a next sibling, and the next sibling is the BFS node corresponding to the 0 at position $\text{select}_0(B, x+1)$, that is, $x+1$. If the bit is a 0, then x has no next sibling. The check is a single bit access plus a constant addition. □

The same encoding supports a fourth operation as a direct corollary. The codeword of x has its terminator at $\text{select}_0(B, x)$ and its previous terminator at $\text{select}_0(B, x-1)$, so its length is the difference, and the degree of x, the number of 1-bits in the codeword, is the length minus one. We obtain `degree`$(x) = \text{select}_0(B, x) - \text{select}_0(B, x-1) - 1$ in $O(1)$ on the word RAM, by two select calls and one subtraction.

LOUDS achieves `parent`, `first_child`, and `next_sibling`, together with the derived `degree`, at the cost of one or two rank/select calls each, all in $O(1)$ time on top of an $o(n)$-bit auxiliary structure. The space exceeds the lower bound of Theorem 3.1.1 only in the additive constant 1 for the super-root and the low-order $o(n)$ overhead, so LOUDS attains the asymptotic optimum of Theorem 3.1.1. The remaining operations of Definition 3.1.1 are out of reach of this encoding alone. The reason is structural rather than incidental. `subtree_size`, `depth`, `ancestor`, `LCA`, and `level_ancestor` all require knowledge that is local in the depth-first traversal of T but spread out across many BFS levels of B. The

descendants of x live in arbitrarily many BFS levels, and their codewords are interleaved with the codewords of all the other descendants of x's ancestors at the same depths. No constant number of rank or select calls on B can group them.

The encoding is valuable as a baseline rather than as a complete answer. Other encodings reorganize B along depth-first lines, putting the descendants of x into a contiguous range, and recover the operations LOUDS leaves behind. The depth-first encodings developed in the next section take this route.

3.2 Depth-first encodings

LOUDS lists nodes in breadth-first order, and the descendants of a node x scatter across an arbitrary number of subsequent BFS levels, interleaved with the descendants of x's ancestors at the same depths. The operations that LOUDS leaves out, namely `subtree_size`, `depth`, `ancestor`, `LCA`, and `level_ancestor`, are precisely those that read a depth-first slice of T. A representation organized along the depth-first traversal places every node together with its descendants in a contiguous range of the encoding, restoring the structural locality that BFS destroys. The two encodings developed in the present section take this route. The first writes one open and one close parenthesis per node, in DFS order, and reduces every operation of Definition 3.1.1 except `child` to a constant number of primitives on the resulting balanced sequence. The second redistributes the same parentheses around each node, recovering `child` in $O(1)$ while preserving the rest of the machinery.

3.2.1 Balanced parentheses

A preorder traversal of T writes an open parenthesis when a node is first visited and a matching close parenthesis when its subtree is exhausted. The result is a string $B[1..2n]$ over { `(` , `)` } in which each node contributes exactly one open and one close, and the parentheses are balanced because every node's close arrives only after the closes of its descendants have been written. The same string read backwards with parentheses swapped is the postorder traversal sequence; either convention reconstructs T uniquely. We fix the preorder convention throughout, and we identify each node v with the position i_v of its open parenthesis in B. Two structural facts make this identification useful. First, the position i_v equals the preorder rank of v, since the k-th open parenthesis written by the traversal is exactly the k-th node visited in preorder. Second, the substring of B between i_v and the position j_v of v's matching close is the parenthesis sequence of the subtree rooted at v; its length is exactly $2 \cdot |T_v|$, since every descendant of v contributes one open and one close inside the substring, and no other node does.

The single quantity that drives every navigational operation on B is the *excess* at a position. For $i \in [1..2n]$ define

$$e(B, i) \;=\; \mathrm{rank}_{(}(B, i) \;-\; \mathrm{rank}_{)}(B, i),$$

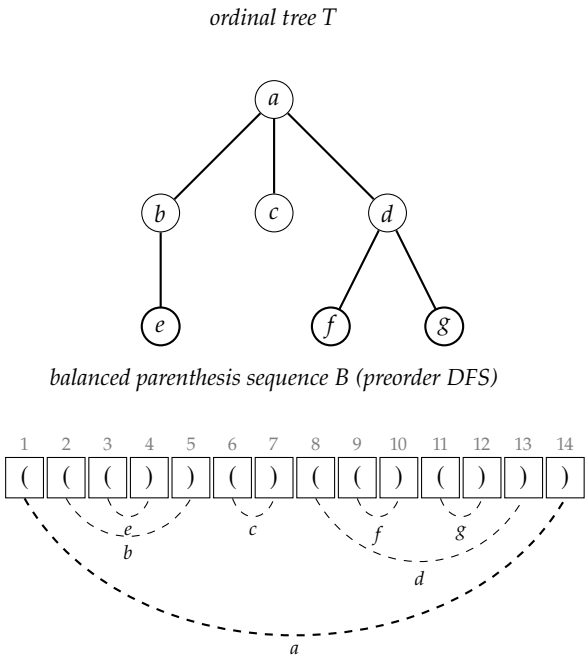

Figure 3.2: Balanced parenthesis encoding of the same seven-node ordinal tree used in Figure 3.1. A preorder DFS visit writes an open parenthesis on entry to a node and a close on exit. Each dashed arc joins the matching pair of one node, labelled by the node it represents. The substring between the two parentheses of a node is exactly the encoding of its subtree, of length $2 \cdot |T_v|$.

The excess at position i is the depth of the node currently being traversed at step i of the DFS. It rises by one at every open parenthesis and drops by one at every close.

the number of open parentheses minus the number of close parentheses among the first i symbols. The excess at i_v equals the depth of v in T, since the parenthesis at i_v closes only after every parenthesis written between positions 1 and i_v that has not yet closed, and the open ones at that moment are exactly the ancestors of v together with v itself. The excess is non-negative for every prefix of B and equals zero only at the end of B, since the sequence is balanced. The matching close of an open parenthesis at position i is the smallest $j > i$ such that $e(B, j) = e(B, i) - 1$; the closest enclosing open parenthesis of i is the largest $i' < i$ with $e(B, i') = e(B, i) - 1$.

[31]: Munro et al. (2001), *Succinct representation of balanced parentheses and static trees*

Four primitives on the excess capture every operation of Definition 3.1.1 that does not require fast access to the i-th child. Given a position i in B, the operation `findclose`(i) returns the position of the close parenthesis that matches the open at i; `findopen`(i) returns the position of the open that matches the close at i; `enclose`(i) returns the position of the open parenthesis of the closest matched pair that strictly encloses i; and `rmq`(i, j) returns a position of minimum excess in the range $[i, j]$. Munro and Raman [31] state the operations as `findclose`, `findopen`, `excess`, `enclose`, and `double_enclose`; we absorb `excess`(i) into the rank identity $e(B, i) = \mathrm{rank}_{(}(B, i) - \mathrm{rank}_{)}(B, i)$ already supplied by Theorem 2.1.1, and we replace `double_enclose` with the more general range-minimum query, which subsumes the LCA reduction below. All four primitives can be answered in constant time on the word RAM after attaching $o(n)$ bits of auxiliary structure to B, and the rest of the navigational vocabulary then follows by short formulas in these primitives.

Theorem 3.2.1 (Balanced parentheses [31]) *Given a balanced parenthesis sequence B of length $2n$, there is a data structure of $o(n)$ bits attached to B such that on the word RAM with word size $\omega = \Omega(\lg n)$ the operations* `findclose`, `findopen`, `enclose`, *and* `rmq` *on B all run in $O(1)$ time. Con-*

sequently, an ordinal tree on n nodes admits a representation in $2n + o(n)$ bits supporting `parent`(v), `subtree_size`(v), `depth`(v), `first_child`(v), `next_sibling`(v), `preorder_rank`(v), *and* `LCA`(u, v) *in $O(1)$ time, with* `child`(v, i) *supported in $O(i)$ time.*

Sketch. The auxiliary structure is a three-level decomposition of B that mirrors the hierarchical layout used by Jacobson [3, 10] for rank and select, but indexes the excess sequence rather than raw bit counts. Writing $b = \lg^2 n$, the sequence B is partitioned into BIG BLOCKS of length b, each big block into SMALL BLOCKS of length $\lg^2 b = 4(\lg \lg n)^2$, and each small block into TINY BLOCKS of length $2 \lg \lg n$. With every big-block boundary the structure stores the excess at that point and the location of the immediately preceding *pioneer*, a parenthesis whose match lies outside the current big block; small-block headers carry the analogous information one level finer. The pioneer count in a string divided into b blocks is at most $2b - 3$ [3], so the cumulative space for pioneer pointers is $O(n/\lg n)$ bits. A precomputed lookup table [32], indexed by the bit pattern of a tiny block and an in-block position, resolves `findclose` and `findopen` inside a tiny block in $O(1)$ word-RAM operations; its size is polynomial in $\lg n$ and therefore $o(n)$. A query `findclose`(i) first uses the tiny-block table; if the match is not local, the small-block header forwards the search to the relevant pioneer, and at most three header lookups, one in each level of the hierarchy, suffice to localize the matching parenthesis. The mirror construction on the right-excess of B handles `findopen`, and an analogous bookkeeping of right-excess minima supports `enclose` and `rmq` in the same asymptotic budget. The reductions of the operations of Definition 3.1.1 to the four primitives are direct: `parent`(v) = `enclose`(i_v), `subtree_size`(v) = (`findclose`$(i_v) - i_v + 1)/2$, `depth`$(v) = e(B, i_v)$, `first_child`$(v) = i_v + 1$ when $B[i_v + 1] =$ (and undefined otherwise, `next_sibling`(v) = `findclose`$(i_v) + 1$ when $B[$`findclose`$(i_v) + 1] =$ (and undefined otherwise, and `preorder_rank`$(v) = \mathrm{rank}_{(}(B, i_v)$. The remaining operation `LCA`(u, v) reduces to a range minimum query on the excess and is treated separately below. □

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[10]: Jacobson (1988), *Succinct Static Data Structures*

[3]: Jacobson (1989), *Space-efficient static trees and graphs*
[32]: Munro (1996), *Tables*

The lookup-table device is the four-Russians technique in the abstract sense [11], but Munro-Raman attribute the load-bearing tabulation construction to [32].

The reductions are immediate consequences of the bijection node $\leftrightarrow$ matched pair. A node's parent is the closest enclosing pair, which is what `enclose`(i_v) returns. The subtree of v occupies the substring $B[i_v..$`findclose`$(i_v)]$, of length $2|T_v|$, so the subtree size is half the difference. The depth of v is the number of open parentheses awaiting their close at position i_v, which is the excess. The first child of v exists exactly when the symbol at $i_v + 1$ is an open parenthesis, since otherwise $B[i_v + 1] =$) matches i_v and v is a leaf. The next sibling lives at `findclose`$(i_v) + 1$ when that position holds an open parenthesis, since `findclose`(i_v) is the close of v's subtree, the next position belongs to the same parent, and an open there starts the next sibling. The preorder rank of v is the count of open parentheses up to i_v, which is the binary $\mathrm{rank}_{(}(B, i_v)$ of Theorem 2.1.1. The i-th child is reachable by $i - 1$ successive applications of `next_sibling` starting from `first_child`(v), each costing $O(1)$ via `findclose`, for total cost $O(i)$.

The lowest common ancestor reduces to a range minimum query on the depth sequence of an Euler tour of T. During a DFS traversal of T, the lowest common ancestor of u and v is the shallowest node visited between the first occurrences of u and v in the traversal. The excess at a position i

is exactly the depth of the node whose parenthesis is being written at step i, so the depth-minimum position in the excess between i_u and i_v corresponds to the parenthesis of the LCA. The reduction itself predates the work of Bender and Farach-Colton [33] and is summarized in their Lemma 3 and Observation 4; their primary contribution, packaged in the same paper, is an $\langle O(n), O(1)\rangle$ preprocessing and query algorithm for the special case of ±1 range minimum queries, where consecutive entries of the queried array differ by exactly one. The level array of an Euler tour satisfies that restriction because traversing an edge changes the depth by one, and the algorithm combines a precomputed table on $\Theta(\lg n)$-length blocks with a sparse-table summary on block minima. On the BP encoding, the excess is implicit in B and need not be materialized as an explicit array; the same hierarchy of pioneers used for `findclose` supports $\texttt{rmq}(i, j)$ on the excess in $O(1)$ time within the $o(n)$-bit budget already counted.

[33]: Bender et al. (2000), *The LCA Problem Revisited*

The ±1 structure is what allows constant-time RMQ on the excess in $o(n)$ extra bits; on a general integer array, the same query takes the more elaborate construction of a Cartesian tree.

Example 3.2.1 For the tree of Figure 3.2, the BP sequence is B = `(((()) ()(((()))))` with whitespace inserted only for readability and a length of 14. Node b sits at $i_b = 2$, and $\texttt{findclose}(2) = 5$, so $\texttt{subtree_size}(b) = (5 - 2 + 1)/2 = 2$, in agreement with the subtree of b containing b and its only child e. The closest enclosing parenthesis of position $i_b = 2$ is at 1, so $\texttt{parent}(b) = a$. The depth of b is $e(B, 2) = 2 - 0 = 2$ if positions are counted with the open at i_b included, matching $\texttt{depth}(b) = 1$ once the convention $e(B, i_v) = \texttt{depth}(v) + 1$ for opens is fixed.

The bound $O(i)$ for $\texttt{child}(v, i)$ is the only blemish on the encoding. For trees of bounded degree the bound is $O(1)$ and the matter ends there; for high-degree trees it is unacceptable, since a query for the i-th child of a node of degree $d \gg 1$ pays $\Theta(i)$ findclose calls in the worst case. Munro and Raman raise this gap as an open question in their original paper. A representation that recovers $\texttt{child}(v, i)$ in $O(1)$ without giving up the rest of the BP machinery is the topic of the next subsection.

3.2.2 DFUDS

The remedy of Benoit, Demaine, Munro, Raman, Raman, and Rao [34] reorganizes the parentheses around each node. LOUDS already represented the degree of each node in unary, but in level order; the BP sequence wrote one open and one close per node, but in depth-first order. Combining the two, write the unary degree of each node in depth-first order. At node v of degree d_v, the traversal emits d_v open parentheses followed by exactly one close. Concatenating these $d_v + 1$ symbols across all nodes in preorder yields a string of length $\sum_v (d_v + 1) = (n-1) + n = 2n - 1$. A leading open parenthesis is prepended to balance the string, bringing the total length to $2n$ and matching the BP budget. The result is the *depth-first unary degree sequence* encoding, abbreviated DFUDS, shown in Figure 3.3.

[34]: Benoit et al. (2005), *Representing Trees of Higher Degree*

LOUDS writes one unary degree codeword per node in BFS order; BP writes one open and one close per node in DFS order; DFUDS writes one unary degree codeword per node in DFS order, recovering the per-node block structure of LOUDS together with the contiguous-subtree property of BP.

The string is balanced. A leaf contributes one close, its leading opens form an empty prefix, and the leading (at the very start of the string supplies the matching open in the parent's block; an internal node of degree d contributes d opens, each of which is matched by the close at the end of its subsequent child's block. By induction on the tree, every

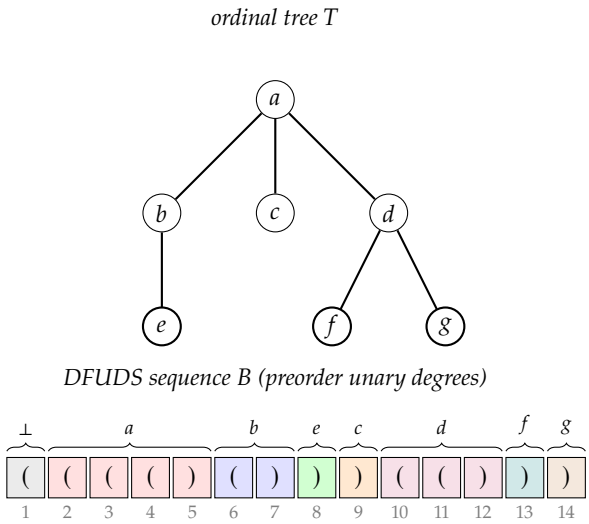

Figure 3.3: DFUDS encoding of the same seven-node ordinal tree of Figure 3.2. Each node v contributes a block of d_v open parentheses followed by one close, written in DFS preorder; a leading open parenthesis (block $\perp$) is prepended so that the resulting string of length $2n = 14$ is balanced. The colored blocks above the sequence highlight the per-node contributions of a, b, e, c, d, f, g in preorder. The position p_v of the first parenthesis of each block is the address by which the node is referenced.

close written by the construction has a matching open before it, and the prepended (matches the very last close.

A node v is identified with the position p_v of the first parenthesis of its block. The integer position $p_v \in [1..2n]$ doubles as a 1-to-$2n$ labelling and converts to the preorder index in $[1..n]$ by a single $\mathrm{rank}_{)}(B, p_v)$ query: every close in B at or before p_v corresponds to a node whose block ended at or before the start of v's block, plus the close that terminates v's parent's run of children.

The DFUDS encoding supports degree, i-th child, parent, and subtree size in constant time on top of Theorem 3.2.1. The degree of the node at position p_v is the length of its run of opens, which is the gap from p_v to the next close in B. A single $\mathrm{rank}_{)}$ followed by a $\mathrm{select}_{)}$ identifies that close, giving $\texttt{degree}(v) = \mathrm{select}_{)}(B, \mathrm{rank}_{)}(B, p_v) + 1) - p_v$. The i-th child of v is found by reading the i-th open parenthesis of v's block and following its matching close with `findclose`: jumping from the close of v's block back by i positions lands on the open parenthesis of the i-th child, and the position one beyond its match is the start of the i-th child's block. In symbols, $\texttt{child}(v, i) = \texttt{findclose}(\mathrm{select}_{)}(B, \mathrm{rank}_{)}(B, p_v)+1)-i)+1$. The whole computation reduces to two rank/select calls on a binary alphabet, one `findclose`, and constant arithmetic. The parent of v is recovered by `findopen` on the close immediately preceding v's block, which lies at $p_v - 1$, followed by a step back to the start of the enclosing block: $\texttt{parent}(v) = \mathrm{select}_{)}(B, \mathrm{rank}_{)}(B, \texttt{findopen}(p_v - 1))) + 1$. Subtree size is read off the BP machinery as before, since the substring of B between the first parenthesis of v's block and its matching close encodes the subtree of v; equivalently, $\texttt{subtree_size}(v) = (\texttt{findclose}(\texttt{enclose}(p_v)) - p_v)/2 + 1$.

Theorem 3.2.2 (DFUDS [34]) *An ordinal tree on n nodes admits a representation in $2n + o(n)$ bits that supports $\texttt{degree}(v)$, $\texttt{child}(v, i)$, $\texttt{parent}(v)$, and $\texttt{subtree_size}(v)$ in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The representation is the DFUDS string of length $2n$ together with the auxiliary structure of Theorem 3.2.1 mounted on it.*

Sketch. The DFUDS string is balanced and has length $2n$, so Theorem

3.2.1 attaches an $o(n)$-bit auxiliary structure that supports `findclose`, `findopen`, `enclose`, and `rmq` in $O(1)$ time, together with rank and select on the open and close alphabets via Theorem 2.1.1 and Theorem 2.1.2. The four operations claimed are obtained by the formulas above, each evaluating to $O(1)$ word-RAM steps. □

The redistribution that DFUDS performs is purely syntactic. Both BP and DFUDS read the same tree in the same DFS order and write $2n$ parentheses, balanced in the same sense; what differs is the meaning attached to a parenthesis. In BP, an open marks node entry and a close marks subtree exit. In DFUDS, the opens of a node's block enumerate its children and the trailing close terminates that enumeration. The shift turns the cost of `child`(v, i) from a chain of i `next_sibling` jumps into a single `findclose` on a position computed by a rank/select pair, while preserving the contiguous-subtree property that powers `subtree_size` and the `rmq`-based reduction of LCA. LOUDS handled the level-order navigational vocabulary, BP handled the depth-first vocabulary at the cost of slow `child`, and DFUDS unifies the two by changing the semantics of the parentheses without changing the underlying machinery. The constructions of the $o(n)$ auxiliary inside Theorem 3.2.1 remain ad-hoc, with one hierarchy per primitive; a single uniform construction supporting every operation of Definition 3.1.1 is the subject of the next section.

3.3 Fully functional encodings

The auxiliary structures of Theorem 3.2.1 and Theorem 3.2.2 achieve $2n + o(n)$ bits with $O(1)$ time for every operation of Definition 3.1.1, but each primitive sits on its own ad-hoc hierarchy: pioneers for `findclose`, a mirror construction for `findopen`, an independent excess-RMQ scheme for `enclose` and `rmq`. Each subsequent operation added to the BP toolkit, `leftmost_leaf`, `leaf_select`, `degree`, `child`, `level_ancestor`, called for yet another auxiliary index. The aggregate is still $o(n)$ bits, but the construction is fragmented and hard to maintain. The present section closes the chapter by replacing the per-primitive hierarchies with a single uniform machinery, in two equivalent forms. The first, due to He, Munro, and Rao, builds an $o(n)$-bit catalog of microtree patterns on top of a tree covering, supporting every operation of Definition 3.1.1 in $O(1)$ at $2n + o(n)$ bits. The second, due to Sadakane and Navarro, mounts a single range min-max tree on the BP sequence and reduces every BP primitive to one root-to-leaf traversal of that tree. Both attain the same bound; the second is simpler and admits a dynamic extension.

The starting point is a recursive decomposition of T into pieces of two carefully chosen sizes. Apply a covering procedure with parameter M to produce a partition of the nodes of T into *minitrees*, connected subgraphs of T that intersect at most at their common roots and whose sizes are between M and $3M - 4$, except for a possible undersized minitree containing the root of T. Apply the same procedure recursively, with parameter M', to each minitree, producing a partition of the minitree into *microtrees*. Geary, Raman, and Raman [35] introduced the algorithm and proved the size bound; He, Munro, and Rao [36] adopt the same

[35]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*

[36]: He et al. (2007), *Succinct Ordinal Trees Based on Tree Covering*

procedure in their construction. Choosing $M = \lceil \lg^4 n \rceil$ at the first level and $M' = \lceil \lg n/24 \rceil$ at the second level produces $O(n/\lg^4 n)$ minitrees and $O(n/\lg n)$ microtrees in total. Figure 3.4 sketches the resulting two-level decomposition on a small example.

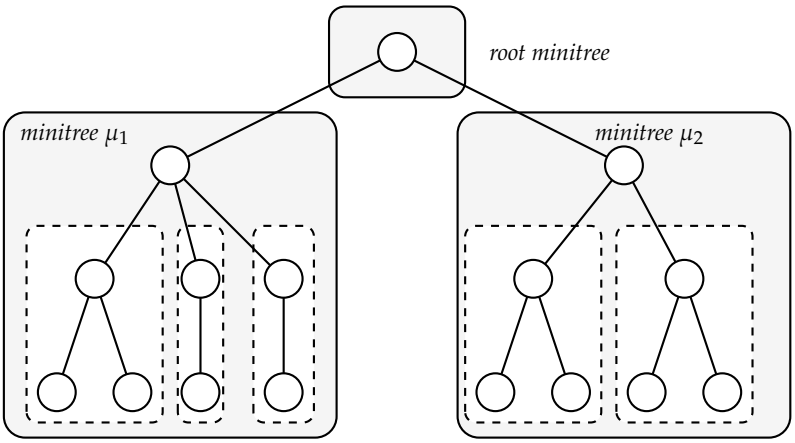

Figure 3.4: A two-level tree covering of an ordinal tree. Solid rectangles enclose minitrees produced by the covering procedure with parameter M; the recursive application of the same procedure with parameter $M' < M$ partitions each minitree into the microtrees enclosed by dashed rectangles. Two minitrees intersect at most at their common roots, and the same property holds within a minitree for its microtrees. The two minitrees μ_1, μ_2 each have size in $[M, 3M - 4]$ except for an undersized fragment containing the root of the whole tree; their microtrees obey the same bound for the parameter M'.

Lemma 3.3.1 (Tree covering size bound [35]) *For any ordinal tree T on n nodes and any integer $M \geq 2$, the covering procedure produces a partition of the nodes of T into pieces, each of which is a connected subgraph of T, such that two pieces intersect at most at their common root. Every piece A has $|A| \leq 3M - 4$, and $|A| \geq M$ unless A contains the root of T. Consequently the number of pieces is $\Theta(n/M)$.*

The size cap $3M - 4$ is what makes microtrees small enough for a catalog while keeping the number of pieces $\Theta(n/M)$. The lower bound M rules out pathological splits and is essential for the count of pieces to be sublinear.

A node x of T that lies on the boundary of two pieces is shared at the level of the partition but is encoded in only one piece, with a record kept for the conversion between x's position in the piece and its preorder index in T. The conversion is $O(1)$ and is treated as a black box for the purposes of the construction.

The choice $M' = \lceil \lg n/24 \rceil$ at the second level is calibrated so that the number of distinct microtree shapes is sublinear in n. Each microtree has at most $\lceil \lg n/8 \rceil$ nodes. The elementary Catalan bound on ordinal trees of size at most M', evaluated at this parameter, gives $C_{M'} \leq 4^{M'} = 2^{2\lceil \lg n/24 \rceil} = O(n^{1/12})$. He, Munro, and Rao state the looser bound $n^{1/4}$ on the number of distinct extended microtree shapes, which subsumes the auxiliary structure attached to each microtree by the extension procedure. Each microtree is identified by its shape from a universe of size $n^{1/4}$, and the choice of M' guarantees that a precomputed table indexed by (shape, in-microtree query) fits in $o(n)$ bits.

The catalog argument is the workhorse. For every microtree shape σ and every operation `op` in Definition 3.1.1, precompute a table $T_\sigma[\texttt{op}, x]$ indexed by an in-microtree node x, returning the answer to $\texttt{op}(x)$ when the query and answer both lie inside the microtree containing x. He, Munro, and Rao establish, by the joint accounting on the $n^{1/4}$ shapes and the $\lceil \lg n/8 \rceil$ nodes per microtree, that the union of these tables occupies $o(n)$ bits. The same lookup-table technique that powers Theorem 2.1.1 appears here at the microtree level, with the microtree shape playing the role of the in-block bit pattern.

Cross-microtree operations are handled by a tier of auxiliary bitvectors and macro trees built on the boundaries. A bitvector of length n marks the positions of tier-1 *preorder changers*, the at most $O(n/\lg^4 n)$ nodes

whose preorder predecessor lies in a different minitree; a second bitvector of the same length marks tier-2 preorder changers, at most $O(n/\lg n)$ nodes whose predecessor lies in a different microtree. Both bitvectors are stored in the compressed form of Theorem 2.1.7, which gives $O(1)$ rank and select on a sparse bitvector at $o(n)$ space. A query that crosses microtree boundaries reduces to a constant number of rank and select calls on these markers, one catalog lookup, and a recursive query at the minitree level. The minitree level repeats the pattern: a macro tree on the $O(n/\lg^4 n)$ minitree roots is preprocessed for $O(1)$ ancestor and LCA via the Bender-Farach-Colton reduction [33], taking $O(n/\lg^4 n \cdot \lg n) = o(n)$ bits.

[33]: Bender et al. (2000), *The LCA Problem Revisited*

Theorem 3.3.2 (Tree covering succinct [36]) *An ordinal tree on n nodes admits a representation in $2n + o(n)$ bits that supports every operation of Definition 3.1.1 in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$. The same representation supports, in $O(1)$ time within the same space bound, the additional operations*

- `height`(x)*, the height of the subtree rooted at x.*
- `leftmost_leaf`(x) *and* `rightmost_leaf`(x)*, the leftmost and rightmost leaves of the subtree rooted at x.*
- `leaf_rank`(x) *and* `leaf_select`(i)*, mapping leaves to their leaf-only positions and back.*
- `level_leftmost`(d) *and* `level_rightmost`(d)*, the leftmost and rightmost nodes at depth d.*
- *the postorder rank and select operations, and the DFUDS rank and select counterparts of* `preorder_rank` *and* `preorder_select`*.*

Sketch. The space bound is $2n + o(n)$ bits inherited from the encoding of extended microtrees of Geary, Raman, and Raman [35], where each microtree is stored verbatim by an index into the catalog of shapes plus the local structure needed for cross-microtree edges. The catalog of microtree-local lookup tables, the rank/select dictionaries on tier-1 and tier-2 preorder changers, and the macro trees on minitree and microtree roots are each $o(n)$ bits.

[35]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*

Query reduction. Each operation `op`(x) on a node x is dispatched in three cases. If `op`(x) is local to the microtree of x, the catalog table answers it in one lookup. If the answer crosses a microtree boundary but stays inside the minitree, a constant number of rank and select calls on the tier-2 changer bitvector identify the relevant microtree on the other side and forward the query to its catalog. If the answer crosses a minitree boundary, the same two-step reduction is applied at the minitree level, using the tier-1 changer bitvector and the macro tree on minitree roots. Operations such as `height` and `LCA` that reduce to a range minimum or maximum on the conceptual depth array are answered by a hierarchy of range-extremum tables, each of size $o(n)$, that exploits the fact that the answer is either local to a microtree, or determined by the per-microtree extremes recorded at the higher level.

The full development requires the careful definition of τ-names, extended microtrees, and the auxiliary arrays for tier-1 and tier-2 preorder changers. The construction is treated in detail in [36]. □

[36]: He et al. (2007), *Succinct Ordinal Trees Based on Tree Covering*

The decomposition predates Theorem 3.3.2 and is a result in its own right. Geary, Raman, and Raman introduced it to obtain a $2n + o(n)$-bit representation that supports `level_ancestor` in $O(1)$ time, an operation that LOUDS, BP, and DFUDS all left out. Farzan and Munro later proposed the same two-level decomposition as a uniform paradigm encompassing ordinal, cardinal, and free trees. Theorem 3.3.2 extends the Geary, Raman, and Raman construction by augmenting it with the auxiliaries needed for every operation of Definition 3.1.1, while keeping the asymptotic space bound.

Geary, Raman, and Raman [35] introduced the two-level cover for the level-ancestor problem; Farzan and Munro [37] generalized the paradigm to cardinal and free trees. Theorem 3.3.2 extends the Geary et al. representation to the full operation list of Definition 3.1.1 and beyond.

A second route to the same bound replaces the catalog of microtree shapes with a single auxiliary tree mounted on the BP sequence, and reduces every operation of Definition 3.1.1 to one path traversal of that tree. The reduction is uniform across the primitives `findclose`, `findopen`, `enclose`, `rmq`, and the level-ancestor query of `level_ancestor`, all of which dispatch to the same primitive on the same auxiliary, instead of consulting the per-primitive hierarchies of Theorem 3.2.1.

The auxiliary is the *range min-max tree*. Let $B[1..2n]$ be the BP sequence of T, and let $e(B, i)$ be the excess at position i of Section 3.2. For pedagogical clarity we describe the construction in its simplest form. Partition B into blocks of length $\Theta(\lg n)$ and build a complete binary tree whose $\Theta(n/\lg n)$ leaves are the blocks of B, with each internal node storing the minimum and maximum excess on the leaves below it. Sadakane and Navarro present the actual construction as a k-ary complete tree with arity $k = \Theta(w/(c \lg w))$ and chunks of size $w/2$, where w is the machine word and c is the parameter governing the redundancy bound; they store three integer arrays e', m', M' holding the per-chunk last excess, minimum, and maximum, occupying $O(nc \lg w/w)$ bits each, and the leaf-level step is resolved by a universal lookup table of $O(\sqrt{2^w} \lg w)$ bits. The total auxiliary space is $O(n/\lg^c n)$ bits for any constant $c > 0$.

The five primitives `findclose`, `findopen`, `enclose`, `rmq`, `level_ancestor` are each cast as a search on the excess sequence for the smallest $j > i$, or the largest $j < i$, with a target excess value. The search starts at the leaf of the range min-max tree containing i, walks upward until a sibling subtree records a min or max that includes the target, then walks downward into that subtree to localize the answer. A precomputed lookup table on $\Theta(\lg n)$-bit block patterns answers the in-leaf step in constant time. With the polylogarithmic case handled directly by the table and the general case reduced to it through the tree, every primitive runs in $O(c)$ time on the word RAM, where $c > 0$ is the constant fixing the redundancy bound.

Theorem 3.3.3 (Range min-max tree [38]) *Let $B[1..2n]$ be the balanced parenthesis sequence of an ordinal tree T on n nodes. For any constant $c > 0$, the BP sequence augmented with the range min-max tree occupies $2n + O(n/\lg^c n)$ bits and supports every operation of Definition 3.1.1 in $O(c)$ time on the word RAM with word size $\Theta(\lg n)$. The same approach extends to a dynamic ordinal tree on n nodes that supports every operation of Definition 3.1.1, together with insertions and deletions, in $O(\lg n)$ time per operation, within $2n + O(n/\lg n)$ bits of space.*

The proof of Theorem 3.3.3 is constructive. Each primitive resolves to one root-to-leaf walk of the range min-max tree, with the leaf step answered

by the lookup table on $\Theta(\lg n)$-bit blocks. The dynamic extension is the principal advantage over Theorem 3.3.2; the two-level catalog of the tree-covering scheme is rigid against insertion and deletion, while the range min-max tree admits per-node updates by recomputing the affected min-max summaries along a single root-to-leaf path. Sadakane and Navarro [38] present a finer variant, beyond the one stated in Theorem 3.3.3, that supports most operations in $O(\lg n/\lg\lg n)$ time within $2n + O(n \lg\lg n/\lg n)$ bits, trading a slightly larger redundancy for the matching lower bound on update time.

[38]: Navarro et al. (2014), *Fully Functional Static and Dynamic Succinct Trees*

The two theorems close the chapter. Theorem 3.3.2 and Theorem 3.3.3 both attain $2n + o(n)$ bits with constant query time for every operation of Definition 3.1.1, the first achieving $O(1)$ outright and the second $O(c)$ for any constant $c > 0$ governing the redundancy. They reach this bound by two genuinely different routes, a catalog of microtree patterns built on a recursive cover and a single auxiliary tree on the excess sequence. The first is the natural endpoint of the line that started with Geary, Raman, and Raman; the second is the modern canonical reference and is the construction of choice when dynamic updates matter. The toolkit for unlabeled ordinal trees is now complete. The next section turns to the question of what changes when each node carries a label drawn from an alphabet Σ.

3.4 Labeled trees and α-operations

The encodings developed so far address ordinal trees in which a node is identified by its position in some traversal order and nothing else. Every operation of Definition 3.1.1 returns or consumes a position; no symbol travels with the node. Theorem 3.3.2 and Theorem 3.3.3 both close the unlabeled question with $2n + o(n)$ bits and $O(1)$ time. The natural next step is the case in which each node carries a label drawn from an alphabet Σ of size σ. Once labels enter, every navigational primitive splits into a label-aware variant: instead of asking for the parent of x, one asks for the closest ancestor of x whose label is a specified $\alpha \in \Sigma$, and instead of counting all descendants of x, one counts the descendants whose label is α. Setting $\sigma = 1$ recovers the unlabeled case as a degenerate instance, so the labeled question subsumes the unlabeled one.

[39]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

We follow the terminology of He, Munro, and Zhou [39], who write *α-node* for a node whose label is α and refer to the label-aware variants as *α-operations*. The α-rank of a node x in a list is the number of α-nodes to the left of x in that list. The five α-operations below are the trees analog of the rank_c and select_c primitives that drive the wavelet tree of Section 2.3 on a sequence over an alphabet of size σ.

Definition 3.4.1 (α-operations on labeled ordinal trees) *Let T be an ordinal tree on n nodes whose nodes are labeled with symbols from an alphabet Σ of size σ. For every $\alpha \in \Sigma$ and every node $v \in T$, a representation of the labeled tree should support the following operations.*

- $\texttt{parent}_\alpha(v)$, *the closest α-labeled ancestor of v (undefined when no α-ancestor exists).*
- $\texttt{depth}_\alpha(v)$, *the number of α-labeled nodes on the path from v to the root, counting v itself when its label is α.*

- `child_select`$_\alpha(v, i)$, *the i-th α-labeled child of v in left-to-right order.*
- `nbdesc`$_\alpha(v)$, *the number of α-labeled nodes in the subtree rooted at v.*
- `level_anc`$_\alpha(v, d)$, *the α-labeled ancestor w of v with* `depth`$_\alpha(v) -$ `depth`$_\alpha(w) = d$.

The unlabeled operations of Definition 3.1.1 are the special case $\sigma = 1$, in which every node is an α-node for the unique label.

The five α-operations above are the trees analog of the rank$_c$ and select$_c$ primitives of Section 2.3: each one specializes a navigational query on the structure of T to the subset of nodes labeled with a given symbol $\alpha \in \Sigma$.

Two routes from Section 3.3 to a labeled representation are immediate. The first relabels T into a 0/1-labeled tree once for every $\alpha \in \Sigma$, marking the α-nodes with 1 and the rest with 0, and stores one copy of Theorem 3.3.2 or Theorem 3.3.3 per copy. Each copy reduces an α-operation to an unlabeled operation on its own marked tree, so each α-operation runs in $O(1)$ time. The bit cost of this scheme is $\sigma \cdot (2n + o(n)) = \Theta(\sigma n)$ bits, dominated by the σn term as soon as σ is non-constant. The second route stores the labels in an explicit array of $\lceil \lg \sigma \rceil$ bits per node, alongside one copy of Theorem 3.3.2 or Theorem 3.3.3 for the structure. The structure-plus-labels storage is $n \lg \sigma + 2n + o(n)$ bits, but resolving a single α-operation now requires a sequential scan of the label array along the navigational path of T, which is $\Omega(\lg n)$ per query in the worst case for `level_anc`$_\alpha$ and `nbdesc`$_\alpha$. Neither route attains the natural target of $n \lg \sigma + O(n)$ bits with sublogarithmic query time.

The information-theoretic lower bound for an n-node ordinal tree whose nodes are labeled with symbols from an alphabet of size σ is $n(\lg \sigma + 2) - O(\lg n)$ bits, by combining the unlabeled bound of Theorem 3.1.1 with the $\lg(\sigma^n) = n \lg \sigma$ bits required to store one label per node. A representation that meets this lower bound to within a low-order term, while supporting every α-operation of Definition 3.4.1 in $O(1)$ time, would close the labeled question in the same sense as Theorem 3.3.2 closed the unlabeled one. Geary, Raman, and Raman achieve $O(1)$ time within a controlled additive overhead, and their bound is the technical centerpiece of the section.

Theorem 3.4.1 (GRR labeled trees [35, 39]) *An ordinal tree on n nodes whose nodes carry labels from an alphabet Σ of size σ admits a representation in*

$$n(\lg \sigma + 2) \;+\; O\left(\frac{\sigma\, n \;\lg\lg\lg n}{\lg\lg n}\right)$$

bits that supports the α-operations of Definition 3.4.1, together with the labeled counterparts of the preorder and postorder rank *and* select, *in $O(1)$ time on the word RAM with word size $\omega = \Omega(\lg n)$.*

Sketch. The construction modifies Theorem 3.3.2 to integrate the labels into the catalog of microtree shapes. Reduce the second-level parameter to $M' = \max(\lceil \lg n/(24 \lg \sigma) \rceil, 2)$ so that the labels of a type-1 microtree fit alongside its tree structure within $\lceil |\mu| \lg \sigma \rceil + 2|\mu|$ bits while keeping the per-microtree lookup table at sublinear total size. For each type-2 microtree, store σ extra bitvectors, the α-th of which marks the nodes of the microtree labeled α. At the canonical root of each microtree, store further bitvectors encoding the distribution of children by label and the per-label skip pointers needed to handle `parent`$_\alpha$ and `level_anc`$_\alpha$ across microtree boundaries. The per-microtree label-aware bookkeeping sums to $o(n)$ bits over all microtree roots once M' is set as above. Macro level. Replicate the macro tree on minitree roots once per $\alpha \in \Sigma$, with the weight

of an edge between two macro nodes equal to the number of α-labeled nodes between them in preorder. The weights are polylogarithmic in n, so the polylog-weight level-ancestor algorithm answers $\texttt{level_anc}_\alpha$ on each copy in $O(1)$ time. The σ copies of the macro tree contribute the $O(\sigma n \lg\lg\lg n/\lg\lg n)$ additive overhead of the bound. The remaining α-operations reduce to a constant number of in-microtree lookups, rank/select calls on the per-label microtree bitvectors, and a single macro-level query. □

Theorem 3.4.1 settles the labeled question for tiny alphabets. The additive term $O(\sigma n \lg\lg\lg n/\lg\lg n)$ is $o(n(\lg\sigma + 2))$ exactly when $\sigma = o(\lg\lg n)$, in which regime the representation matches the lower bound to within a low-order correction. The bound becomes uninformative for larger alphabets. When σ is, say, $\sqrt{n}$ or $n^{\Omega(1)}$, the additive term $\Theta(\sigma n \lg\lg\lg n/\lg\lg n)$ already exceeds the trivial $n \lg\sigma$ baseline of storing the labels in an array, so the construction loses its only space advantage and the relevant question is open.

The threshold $\sigma = o(\lg\lg n)$ is what makes the additive overhead $o(n \lg\sigma)$ in Theorem 3.4.1. For σ polynomial in n, the overhead $\Theta(\sigma n \lg\lg\lg n/\lg\lg n)$ dwarfs the entropy term $n \lg\sigma$ and the bound ceases to be informative.

The set-collection scenarios sketched at the opening of this chapter, in which labels are drawn from very large alphabets that may grow polynomially in n, fall outside the reach of Theorem 3.4.1 for exactly this reason. The open question, posed by Geary, Raman, and Raman in their concluding remarks and answered by He, Munro, and Zhou [39], is whether a representation exists that occupies $nH_0(PLS_T) + O(n)$ bits, where $H_0(PLS_T)$ is the zeroth-order empirical entropy of the preorder label sequence of T and is bounded above by $\lg\sigma$, while supporting the α-operations of Definition 3.4.1 in $O(1)$ time when $\sigma = O(\text{polylog}(n))$ and in $O(\lg\lg\sigma)$ time for general Σ. The next chapter develops the answer.

[39]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

Tree Extraction and Labeled Trees 4

4.1 Tree extraction 43
4.2 α-operations on labeled trees 46
4.3 Path queries 49

The construction of Theorem 3.4.1 answers the labeled-tree question in a regime where the alphabet is tiny, and leaves the thesis without a usable representation once the labels are drawn from a universe of polynomial size. The α-operations of Definition 3.4.1 motivated that bound, and the same operations motivate the present chapter. The technique that breaks the $\Theta(\sigma n)$ barrier is tree extraction, a deletion primitive lifted from tree edit distance and turned into a space-efficient projection of an ordinal tree onto a distinguished subset of its nodes.

Tree extraction delivers what the labeled-tree representation of Theorem 3.4.1 could not: a space term that depends on the entropy of the labels rather than on the size of the alphabet, and navigational primitives whose time is sublogarithmic in u.

4.1 Tree extraction

The representation we seek must map every α-operation on a labeled ordinal tree T to a small number of operations on auxiliary structures whose size depends on n and on the entropy of the labels, not on σ and n jointly. A primitive that isolates, for each label α, the part of T relevant to that label, and does so without storing a separate tree per label, is the technical device that realises the reduction. Tree extraction is that primitive. Given a subset X of the nodes of T containing the root, the X-extraction of T is a new ordinal tree T_X on exactly the nodes of X, whose shape encodes the ancestor, order, and depth relations among those nodes inherited from T.

Definition 4.1.1 (X-extraction) *Let T be an ordinal tree on n nodes and let $X \subseteq V(T)$ be a subset containing the root. The X-extraction of T is the ordinal tree T_X obtained from T by deleting, one by one in level order, every node $v \notin X$. When v is deleted, the children of v are inserted into the list of children of the parent of v, in place of v, preserving their original left-to-right order among themselves. The deletion leaves a tree on exactly the nodes of X, and the assignment $v \mapsto v$ is a natural bijection between $V(T) \cap X$ and $V(T_X)$.*

The deletion rule above is the delete operation of the tree edit distance of Tai [40], who gave the three edit operations on ordered labeled trees (change, delete, insert) and noted that the deletion of a non-root node carries with it the edge to the parent while the subtree below slides up into the vacated slot. Bille's survey of tree edit distance [41] records the rule in the form we have adopted, in which the children of the deleted node are inserted into the parent's child list as a contiguous left-to-right subsequence. The recasting of the rule as a data-structure primitive, with the preservation properties it enables, is due to He, Munro, and Zhou [42], whose ISAAC 2011 paper on path queries in weighted trees states three preservation lemmas on the X-extraction and uses them to reduce path queries to depth queries on extracted forests. The same three lemmas reappear, with proofs, in the TALG 2016 version of the work [43]. Figure 4.1 shows the operation on a concrete twelve-node tree.

Tai's 1979 delete operation removes a node together with its edge to the parent and re-attaches the orphaned children at the same slot. Bille's survey writes the rule in the form we use here, promoting the children as a left-to-right subsequence; He, Munro, and Zhou recast it as a data-structure primitive on ordinal trees.

[40]: Tai (1979), *The tree-to-tree correction problem*

[41]: Bille (2005), *A survey on tree edit distance and related problems*

[42]: He et al. (2011), *Path queries in weighted trees*

[43]: He et al. (2016), *Data structures for path queries*

We now record the three preservation properties of T_X that justify calling it a structural projection of T onto the nodes of X. The first says that

The definition fixes the deletion order as level order for concreteness, but He, Munro, and Zhou [43] observe that T_X does not depend on the order in which non-X nodes are deleted. Any sequence that respects the bottom-up or level-order convention produces the same tree.

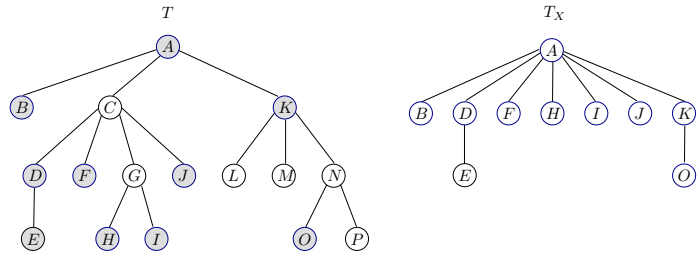

Figure 4.1: X-extraction on a twelve-node ordinal tree, reproduced from Gagie, He, and Navarro [9]. On the left, T with $X = \{A, B, D, E, F, H, I, J, K, O\}$ drawn as the shaded nodes; on the right, the extracted tree T_X. The deletion of each unshaded node promotes its children into the sibling list of its parent, and the natural bijection between X and $V(T_X)$ sends every shaded node to the position it occupies on the right.

ancestry transfers between T and T_X; the second that preorder and postorder transfer; the third identifies the change in depth of an X-node with the number of non-X nodes on its path to the nearest X-ancestor.

Lemma 4.1.1 (Ancestry preservation [43]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root of T. For every pair $u, v \in X$, the node u is an ancestor of v in T if and only if u is an ancestor of v in T_X.*

Proof. Fix a level-order enumeration $z_1, z_2, \ldots, z_k$ of the nodes of T not in X, and write $T^{(0)} = T$ and $T^{(i)} = T^{(i-1)}$ with z_i deleted, so that $T^{(k)} = T_X$. We show, by induction on i, that ancestry among the nodes of X is the same in $T^{(i)}$ as in T. The claim is immediate for $i = 0$.

Assume the claim for $i - 1$ and consider the single deletion of $z = z_i$ in $T^{(i-1)}$. Let p be the parent of z in $T^{(i-1)}$ and let $c_1, \ldots, c_d$ be the children of z in $T^{(i-1)}$, in left-to-right order. After the deletion, p acquires $c_1, \ldots, c_d$ as consecutive children in place of z, and every other parent-child edge is untouched. Take $u, v \in X$ with u an ancestor of v in $T^{(i-1)}$. The unique path from v up to u in $T^{(i-1)}$ either avoids z, in which case the path is intact in $T^{(i)}$ and u remains an ancestor of v, or passes through z, in which case the sub-path of the form $v \to \cdots \to c_j \to z \to p \to \cdots \to u$ becomes $v \to \cdots \to c_j \to p \to \cdots \to u$ in $T^{(i)}$, a path in $T^{(i)}$ that still makes u an ancestor of v. Conversely, if u is an ancestor of v in $T^{(i)}$ with $u, v \in X$, the promotion rule is the only way a new edge is created, and it merely splices p between z's old position and z's children; walking backwards along the ancestry path in $T^{(i)}$ and reinserting z whenever an edge of the form $p \to c_j$ was produced by the deletion reconstructs a path in $T^{(i-1)}$ witnessing the same ancestry. Both directions extend the claim from $i - 1$ to i, and the induction closes at $i = k$. □

Lemma 4.1.2 (Preorder preservation [43]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root. For every pair $u, v \in X$, u precedes v in the preorder traversal of T if and only if u precedes v in the preorder traversal of T_X.*

Sketch. A preorder traversal of an ordinal tree visits each node strictly between its parent and its next right sibling, recursing into the subtree on the way. A single deletion that promotes the children of z into the sibling list of p at the slot previously occupied by z leaves the preorder position of every surviving node unchanged, as the traversal still enters each c_j immediately after whatever preceded z in p's child list or after the closure of c_{j-1}. Iterating the argument over the level-order deletion sequence of Lemma 4.1.1 yields the preorder statement. □

The postorder traversal of T_X obeys the same invariance. For $u, v \in X$, u precedes v in postorder in T if and only if u precedes v in postorder in T_X, by the dual of the argument above applied to the reversed traversal at each internal node.

Lemma 4.1.3 (Depth identity [43]) *Let T be an ordinal tree and let $X \subseteq V(T)$ contain the root, under the convention that a node is its own 0-th ancestor. For every $v \in V(T)$, write $\mathrm{dep}_X(T, v)$ for the number of ancestors of v in T that belong to X, and let $\mathrm{anc}_X(T, v)$ denote the lowest such ancestor. Let w_X be the image of $w = \mathrm{anc}_X(T, v)$ under the natural bijection between X and $V(T_X)$. Then*

$$\mathrm{dep}_{T_X}(w_X) \;=\; \mathrm{dep}_X(T, v).$$

In particular, when $v \in X$ we have $w = v$, and the identity reads $\mathrm{dep}_{T_X}(v_X) = \mathrm{dep}_X(T, v)$.

Proof. Fix a level-order enumeration $z_1, \ldots, z_k$ of the nodes of T not in X, and write $T^{(0)} = T$, $T^{(i)} = T^{(i-1)}$ with z_i deleted, so that $T^{(k)} = T_X$. Let $w^{(i)}$ denote the image of w in $T^{(i)}$, which is well defined because $w \in X$ is never deleted. We show, by induction on i, the pair of equalities

$$\mathrm{dep}_X(T^{(i)}, v) \;=\; \mathrm{dep}_X(T, v) \qquad \text{and} \qquad \mathrm{dep}_{T^{(i)}}(w^{(i)}) \;=\; \mathrm{dep}_X(T, v).$$

For $i = 0$, every ancestor of v in T is recorded in $T^{(0)} = T$, so the first equality is immediate, and by Lemma 4.1.1 together with the choice of w as the lowest X-ancestor of v, every ancestor of w in T belongs to X, so the depth of w in T equals the number of X-ancestors of v, which is exactly $\mathrm{dep}_X(T, v)$.

Assume the claim for $i - 1$. The deletion of z_i promotes the children of z_i into the sibling list of its parent and leaves all other parent-child edges untouched. Since $z_i \notin X$, it contributes nothing to the X-ancestor count of v, so $\mathrm{dep}_X(T^{(i)}, v) = \mathrm{dep}_X(T^{(i-1)}, v)$. For the second equality, the depth of $w^{(i)}$ in $T^{(i)}$ drops by one relative to $T^{(i-1)}$ exactly when z_i is a proper ancestor of $w^{(i-1)}$ in $T^{(i-1)}$; but every proper ancestor of $w^{(i-1)}$ in $T^{(i-1)}$ lies on the path from $w^{(i-1)}$ to the root of $T^{(i-1)}$, and by ancestry preservation these are the ancestors of w in T, all of which belong to X and hence are not z_i. So the depth of $w^{(i)}$ in $T^{(i)}$ equals the depth of $w^{(i-1)}$ in $T^{(i-1)}$, which by induction equals $\mathrm{dep}_X(T, v)$.

At $i = k$, $T^{(k)} = T_X$ and $w^{(k)} = w_X$, so $\mathrm{dep}_{T_X}(w_X) = \mathrm{dep}_X(T, v)$. □

Lemma 4.1.1, Lemma 4.1.2, and Lemma 4.1.3 together make T_X a structural projection of T onto X. Ancestry is preserved, so any navigational query on T that only consults ancestor or descendant relations among X-nodes can be answered on T_X. Preorder and postorder are preserved, so subtree-rank and subtree-select primitives on X-nodes transfer. The depth identity equates the number of X-ancestors of a node in T with the depth of its image in T_X, which is the identity the binary partition of Section 4.3 exploits to reduce path counting to depth queries on extracted forests. These three properties recur as building blocks in the two representations developed in the remainder of this chapter.

4.2 α-operations on labeled trees

The three preservation lemmas of Section 4.1 make the extracted tree T_X a structural projection of the ordinal tree T onto the subset $X \subseteq V(T)$. Ancestry transfers in both directions, preorder and postorder transfer, and the difference in depth between an X-node and its lowest X-ancestor counts the non-X nodes between them. We now put this projection to work on the α-operations problem that Section 3.4 left open, where the representation of Theorem 3.4.1 had forced a $\Theta(\sigma n)$ term that the alphabet sizes of interest in this thesis cannot absorb.

Definition 4.2.1 (α-operations on the ancestor axis) *Let T be an ordinal tree on n nodes, each node carrying a label from $[1..u]$. For a node v of T, a label $\alpha \in [1..u]$, and an integer $i \geq 1$, we define three α-operations on the path from v to the root:*

- $\texttt{parent}_\alpha(v)$*: the lowest ancestor of v whose label is α, or $\perp$ if no ancestor of v carries label α.*
- $\mathrm{rank}_\alpha(v)$*: the number of ancestors of v whose label is α, with the convention that the root counts as an ancestor of v when v is the root.*
- $\mathrm{select}_\alpha(v, i)$*: the i-th ancestor of v with label α, counted from the top down along the root-to-v path, or $\perp$ if fewer than i such ancestors exist.*

He, Munro, and Zhou [39] list a larger family of α-operations, including $\texttt{depth}_\alpha$, $\texttt{level_anc}_\alpha$, and descendant-axis queries. We restrict the exposition to the three ancestor-axis operations required by the set-collection constructions of later chapters; the descendant-axis operations reduce to the same machinery.

For every label $\alpha \in [1..u]$, we build an extracted tree T_α that retains only the nodes of T whose behaviour is relevant to α. Let X_α consist of a fresh root r_α, every node of T whose label is α, and every parent of such a node, where r_α is attached as the sole parent of the original root of T in the augmented tree. The X_α-extraction of the augmented tree, as given by Definition 4.1.1, is the desired T_α. Each node of T_α carries a single bit of label information, namely the marker 1 if its corresponding node in T is an α-node, and the marker 0 otherwise; in particular the root r_α of T_α always carries marker 0. The combined size of the family $\{T_\alpha\}_{\alpha \in [1..u]}$ is linear in n, since each α-node of T contributes one 1-marked node and at most one further 0-marked parent to T_α, while the fresh root r_α adds a single 0-marked node, so $|T_\alpha| \leq 2n_\alpha + 1$ and hence $\sum_{\alpha \in [1..u]} |T_\alpha| \leq 2n + u$, where n_α counts the α-nodes of T. By Lemma 4.1.1 the α-ancestors of a node $v \in V(T)$ correspond bijectively, through the natural injection of $V(T) \cap X_\alpha$ into $V(T_\alpha)$, to the 1-marked ancestors of the image $v' \in V(T_\alpha)$ of the nearest X_α-node on or above v. The three α-operations on T reduce to unlabeled ancestor operations on T_α restricted to marker 1.

Storing one tree per label is wasteful in constants. We concatenate the family into a single ordinal tree $\mathcal{T}$ with a fresh root $\mathcal{R}$, whose children are the roots of $T_1, T_2, \ldots, T_u$ in that order. The merged tree $\mathcal{T}$ has at most $2n + u$ nodes, and the preorder traversal of each T_α occurs as a contiguous substring of the preorder traversal of $\mathcal{T}$; the same holds for the DFUDS traversal once the new root r_α of each T_α is removed. In parallel we concatenate the leaf-indicator bitvectors $L_\alpha[1..n_\alpha]$, whose i-th bit is 1 iff the i-th 1-marked node in preorder of T_α corresponds to a leaf of T, into a single bitvector $\mathcal{L}$ of length n, ordered by $\alpha = 1, 2, \ldots, u$. In the representation we build below, $\mathcal{T}$ plays the role of a single 0/1-labeled ordinal tree on which we support α-operations in the marker alphabet $\{0, 1\}$, as Figure 4.2 makes concrete on a five-node example.

The representation rests on a decomposition of the workload across four building blocks, in the form given by He, Munro, and Zhou for general labeled ordinal trees.

Lemma 4.2.1 (Framework decomposition [39]) *Let T be an ordinal tree on n nodes with labels drawn from $[1..u]$, let $PLS_T[1..n]$ be the preorder label sequence of T, and let $\mathcal{T}$ and $\mathcal{L}$ be the merged tree and merged leaf-indicator bitvector built above. Suppose that four data structures are available:*

- $\mathcal{D}_1$**:** *an unlabeled ordinal tree on the shape of T, occupying $\mathcal{S}_1(n)$ bits and supporting the standard unlabeled navigational operations on T in constant time;*
- $\mathcal{D}_2$**:** *a representation of PLS_T as a string over $[1..u]$, occupying $\mathcal{S}_2(PLS_T)$ bits and supporting* rank_α *and* select_α *for every $\alpha \in [1..u]$;*
- $\mathcal{D}_3$**:** *a 0/1-labeled ordinal tree on $\mathcal{T}$, occupying $\mathcal{S}_3(2n+u)$ bits and supporting the α-operations in the marker alphabet $\{0,1\}$ together with the unlabeled navigational operations in constant time;*
- $\mathcal{D}_4$**:** *the bitvector $\mathcal{L}$ of length n, occupying $\mathcal{S}_4(n)$ bits and supporting* rank_b *and* select_b *for $b \in \{0,1\}$ in constant time.*

The four structures together represent T in $\mathcal{S}_1(n)+\mathcal{S}_2(PLS_T)+\mathcal{S}_3(2n+u)+\mathcal{S}_4(n)$ bits and support every α-operation of Definition 4.2.1 with a constant number of calls to the operations of $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4$.

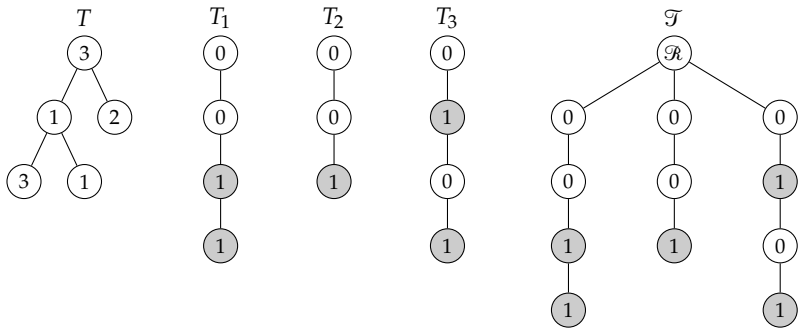

Figure 4.2: From left to right, an ordinal tree T on five nodes with labels in $\{1,2,3\}$; the three extracted trees T_1, T_2, T_3 built on the X_α-extractions described in the text, with 0/1 markers drawn as empty and shaded circles; and the merged tree $\mathcal{T}$ obtained by appending the T_α as subtrees of a fresh root $\mathcal{R}$. The α-operations on T are answered by the 0/1 α-operations on the single tree $\mathcal{T}$, which is the role of the building block $\mathcal{D}_3$ of Lemma 4.2.1.

Proof. We instantiate each $\mathcal{D}_i$, then describe the navigation from T to T_α, and finally reduce each operation of Definition 4.2.1 to a constant number of calls to the four blocks. The unlabeled navigational operations used in $\mathcal{D}_1$ and $\mathcal{D}_3$ include parent, depth, level-ancestor, and lowest-common-ancestor; all of them are supported in constant time by the tree-covering representation of Theorem 3.3.2.

Encoding. We realise $\mathcal{D}_1$ on the shape of T via Theorem 3.3.2, which gives $2n + o(n)$ bits with constant-time unlabeled navigational operations. For $\mathcal{D}_2$ we store PLS_T as a string of length n over $[1..u]$ using the sequence representation of Belazzougui and Navarro [44], which takes $nH_0(PLS_T) + o(nH_0(PLS_T)) + o(n) = O(n \lg u)$ bits and supports rank_α and select_α in $O(\lg \lg_\omega u)$ time under the word RAM with $\omega = \Omega(\lg n)$. This representation builds on the succinct indexes of Barbay, He, Munro, and Srinivasa Rao [45] and the large-alphabet string rank/select of Golynski, Munro, and Rao [46], improving their $O(\lg \lg u)$ time bound to the word-RAM-aware $O(\lg \lg_\omega u)$. For $\mathcal{D}_3$ we apply the 0/1-labeled case of the framework itself, namely Theorem 1 of [39], to the merged tree $\mathcal{T}$. That theorem occupies $O(|\mathcal{T}|) = O(n + u)$ bits and supports the α-operations in the alphabet $\{0,1\}$ in constant time, plus the two

[44]: Belazzougui et al. (2015), *Optimal lower and upper bounds for representing sequences*

[45]: Barbay et al. (2011), *Succinct indexes for strings, binary relations and multi-labeled trees*

[46]: Golynski et al. (2006), *Rank/select operations on large alphabets: a tool for text indexing*

[39]: He et al. (2014), *A framework for succinct labeled ordinal trees over large alphabets*

auxiliary operations $\texttt{deepest}_\alpha$ and $\texttt{min_depth}_\alpha$ of Lemma 4.2.1. The recursion on the framework is not circular because Theorem 1 is proved directly at $\sigma = 2$, where the string index at $\mathcal{D}_2$ reduces to a bitvector and the time blows down to $O(1)$. For $\mathcal{D}_4$ we store $\mathcal{L}$ by the bitvector of Clark and Munro [47], which gives $n + o(n)$ bits with constant-time rank_b and select_b. Summing, the representation occupies $(2n + o(n)) + O(n \lg u) + O(n + u) + (n + o(n)) = O(n \lg u)$ bits, equivalently $O(n)$ words.

[47]: Clark et al. (1996), *Efficient suffix trees on secondary storage (extended abstract)*

Navigation from T to T_α. Fix a node $v \in V(T)$ and a label $\alpha \in [1..u]$. We describe the mapping $f_T(v, \alpha)$ that returns, if it exists, the image in $V(T_\alpha)$ of the closest X_α-node weakly above v. If the label of v is α, then $v \in X_\alpha$ and the mapping returns the image v' of v in T_α, computed by the two-step identity

$$v' \;=\; \text{select}_1\big(T_\alpha,\, \text{rank}_\alpha(PLS_T, \texttt{preorder}(v))\big),$$

where the inner call uses $\mathcal{D}_2$ to count the α-occurrences in $PLS_T[1..\texttt{preorder}(v)]$ and the outer call uses $\mathcal{D}_3$ to locate the corresponding 1-marked node in T_α among the 1-marked nodes of $\mathcal{T}$. The α-successor and α-predecessor of $\texttt{preorder}(v)$ in PLS_T are likewise obtained in $O(\lg \lg_\omega u)$ time through $\mathcal{D}_2$. If the label of v is not α, we test whether v has any α-descendant by reading the α-boundary $[\texttt{pre_rank}(v), \texttt{pre_rank}(v) + \texttt{nbdesc}(v) - 1]$ of the subtree rooted at v in PLS_T through a rank_α query on $\mathcal{D}_2$; if the count is zero, $f_T(v, \alpha) = \perp$. Otherwise let p, q be the first and last α-descendants of v in preorder, computed through $\mathcal{D}_2$. The X_α-image v' of v, when v is not an α-node, is the lowest common ancestor of p' and q' in T_α, which is computed by one call to $\mathcal{D}_3$. The whole mapping uses a constant number of calls to $\mathcal{D}_1$, $\mathcal{D}_2$, and $\mathcal{D}_3$, and runs in $O(\lg \lg_\omega u)$ time because $\mathcal{D}_2$ is the dominating cost.

Reduction of the three α-operations. We answer each operation of Definition 4.2.1 by combining f_T with unlabeled and marker-1 navigation on $\mathcal{T}$. For $\text{rank}_\alpha(v)$, let y equal v if v has label α, and $\texttt{parent}_\alpha(v)$ otherwise. Then $\text{rank}_\alpha(v)$ equals the 1-depth of $y' = f_T(y, \alpha)$ in T_α, which is a single call to the marker-1 depth on $\mathcal{D}_3$, since each α-ancestor of y in T corresponds by Lemma 4.1.1 to a 1-marked ancestor of y' in T_α. For $\texttt{parent}_\alpha(v)$, Lemma 4.1.1 supplies the bijection between α-ancestors of v in T and 1-marked ancestors of $v' = f_T(v, \alpha)$ in T_α; the lowest such ancestor in T_α is $\texttt{parent}_1(T_\alpha, v')$, computed in $O(1)$ through $\mathcal{D}_3$, and its image in T is recovered by one select_α on PLS_T through $\mathcal{D}_2$. For $\text{select}_\alpha(v, i)$ we first test whether i exceeds $\text{rank}_\alpha(v)$; if so, return $\perp$. Otherwise, using the same bijection, the i-th α-ancestor of v from the top corresponds to the 1-level-ancestor $\texttt{level_anc}_1(T_\alpha, v', i)$, computed in $O(1)$ through $\mathcal{D}_3$; its image in T is recovered, as in the previous case, by one select_α on PLS_T through $\mathcal{D}_2$. In all three cases the number of calls to each building block is $O(1)$.

Time bound. Each α-operation costs a constant number of calls to $\mathcal{D}_1, \mathcal{D}_3, \mathcal{D}_4$, each of which runs in $O(1)$ time by the constant-time specifications in the encoding phase, and a constant number of calls to $\mathcal{D}_2$, each of which runs in $O(\lg \lg_\omega u)$ time. The bottleneck is $\mathcal{D}_2$, so the total cost is $O(\lg \lg_\omega u)$. □

Theorem 4.2.2 (α-operations via tree extraction [9, 39]) *Let T be an ordinal tree on n nodes, each carrying a label from $[1..u]$, and assume the word RAM*

model with word size $\omega = \Omega(\lg n)$. There exists a representation of T in $O(n)$ words of space that supports $\texttt{parent}_\alpha(v)$, $\mathrm{rank}_\alpha(v)$, *and* $\mathrm{select}_\alpha(v, i)$ *of Definition 4.2.1 in $O(\lg \lg_\omega u)$ time, and that can be built in $O(n \lg u)$ time.*

Proof. Lemma 4.2.1 gives the decomposition; the four-phase argument of its proof instantiates the building blocks and performs the reduction, yielding the stated space and time bounds. □

The representation of Theorem 4.2.2 answers the ancestor-axis α-operations in time sublogarithmic in the alphabet size and in space proportional to n words, which is the precision the downstream constructions of Cap. 6 require for membership in labeled deletion trees. Path queries that aggregate over ranges of labels rather than a single label are the subject of the next section.

4.3 Path queries

The α-operations of Definition 4.2.1 aggregate information along the root-to-v path when the query is pinned to a single label. Many downstream uses of labeled trees ask a broader question. How many ancestors of v carry a label in a query range $[p..q]$, and which ancestor carries the i-th smallest label. We call these *path queries*, and we restrict attention to the root-to-node version, which is what the set-collection constructions of later chapters need. The construction develops in two steps, each with its own query time bound, and this section covers the first of them.

He, Munro, and Zhou [43] also handle node-to-node paths and path-median queries. The set-collection constructions of later chapters only need the root-to-node case, which already admits the sharper time bounds of the present section.

4.3.1 Binary partition

The closing of Section 2.3 promised that the hierarchical binary partition of the alphabet, which organises rank and select on a sequence through a chain of bitmaps, would transfer from sequences to labeled ordinal trees. We cash the promise in now. The wavelet tree of Grossi, Gupta, and Vitter [27] splits the alphabet $[1..u]$ at each recursive level by a midpoint, stores at every range a bitmap that records, for each symbol of the underlying subsequence, whether it falls in the left or right half of the range, and answers rank_c and select_c through a root-to-leaf traversal whose per-level cost is a single binary rank or select. We replicate this plan on a labeled ordinal tree T on n nodes with labels drawn from $[1..u]$. The alphabet is partitioned by the same range tree, and at every range $[a..b]$ we store a 0/1-labeled extracted tree $T_{a,b}$ whose 0 markers flag the nodes with labels in the left half and whose 1 markers flag those with labels in the right half. Level transitions move between extracted trees through a single rank or select on the marker sequence, in constant time, and the three preservation lemmas of Section 4.1 guarantee that depths and ancestries transfer across the transitions in the form the queries need.

[27]: Grossi et al. (2003), *High-order entropy-compressed text indexes*

Definition 4.3.1 (Path queries) *Let T be an ordinal tree on n nodes with labels in $[1..u]$, and let $v \in V(T)$. For a range $[p..q] \subseteq [1..u]$ and an integer $i \geq 1$, we define two operations on the root-to-v path of T:*

- `path_count`$(v, [p..q])$: *the number of ancestors of v, counted with multiplicity by label, whose label belongs to $[p..q]$.*
- `path_select`(v, i): *the ancestor of v whose label is the i-th smallest among the labels appearing on the root-to-v path. Return $\perp$ if i exceeds the length of the path.*

Under the convention that a node is its own 0-th ancestor, the root-to-v path includes v itself.

The alphabet is organised by a *conceptual range tree* on $[1..u]$. The root range is $[1..u]$. A non-leaf range $[a..b]$ with $a < b$ has two child ranges $[a..m]$ and $[m+1..b]$, where $m = \lfloor (a+b)/2 \rfloor$. The recursion stops at the u leaf ranges of the form $[\alpha..\alpha]$, one for every value $\alpha \in [1..u]$. The conceptual range tree has height $\lceil \lg u \rceil$ and is balanced up to a unit imbalance in sibling lengths. No material storage is devoted to the range tree itself; each range is an abstract index under which we store the labeled tree associated with it.

For each range $[a..b]$ in the conceptual range tree we store an extracted ordinal tree $T_{a,b}$. Let $R_{a,b}$ denote the set of nodes of T whose label lies in $[a..b]$, and let $F_{a,b}$ be the $R_{a,b}$-extraction of T in the sense of Definition 4.1.1, where $R_{a,b}$ is augmented by the root of T to meet the root-inclusion requirement of the definition. The object $T_{a,b}$ is $F_{a,b}$ with a dummy root $\rho_{a,b}$ attached above, following He, Munro, and Zhou [43], so that $T_{a,b}$ is a genuine ordinal tree rather than a forest and its nondummy nodes correspond bijectively to the nodes of $R_{a,b}$. At the top of the recursion, $T_{1,u}$ contains every node of T under the dummy root, and its preservation properties reduce to those of T itself. At every nonleaf range $[a..b]$ with children $[a..m]$ and $[m+1..b]$, each nondummy node of $T_{a,b}$ carries a single marker bit. The marker is 0 if the label of the corresponding node of T lies in $[a..m]$ and 1 if it lies in $[m+1..b]$. The child trees $T_{a,m}$ and $T_{m+1,b}$ are, up to a relabelling of the dummy root, the extractions of $T_{a,b}$ onto its marker-0 and marker-1 nondummy nodes, respectively, under the root-extraction convention that the dummy root is preserved. At a leaf range $[\alpha..\alpha]$, $T_{\alpha,\alpha}$ collects the nodes of T whose label equals α. The full hierarchy of extracted trees, displayed in Figure 4.3, makes the level-by-level transition visible.

[43]: He et al. (2016), *Data structures for path queries*

Each $T_{a,b}$ is accessed through three navigation primitives on every node, namely the depth of the node within $T_{a,b}$, a link back to the corresponding node of T, and a pair of links leading to the corresponding images in the two children $T_{a,m}$ and $T_{m+1,b}$. In the succinct encoding each primitive is realised as a marker-0 or marker-1 α-operation of Definition 4.2.1 applied to $T_{a,b}$, delivered in $O(1)$ time on the pointer machine by Theorem 4.2.2 at $\sigma = 2$. The abstract rule that justifies the navigation is the preservation machinery of Section 4.1. By Lemma 4.1.1 the image v' in $T_{a,m}$ of a node $v \in V(T_{a,b})$ whose label falls in $[a..m]$ is the natural bijective image of v through the X_0-extraction; if the label of v falls in $[m+1..b]$ instead, v' is the lowest marker-0 ancestor of v in $T_{a,b}$, and the same formula, with marker 1, produces the image in $T_{m+1,b}$. The binary alphabet of markers makes every such transition a single $\texttt{parent}_\alpha$ or rank_α call on $T_{a,b}$.

Theorem 4.3.1 (Path queries on the pointer machine [9, 43]) *Let T be an ordinal tree on n nodes with labels in $[1..u]$. On the pointer machine, T can be represented in $O(n)$ words of space that support the operations*

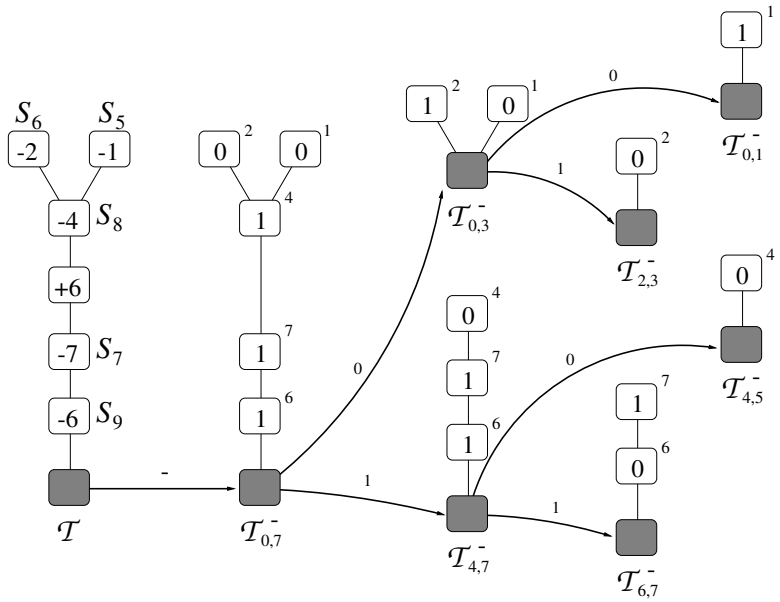

Figure 4.3: Hierarchical binary partition of a labeled ordinal tree for root-to-node path queries, reproduced from Gagie, He, and Navarro [9]. Starting from the root range, each intermediate tree $\mathcal{T}_{a,b}$ is 0/1-labeled by the midpoint of its range. Bold arrows lead to the child trees $\mathcal{T}_{a,m}$ on marker 0 and $\mathcal{T}_{m+1,b}$ on marker 1. The binary alphabet at every level makes each level transition cost $O(1)$ via a single rank or select on the marker sequence.

`path_count`$(v, [p..q])$ and `path_select`(v, i) of Definition 4.3.1 in $O(\lg u)$ time, and that can be built in $O(n \lg u)$ time.

Proof. We instantiate the range tree and the family $\{T_{a,b}\}$ as described above, then analyse space, then describe the two query algorithms, then bound their time.

Encoding. Every $T_{a,b}$ is a 0/1-labeled ordinal tree on $|R_{a,b}| + 1$ nodes, counting the dummy root, where $R_{a,b}$ is the set of nodes of T with label in $[a..b]$. We encode each $T_{a,b}$ through the labeled-tree primitive of Theorem 4.2.2 specialised to alphabet size $\sigma = 2$, which encodes a σ-labeled ordinal tree on N nodes in $O(N \lg \sigma)$ bits and supports the level-transition queries in constant time on the pointer machine. At $\sigma = 2$ this reduces to $O(|R_{a,b}|)$ bits per range. The level-by-level accounting is a telescope. At a fixed level ℓ of the conceptual range tree, the ranges partition $[1..u]$ into at most 2^ℓ disjoint sub-ranges, and accordingly partition the labeled nodes of T into disjoint sets, so

$$\sum_{[a..b] \text{ at level } \ell} |R_{a,b}| = n,$$

and the total size of the trees at level ℓ, including the dummy roots, is $n + O(2^\ell)$. A range of length 1 with no α-labeled node carries only the dummy root and need not be materialised, so we omit empty ranges and every materialised level contributes $O(n)$ bits. Across the $\lceil \lg u \rceil$ levels the partition occupies $O(n \lg u)$ bits in total, which under $\omega \geq \lg n$ is $O(n)$ words. This is the succinct reading of the partition adopted by Gagie, He, and Navarro [9], who treat each $T_{a,b}$ as a bit-level succinct tree to keep the aggregate cost at $O(n)$ words. The original pointer-machine construction of He, Munro, and Zhou [43] materialises each node of $T_{a,b}$ as an $O(1)$-word record holding explicit pointers to T and to the two child trees, which inflates the space to $O(n \lg u)$ words; the word bound stated above matches once one substitutes the bit-level encoding for the pointer record, as we do here.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

Path counting. Given $v \in V(T)$ and a query range $[p..q] \subseteq [1..u]$, we

move from v to its image $v_{1,u}$ in $T_{1,u}$ and recurse on the range tree. The recursion acts on a pair $([a..b], w)$ where $w \in V(T_{a,b})$ is the image of the closest $R_{a,b}$-ancestor of v weakly above v, and returns the number of ancestors of v in T with label in $[a..b] \cap [p..q]$.

Case $[a..b] \subseteq [p..q]$**.** Every label in $[a..b]$ is already in $[p..q]$. We return $\mathrm{dep}_{T_{a,b}}(w)$, excluding the dummy root from the count. By Lemma 4.1.3 applied to the set $X = R_{a,b} \cup \{\text{root of } T\}$ and the node v, the depth of w in the extracted tree equals the number of ancestors of v in T whose label belongs to $[a..b]$, which under the case assumption is the answer contributed by this range.

Case $[a..b] \cap [p..q] = \emptyset$. No ancestor with label in $[a..b]$ qualifies. We return 0.

Case $[a..b]$ properly overlaps $[p..q]$. We recurse on $([a..m], w_0)$ and $([m+1..b], w_1)$, where w_0, w_1 are the images of w in $T_{a,m}$ and $T_{m+1,b}$, each obtained from w in $O(1)$ time by a single $\texttt{parent}_\alpha$ call on $T_{a,b}$ with $\alpha \in \{0,1\}$ under Theorem 4.2.2. We return the sum of the two recursive answers.

Correctness is an induction on the range tree. At $[1..u]$ the recursion is invoked with the image of v and the query range $[p..q]$, and the result is the number of ancestors of v in T with label in $[1..u] \cap [p..q] = [p..q]$, which is exactly $\texttt{path_count}(v, [p..q])$.

Path selection. Given v and i, we drive the same recursion on the range tree, now tracking the rank i of the label to locate. The recursion acts on $([a..b], w, j)$ and returns the ancestor of v whose label is the j-th smallest among the ancestors with label in $[a..b]$, or $\perp$ if no such ancestor exists.

Case $a = b$**.** The range is a singleton $\{\alpha\}$. If $j \le \mathrm{dep}_{T_{\alpha,\alpha}}(w)$, we locate the j-th marker-α ancestor of v by a single select_α call on $T_{a,b}$ and map it back to T through the label bijection. Otherwise we return $\perp$.

Case $a < b$**.** Let $d_L = \mathrm{dep}_{T_{a,m}}(w_0)$ be the number of ancestors of v with label in $[a..m]$. If $j \le d_L$, recurse into $([a..m], w_0, j)$. If $j > d_L$, recurse into $([m+1..b], w_1, j - d_L)$.

The recursion is invoked with $([1..u], v_{1,u}, i)$. Correctness is again a range-tree induction; the decision at each nonleaf range uses the depth identity of Lemma 4.1.3 to count ancestors in $[a..m]$, then subtracts to shift the rank when recursing into the right child.

Time. Path counting touches every range $[a..b]$ on the frontier between ranges contained in $[p..q]$ and ranges disjoint from it. The frontier of a canonical decomposition of a range on a balanced range tree has size $O(\lg u)$, and the recursion spends $O(1)$ time per range through a constant number of α-operations on $T_{a,b}$, so path counting runs in $O(\lg u)$ time. Path selection descends a single root-to-leaf path of the range tree, of length $\lceil \lg u \rceil$, spending $O(1)$ time per range through the same primitive. Both queries run in $O(\lg u)$ time. □

4.3.2 Sublogarithmic refinement

The bound of Theorem 4.3.1 is the tree analogue of the $O(\lg u)$ cost that a wavelet tree pays on sequences, one level of the binary alphabet partition at a time. On sequences, Ferragina, Manzini, Mäkinen, and Navarro [29] refined that cost to $O(\lg u / \lg\lg n)$ under the word RAM,

[29]: Ferragina et al. (2007), *Compressed Representations of Sequences and Full-Text Indexes*

by increasing the branching factor of the partition to $f = \lceil \lg^{\epsilon} n \rceil$ and supporting the resulting f-ary rank and select in constant time per level through packed-word arithmetic. We ask whether the same refinement transfers to labeled ordinal trees, and the answer, due to He, Munro, and Zhou [43], is that it does, subject to two word-RAM primitives on f-labeled sequences that replace binary rank and select.

[43]: He et al. (2016), *Data structures for path queries*

We fix a constant $\epsilon \in (0, 1)$ and set the fanout to $f = \lceil \lg^{\epsilon} n \rceil$. The conceptual range tree on $[1..u]$ is built by splitting each nonleaf range into f subranges of as nearly equal length as possible, following He, Munro, and Zhou [43], so that at level ℓ there are at most $f^{\ell-1}$ ranges and the height is $h = \lceil \log_f u \rceil$. A direct computation gives

Path counting calibrates the fanout to $f = \lceil \lg^{\epsilon} n \rceil$, giving height $\Theta(\lg u / \lg \lg n) = \Theta(\lg_{\omega} u)$. Path selection will instead use $f' = \lceil \lg^{\epsilon'} u \rceil$, giving height $\Theta(\lg u / \lg \lg u)$. The two denominators agree only when u is polynomial in n; outside that regime the two queries genuinely separate.

[43]: He et al. (2016), *Data structures for path queries*

$$h = \frac{\lg u}{\lg f} = \frac{\lg u}{\epsilon \lg \lg n} = \Theta\left(\frac{\lg u}{\lg \lg n}\right).$$

Under the word RAM with $\omega = \Omega(\lg n)$, the quantity $\lg u / \lg \lg n$ is $\Theta(\lg_{\omega} u)$ when $\omega = \Theta(\lg n)$, so $h = \Theta(\lg_{\omega} u)$ and the binary-partition height of Theorem 4.3.1 drops from $\lceil \lg u \rceil$ to $\lceil \log_f u \rceil$. The saving is paid for at every level, where the alphabet of markers is now $[1..f]$. The level transition of Theorem 4.3.1, which cost $O(1)$ via a single rank on a bitvector, has to be replaced by a constant-time operation on an f-labeled sequence.

The first primitive counts, in a sequence over a small integer alphabet, how many entries up to a given position fall below a threshold.

Lemma 4.3.2 (Counting below a threshold [48]) *Let $S[1..n]$ be a sequence over the alphabet $[1..f]$ with $f = O(\lg^{\epsilon} n)$ for some constant $\epsilon \in (0, 1)$. Assume that S is stored in a representation from which any $\lceil \lg^{\lambda} n \rceil$ consecutive entries are accessible in $O(1)$ time, for some constant $\lambda \in (\epsilon, 1)$. There exists an auxiliary structure of $o(n)$ bits that supports, in $O(1)$ time, the query*

$$\texttt{count}_{\beta}(S, i) = |\{\, j \le i : S[j] \le \beta \,\}|$$

for every $\beta \in [1..f]$ and every $i \in [1..n]$.

Sketch. Partition S into blocks of $\lceil \lg^2 n \rceil$ entries and further into subblocks of $\lceil \lg^{\lambda} n \rceil$ entries. Store a two-dimensional array $A[1..n/\lceil \lg^2 n \rceil, 1..f]$ that holds, for each block, the count of entries not exceeding β up to the end of the block, and an analogous array B that holds the same counts within each block up to the end of the subblock. A global lookup table C records, for every possible subblock pattern, every prefix length inside the subblock, and every threshold β, the count of entries of the pattern up to that prefix that are at most β. A query at position i decomposes i as a block index plus a subblock index plus a subblock offset and reads one entry from each of A, B, C in constant time. The space cost of A and B is $o(n)$ bits because $f = O(\lg^{\epsilon} n)$ keeps the second dimension subpolynomial, and the table C is subpolynomial in n because each subblock carries only $O(\lg^{\lambda+\epsilon} n)$ bits of information. □

The second primitive counts, at any position, how many earlier entries carry the same value as the entry at that position.

Lemma 4.3.3 (Constant-time partial rank [49]) *Let $S[1..n]$ be a sequence over the alphabet $[1..f]$. There is a representation of S in $O(n \lg f)$ bits supporting, in $O(1)$ time, the query*

$$\texttt{partial_rank}(S, i) \;=\; |\{\, j \le i : S[j] = S[i] \,\}|.$$

Sketch. The representation is a standard rank-indexed succinct encoding of the sequence, equivalent to the rank structure used as a substructure in ball-inheritance for 2-d range reporting in the word RAM. For every position i, the value $\texttt{partial_rank}(S, i)$ equals the rank of $S[i]$ up to i among all entries equal to $S[i]$, and since the partial rank only counts the symbol already at i it can be stored implicitly, using $O(\lg f)$ bits per position in a packed layout, and answered by one table lookup in constant time. □

[43]: He et al. (2016), *Data structures for path queries*

At each level $\ell \in \{1, 2, \ldots, h\}$ of the f-ary range tree we merge the family $\{T_{a,b}\}$ at that level into a single labeled ordinal tree T_ℓ, following the construction of He, Munro, and Zhou [43]. The ranges at level ℓ partition the n weighted nodes of T into disjoint groups, so the forests $\{F_{a,b}\}$ at level ℓ are node-disjoint; collecting them under a common dummy root in the order imposed by the ranges produces T_ℓ, whose n nondummy nodes (alongside the dummy root) correspond bijectively to the nodes of T. Each nondummy node of T_ℓ carries a label from $[1..f]$, namely the index of the child range at level $\ell + 1$ to which the node descends, so the labels of T_ℓ record the level-ℓ to level-$(\ell + 1)$ transition of every node. We store T_ℓ via Theorem 4.2.2 at alphabet size $\sigma = f$, which consumes $O(|T_\ell| \lg f) = O(n \lg f)$ bits and supports the marker-based α-operations of Definition 4.2.1 in time $O(\lg \lg_\omega f) = O(1)$, because $\omega = \Theta(\lg n)$ and $f = O(\lg^\epsilon n)$ give $\lg_\omega f = (\epsilon \lg \lg n)/\lg \lg n = \Theta(1)$. We also index the preorder label sequence of T_ℓ by Lemma 4.3.2 and Lemma 4.3.3; both fit within the $O(n \lg f)$-bit budget of the primary encoding.

Theorem 4.3.4 (Path queries on the word RAM [43]) *Let T be an ordinal tree on n nodes with labels in $[1..u]$, under the word RAM model with word size $\omega = \Omega(\lg n)$. There exists a representation of T in $O(n)$ words of space that supports the operation $\texttt{path_count}(v, [p..q])$ of Definition 4.3.1 in $O(\lg_\omega u)$ time and the operation $\texttt{path_select}(v, i)$ of Definition 4.3.1 in $O(\lg u / \lg \lg u)$ time.*

[43]: He et al. (2016), *Data structures for path queries*

Proof. The statement combines three results of He, Munro, and Zhou [43]. The linear-space encoding comes from their Theorem 5.1, which realises the range tree of Theorem 4.3.1 through Theorem 4.2.2 rather than through word-record pointers. The sublogarithmic time bounds come from their Lemma 6.6 for path counting and their Theorem 6.9 for path selection, each raising the fanout of the range tree and replacing per-node navigation by packed-word operations. The two queries calibrate the fanout differently, so their encodings and analyses proceed in parallel.

Encoding for path counting. Fix $\epsilon \in (0, 1)$ and set $f = \lceil \lg^\epsilon n \rceil$. The f-ary conceptual range tree on $[1..u]$ has height $h = \lceil \log_f u \rceil$. For each level $\ell \in \{1, \ldots, h\}$ store the merged tree T_ℓ through Theorem 4.2.2 at

alphabet size f, using $O(n \lg f)$ bits since T_ℓ has $n+1$ nodes. Summing over levels,

$$\sum_{\ell=1}^{h} O(n \lg f) \;=\; O(nh \lg f) \;=\; O\left(n \cdot \frac{\lg u}{\lg f} \cdot \lg f\right) \;=\; O(n \lg u) \text{ bits,}$$

which is $O(n)$ words under $\omega = \Omega(\lg n)$. The auxiliary index of Lemma 4.3.2, applied to the preorder label sequence of T_ℓ, contributes $o(n)$ bits per level, hence $o(n \lg_\omega u)$ bits overall. The auxiliary representation of Lemma 4.3.3 contributes $O(n \lg f)$ bits per level, hence $O(n \lg u)$ bits overall. Both are subsumed by the primary encoding.

Path counting. Given $v \in V(T)$ and $[p..q] \subseteq [1..u]$, the recursion of Theorem 4.3.1 runs on the f-ary range tree. On input $([a..b], w)$ at level ℓ, with w the image in $T_{a,b}$ of the deepest $R_{a,b}$-ancestor of v, the recursion returns the number of ancestors of v with label in $[a..b] \cap [p..q]$. If $[a..b] \subseteq [p..q]$, return $\mathrm{dep}_{T_{a,b}}(w)$, which counts the ancestors of v with label in $[a..b]$ by Lemma 4.1.3. If $[a..b] \cap [p..q] = \emptyset$, return 0. Otherwise recurse on the child ranges $[a_1..b_1], \ldots, [a_f..b_f]$ that intersect $[p..q]$.

Each recursive call needs the image w_i of w in the child tree T_{a_i,b_i}. The label of w in T_ℓ identifies which child range w descends into, so the preorder rank of w_i in $T_{\ell+1}$ is obtained from that of w in T_ℓ by a rank_i call on the label sequence of T_ℓ. This call decomposes into one Lemma 4.3.2 difference and one Lemma 4.3.3 correction, each $O(1)$. The depth $\mathrm{dep}_{T_{a,b}}(w)$ in the containment case is computed through Theorem 4.2.2 on T_ℓ in $O(\lg \lg_\omega f) = O(1)$ time, since $\omega = \Theta(\lg n)$ and $f = O(\lg^\epsilon n)$ give $\lg_\omega f = \Theta(1)$. The canonical decomposition of $[p..q]$ visits $O(h)$ ranges, so path counting runs in $O(h) = O(\lg u / \lg \lg n) = O(\lg_\omega u)$ time.

Encoding for path selection. Fix $\epsilon' \in (0,1)$, set $f' = \lceil \lg^{\epsilon'} u \rceil$, and build a separate conceptual range tree on $[1..u]$ with branching factor f' and height $h' = \lceil \log_{f'} u \rceil + 1 = \Theta(\lg u / \lg \lg u)$. At every nonbottom level ℓ, store the merged tree T'_ℓ through Theorem 4.2.2 at alphabet size f'. The telescope $\sum_\ell O(n \lg f') = O(n \lg u)$ bits is $O(n)$ words.

Selection requires one further auxiliary structure per level, an array $X[1..n_1, 1..\Delta]$ where n_1 is the number of tier-1 roots of T'_ℓ under the tree-covering framework of [35, 37] that underlies Lemma 4.3.3, and $\Delta = O(\lg^{\epsilon'} u)$ is the number of sections into which a packed $\lceil \lg n \rceil$-bit counter word is split. For each tier-1 root r_i^1 and section index p, the entry $X[i,p]$ prepends one marker bit to $sect_p(A[r_i^1, \beta])$ for every $\beta \in [1..f']$ and concatenates the f' padded fragments into a single machine word, where $A[r_i^1, \beta]$ counts the β-bounded labels on the path from r_i^1 to the root. The total bit cost is $n_1 \cdot \Delta \cdot O(\lg n) = O(n \lg^{\epsilon'} u / \lg n) = o(n)$ per level. Aggregate space stays $O(n)$ words.

[35]: Geary et al. (2006), *Succinct Ordinal Trees with Level-Ancestor Queries*
[37]: Farzan et al. (2008), *A Uniform Approach Towards Succinct Representation of Trees*

Path selection. Given v and i, descend a single root-to-leaf branch of the f'-ary range tree tracking the residual rank. At a range $[a..b]$ of level ℓ with child ranges $[a_1..b_1], \ldots, [a_{f'}..b_{f'}]$, the recursion must locate the smallest γ for which the ancestors of v with label in $[a..b_\gamma]$ number at least i. By Lemma 4.1.3, this count is $\sum_{j \le \gamma} \mathrm{dep}_{T_{a_j,b_j}}(w_j)$, where w_j is the image of w in T_{a_j,b_j}. The array X packs the sections of these partial sums across the f' children into $O(1)$ machine words, and a constant number of word-parallel comparisons identify two candidates α_1, α_2 with $\gamma \in [\alpha_1, \alpha_2)$. When $\alpha_2 - \alpha_1 \le 1$, γ is read off in $O(1)$; otherwise

a binary search of cost $O(\lg f')$ disambiguates. Across the h' levels the binary search triggers at most $\Delta - 1$ times, and non-search operations stay $O(1)$ per level, so the total time is

$$O(h') + \Delta \cdot O(\lg f') \;=\; O\left(\frac{\lg u}{\lg\lg u}\right) + O\left(\lg^{\epsilon'} u \cdot \lg\lg u\right) \;=\; O\left(\frac{\lg u}{\lg\lg u}\right),$$

since $\epsilon' < 1$. At the bottom level the label is fixed by the singleton range, and the corresponding ancestor of v follows from the stored preorder rank. □

The linear-space cost recorded in Theorem 4.3.4 is a consequence of the thesis-wide convention of measuring space in machine words. He, Munro, and Zhou sharpen the same construction to an entropy-compressed bit-level bound driven by the zeroth-order entropy $H(W_T)$ of the multiset of node labels of T, which satisfies $H(W_T) \le \lg u$ with equality only when the labels appearing in T are all distinct. Their refinement of the path-counting encoding uses $n(H(W_T) + 2) + o(n)$ bits when $u = O(\lg^{\epsilon} n)$ for some constant $\epsilon \in (0, 1)$, and $nH(W_T) + O(n \lg u / \lg\lg n)$ bits otherwise, while retaining the path-counting time of Theorem 4.3.4. Their refinement of the path-selection encoding uses $n(H(W_T) + 2) + o(n)$ bits when $u = O(1)$, and $nH(W_T) + O(n \lg u / \lg\lg u)$ bits otherwise, while retaining the path-selection time of Theorem 4.3.4. The question whether these bit-level refinements compose with the compression measures introduced by the set-collection constructions of later chapters is taken up in the open-problem discussion of Chapter 7.

The three theorems of this chapter supply the labeled-tree machinery that the downstream constructions of the thesis need. The α-operations of Theorem 4.2.2 run in $O(\lg\lg_{\omega} u)$ time, the binary-partition path queries of Theorem 4.3.1 in $O(\lg u)$ time, and the sublogarithmic word-RAM refinement of Theorem 4.3.4 answers path counting in $O(\lg_{\omega} u)$ time and path selection in $O(\lg u / \lg\lg u)$ time. Later chapters of the thesis appeal to these three theorems when building compressed representations for collections of sets.

Compressibility Measures for Set Collections

5

5.1 Setup and baseline 57
5.2 Containment and insertion 58
5.3 Symmetric-difference compressibility 62
5.4 Atom-based information-theoretic bound 69
5.5 Set-Union Matching 73

The tree extraction framework of Chapter 4 is not used in this chapter. Compressibility is studied here as a combinatorial property of $\mathcal{S}$, independent of any data-structure realisation. The translation of each measure into a queryable representation is the subject of the next chapter.

Chapter 2 closed with a representation that meets the per-set baseline $H_{wc}(\mathcal{S})$ of Definition 1.2.3 up to lower-order terms, by laying a sparse bitvector (Theorem 2.2.2) over each S_i independently. When the sets in $\mathcal{S}$ are correlated, this baseline is loose. Two sets with small symmetric difference pay full price twice; a containment $S_i \subset S_j$ contributes nothing to the cost. We now develop combinatorial measures that drop strictly below H_{wc} by exploiting containment, insertion, and symmetric-difference structure among the sets. A lower bound based on the membership patterns of universe elements closes the combinatorial picture on how much compression is attainable in principle, and a polynomial-time greedy construction delivers an encoding that always improves on the baseline.

5.1 Setup and baseline

We return to the setting of Section 1.2. A collection $(\mathcal{S}, U)$ consists of a totally ordered universe $U = [1..u]$ of size u and a family $\mathcal{S} = \{S_1, \ldots, S_m\}$ of m finite subsets of U, with $n = \sum_{i=1}^{m} |S_i|$ counting elements across sets with multiplicity (Definition 1.2.1). A representation of $\mathcal{S}$ must support the five operations of Definition 1.2.2, namely member, access, rank, predecessor, and successor. The independent-encoding baseline of Definition 1.2.3,

Chapter 1 fixed the problem and its operations; Chapter 2 showed how to meet the per-set baseline with sparse bitvectors. The present chapter asks what strictly less than that baseline buys us when $\mathcal{S}$ has structure, and which combinatorial quantity tracks the gain.

$$H_{wc}(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{u}{|S_i|},$$

is the per-set information content of the collection when each S_i is viewed as an isolated subset of U. No inter-set relationship contributes to this cost.

The baseline is achievable up to lower-order terms. Applying Theorem 2.2.2 to the characteristic bitvector of each S_i separately stores the whole collection in

$$H_{wc}(\mathcal{S}) \;+\; O(n + m \lg n) \quad \text{bits},$$

supporting access on any $S \in \mathcal{S}$ in $O(1)$ time, rank in $O(1 + \lg(u/|S|))$ time, and member as an access at the rank position [24]. Predecessor and successor reduce to one rank plus one access. The $O(n)$ slack accounts for the remainder arrays in the m per-set Elias-Fano layouts (Subsection 2.2.2); the $O(m \lg n)$ slack pays for a directory of pointers into the m separate bitvectors, so that each set can be addressed in $O(1)$ time.

[24]: Okanohara et al. (2007), *Practical entropy-compressed rank/select dictionary*

The set-by-set representation is the right starting point when the collection really is a bag of unrelated sets. For every S_i, there are $\binom{u}{|S_i|}$ subsets of U of that size, and distinguishing them costs $\lg \binom{u}{|S_i|}$ bits. When $\mathcal{S}$ is genuinely unstructured, no encoding can do better than summing the per-set information contents. The difficulty is that collections arising in practice are almost never unstructured. Two sets with symmetric

In the applications of Section 1.1, $\mathcal{S}$ is never a bag of unrelated sets. Adjacency lists of neighbouring web nodes share most of their links, and k-mer colour sets change by one element between consecutive k-mers. The baseline ignores this overlap entirely.

difference of size one pay full price twice under H_{wc}, and nothing in the baseline notices that one is essentially a copy of the other. If $S_i \subset S_j$, describing S_i as a subset of S_j costs $\lg \binom{|S_j|}{|S_i|}$ bits, strictly less than $\lg \binom{u}{|S_i|}$ when $|S_j| < u$. If the symmetric difference $S_i \triangle S_j$ is small, a description of S_j as an edit of S_i costs only $|S_i \triangle S_j| \lg u$ bits. Every such relationship is invisible to H_{wc}.

The convention $U = \bigcup_i S_i$ is the one adopted by Alanko et al. [7] and by Gagie, He, and Navarro [9]; under it every universe element has a nonempty pattern of membership across $\mathcal{S}$, which sharpens the combinatorial analysis of inter-set structure.

The rest of this chapter answers the combinatorial side of the question. We assume from here on that $U = \bigcup_{i=1}^{m} S_i$, so that every universe element belongs to at least one set of the collection. Universe elements outside $\bigcup_i S_i$, when present in the input, are handled by applying the results to the restriction $U' = \bigcup_i S_i$. Under this convention we introduce three measures that drop strictly below H_{wc} whenever $\mathcal{S}$ has inter-set structure, together with an information-theoretic lower bound based on the membership patterns of universe elements, and a polynomial-time construction that always improves on the baseline. Throughout, cost is measured in bits of information content. The translation of each measure into a data structure that supports the five operations of Definition 1.2.2 is deferred to the next chapter, where the tree extraction apparatus of Chapter 4 is invoked.

5.2 Containment and insertion

Given a set S and a reference set R, how many bits are needed to describe S once R is known? Two primitive cases admit a clean answer. When $S \subseteq R$, describing S reduces to choosing $|S|$ positions out of the $|R|$ elements of R, at a cost of $\lg \binom{|R|}{|S|}$ bits. When $R \subseteq S$, describing S reduces to listing the $|S \setminus R|$ elements that must be added to R to obtain S, each drawable from $U \setminus R$. The two cases are duals of one another. One specifies which elements of a larger reference are retained, the other specifies which elements outside a smaller reference are inserted.

The question is posed on a single pair (S, R). The next two subsections apply each primitive to the whole collection, obtaining two compressibility measures whose definitions disagree on which direction of the containment is assumed but whose information-theoretic flavour is the same.

Raised to a whole collection $\mathcal{S}$, each primitive yields a compressibility measure. Choosing the reference $R = p(S)$ to be a smallest proper superset of S in $\mathcal{S}$ instantiates the first primitive and gives the containment entropy $L(\mathcal{S})$ of Alanko, Bille, Gørtz, Navarro, and Puglisi [7]. Choosing $R = p'(S)$ to be a largest strict subset of S in $\mathcal{S}$ instantiates the second and gives the insertion compressibility $I(\mathcal{S})$ of Gagie, He, and Navarro [9]. The remainder of this section develops the two primitives as counting bounds, then applies each to $\mathcal{S}$ to obtain the corresponding measure and its known space realisation.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

5.2.1 Reference encoding

We isolate the two primitives as purely combinatorial counting bounds, independently of any data-structure realisation. Throughout, U is the universe of size u and $S, R \subseteq U$ are finite.

The two primitives do not exhaust the space of reference encodings. When R and S are incomparable, one records the symmetric difference $S \triangle R$; this is taken up in the next section.

Proposition 5.2.1 (Subset primitive) *If $S \subseteq R \subseteq U$, then S can be described in $\lg \binom{|R|}{|S|}$ bits given R. The bound is tight over the class of subsets of R of size $|S|$.*

Proof. There are exactly $\binom{|R|}{|S|}$ subsets of R of size $|S|$, and S is one of them. An encoding restricted to this class uses $\lg \binom{|R|}{|S|}$ bits by indexing. Since every subset of R of size $|S|$ is a valid value of S, no shorter prefix-free code exists. □

The subset primitive strictly improves on the independent cost $\lg \binom{u}{|S|}$ of Definition 1.2.3 whenever $|R| < u$. Indeed $\binom{|R|}{|S|} \leq \binom{u}{|S|}$ with equality only when $|R| = u$, so $R = U$ recovers the baseline. Every proper reference set yields a strict saving.

Proposition 5.2.2 (Superset primitive) *If $R \subseteq S \subseteq U$, then S can be described by specifying the $|S \setminus R|$ elements of $S \setminus R$. An encoding with alphabet $U \setminus R$ uses $|S \setminus R| \lg(u - |R|)$ bits, and one with alphabet U uses $|S \setminus R| \lg u$ bits.*

Proof. The elements of $S \setminus R$ are distinct members of $U \setminus R$. Encoding each as an integer in $[1..u - |R|]$ costs $\lg(u - |R|)$ bits; encoding each as an integer in $[1..u]$ costs $\lg u$ bits. The second encoding discards the knowledge of R and is looser. □

The two primitives describe S given R in opposite directions. The subset primitive measures S as a fraction of a larger reference; the superset primitive measures how many elements must be added to a smaller reference. Neither exhausts the possibilities when R and S are incomparable. The symmetric-difference primitive, where R plays the role of a close but unordered neighbour of S, is developed in the next section.

5.2.2 Containment entropy

Alanko et al. [7] apply Proposition 5.2.1 to every set in $\mathcal{S}$ at once. For each S_i, the reference is a smallest proper superset of S_i among the sets of $\mathcal{S}$. If none exists, the reference is U itself. The choice is canonical enough to induce a tree on $\mathcal{S} \cup \{U\}$.

Alanko et al. write n_i for $|S_i|$, p_i for $|p(S_i)|$, n for the total size, and s for the number of sets. We keep the thesis notation from Definition 1.2.1.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

Definition 5.2.1 (Containment hierarchy [7]) *Let $\mathcal{S} = \{S_1, \ldots, S_m\}$ be a collection over universe U. The containment hierarchy of $\mathcal{S}$ has U at the root, and for each $S_i \in \mathcal{S}$ the parent $p(S_i)$ is a smallest set in $\mathcal{S}$ strictly containing S_i; if no such set exists, then $p(S_i) = U$.*

Definition 5.2.2 (Containment entropy [7]) *The containment entropy of $\mathcal{S}$ is*

$$L(\mathcal{S}) = \sum_{i=1}^{m} \lg \binom{|p(S_i)|}{|S_i|},$$

where $p(S_i)$ is the parent of S_i in the containment hierarchy.

The definition is an instance-by-instance application of Proposition 5.2.1. Every S_i is a subset of its parent $p(S_i)$, so the subset primitive gives a description of length $\lg \binom{|p(S_i)|}{|S_i|}$ bits. Summing over i yields the containment entropy. When the hierarchy is trivial and every $p(S_i)$ equals U, the containment entropy collapses to the baseline.

Proposition 5.2.3 (Containment bound[7]) *For every collection $\mathcal{S}$,*

$$L(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. Since $p(S_i) \subseteq U$, we have $|p(S_i)| \le u$, and hence $\binom{|p(S_i)|}{|S_i|} \le \binom{u}{|S_i|}$. Taking logarithms and summing over i gives $L(\mathcal{S}) \le \sum_i \lg \binom{u}{|S_i|} = H_{wc}(\mathcal{S})$. □

Translating the bound into space requires a device that shortens every root-to-leaf path in the hierarchy. A direct representation stores each S_i as a sparse bitvector of length $|p(S_i)|$ indicating which elements of the parent are retained. Queries on S_i then walk the chain of ancestors up to U, which can be as long as the hierarchy itself. Alanko et al. introduce a path-compressed variant in which the parent of S_i skips all intermediate ancestors whose size is at most $2|S_i|$.

Definition 5.2.3 (Contracted hierarchy [7]) *The contracted hierarchy of $\mathcal{S}$ has U at the root. For each S_i, its parent $c(S_i)$ is the highest ancestor S_t of $p(S_i)$ in the containment hierarchy satisfying $|S_t| \le 2|S_i|$; if no such ancestor exists, $c(S_i) = p(S_i)$.*

Alanko et al. prove that the contracted hierarchy has depth $O(\log(u/|S_i|))$ at every node S_i, because consecutive ancestors at least double in size (Lemma 4 of [7]), and that encoding S_i as a sparse bitvector relative to $c(S_i)$ inflates the bit cost by at most a constant factor, leaving the $L(\mathcal{S})$ term unchanged asymptotically (Lemma 5 of [7]). The sparse bitvector machinery is the one built in Section 2.2.

Theorem 5.2.4 (Containment-bounded representation [7]) *Let $\mathcal{S}$ be a collection of m sets over a universe of size u, with total size $n = \sum_i |S_i|$ as in Definition 1.2.1. The contracted hierarchy of $\mathcal{S}$ can be stored in*

$$L(\mathcal{S}) + O(n + m \lg n) \quad \textit{bits}$$

using one sparse bitvector per edge, via Theorem 2.2.2.

The $O(n + m \lg n)$ slack is the familiar cost accounted for in the baseline of Section 5.1. Each of the m sparse bitvectors spends $O(|S_i|)$ additive bits in its Elias-Fano layout, summing to $O(n)$, and $O(\lg n)$ bits per set are spent on a directory that addresses the m separate structures.

Alanko et al. observe in Section 3 of [7] that the containment hierarchy is not the cheapest possible. When many sets are subsets of both S_i and S_j, introducing a new set $S_i \cap S_j$ as an internal node could shorten further the encoding, even though $S_i \cap S_j$ is not in $\mathcal{S}$. They remark that an advantage of $L(\mathcal{S})$ is that it is computable in polynomial time from $\mathcal{S}$, whereas more general notions that allow the introduction of new sets may not be. The question of how to augment the hierarchy with synthetic nodes, in particular which augmentation minimises the total encoding cost, is left open in [7].

5.2.3 Insertion compressibility

Compressed Set Set Difference

[9] call the mea- sibility because by the elements erence. The dual describes the el- arger reference vo are the same posite directions.

Gagie, He, and Navarro [9] apply Proposition 5.2.2 to every set in $\mathcal{S}$. For each $S \in \mathcal{S}$, the reference is a largest strict subset of S in $\mathcal{S}$. If none exists, the reference is the empty set.

Definition 5.2.4 (Insertion compressibility [9]) *Let $\mathcal{S}$ be a collection. For each $S \in \mathcal{S}$, let $p'(S)$ be a largest strict subset of S in $\mathcal{S}$, or $\emptyset$ if no such subset exists. The insertion compressibility of $\mathcal{S}$ is*

$$I(\mathcal{S}) \;=\; \sum_{S \in \mathcal{S}} |S \setminus p'(S)| \;=\; \sum_{S \in \mathcal{S}} \big(|S| - |p'(S)|\big).$$

The description length implied by the superset primitive is $|S \setminus p'(S)| \lg u$ bits per set. Dropping the $\lg u$ factor, $I(\mathcal{S})$ counts the total number of inserted elements across all sets, which is bounded by the total size of $\mathcal{S}$.

Proposition 5.2.5 (Insertion bound [9]) *For every collection $\mathcal{S}$ with total size $n = \sum_i |S_i|$,*

$$I(\mathcal{S}) \;\le\; n.$$

Proof. By Definition 5.2.4, $I(\mathcal{S}) = \sum_{S \in \mathcal{S}} |S \setminus p'(S)|$. For each $S \in \mathcal{S}$, $|S \setminus p'(S)| \le |S|$, so $I(\mathcal{S}) \le \sum_{S \in \mathcal{S}} |S| = n$. □

Equality holds when $p'(S) = \emptyset$ for every S, which happens whenever no two sets of $\mathcal{S}$ are in a containment relation. At the opposite extreme, a deep chain $S_1 \subset S_2 \subset \cdots \subset S_m$ gives $I(\mathcal{S}) = |S_m|$, a quantity that can be far below n when the chain is long. A direct comparison with $H_{wc}(\mathcal{S})$ is unit-dependent: the insertion representation pays $I(\mathcal{S}) \lg u$ bits against $H_{wc}(\mathcal{S}) = \sum_i \lg \binom{u}{|S_i|}$ bits, and neither dominates the other across all collections. The value of $I(\mathcal{S})$ is that it exposes sequential containment structure, which is invisible to the per-set baseline.

The dual construction that realises $I(\mathcal{S})$ structurally is the insertion graph of [9]. Its nodes are the sets of $\mathcal{S}$ together with $\emptyset$, and each S has a single outgoing edge to $p'(S)$ of weight $|S \setminus p'(S)|$. The edge weights sum to $I(\mathcal{S})$ by Definition 5.2.4. Since p' is a functional parent map and $|p'(S)| < |S|$ strictly, the graph has no cycles and every path eventually reaches $\emptyset$; it is a forest rooted at $\emptyset$.

The graph is not directly a labelled ordinal tree, because its edges carry numeric weights rather than element labels. Gagie et al. expand each weighted edge into a chain.

The chain expansion is the move that makes the insertion graph usable as a labelled ordinal tree. Weights are replaced by element labels; the multiset of labels on the root-to-$v(S)$ path is exactly the set S.

Definition 5.2.5 (Insertion tree [9]) *Let $\mathcal{S}$ be a collection, with $p'(S)$ as in Definition 5.2.4. The insertion tree of $\mathcal{S}$ is the rooted labelled ordinal tree defined by the following rules. The root is $v(\emptyset)$. For every $S \in \mathcal{S}$ there is a node $v(S)$, and the tree contains a chain of $|S \setminus p'(S)|$ edges from $v(p'(S))$ to $v(S)$, each labelled with a distinct element of $S \setminus p'(S)$, in arbitrary order. The tree may contain additional internal nodes introduced by these chains but not associated with any set of $\mathcal{S}$.*

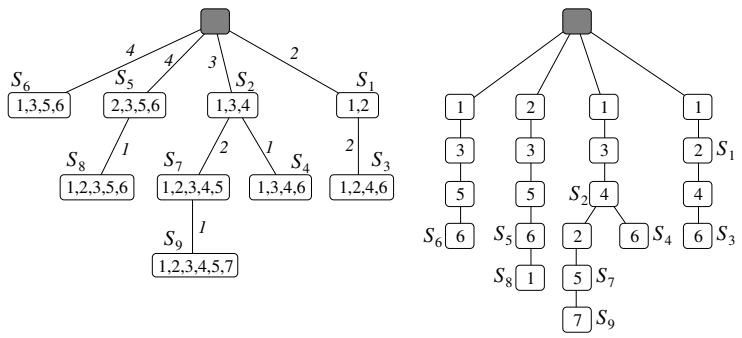

Figure 5.1: Reproduced from Gagie, He, and Navarro [9]. Left: an insertion graph for a collection $\mathcal{S} = \{S_1, \ldots, S_9\}$, with edge weights written as slanted values. The edge weights sum to $I(\mathcal{S}) = 20$. Right: the corresponding insertion tree obtained by expanding each edge $(v(p'(S)), v(S))$ of weight $w = |S \setminus p'(S)|$ into a chain of w labelled edges. The tree has $1+I(\mathcal{S}) = 21$ nodes. Each set S_i labels the chain endpoint $v(S_i)$.

Proposition 5.2.6 (Insertion tree structure[9]) *The insertion tree of $\mathcal{S}$ has $1 + I(\mathcal{S})$ nodes. Every edge label lies in $[1..u]$, and for every $S \in \mathcal{S}$ the set of labels on the path from $v(\emptyset)$ to $v(S)$ is exactly S.*

Proof. The root contributes one node. Each $S \in \mathcal{S}$ contributes a chain of $|S \setminus p'(S)|$ edges, hence $|S \setminus p'(S)|$ new nodes. Summing over $\mathcal{S}$ adds $\sum_S |S \setminus p'(S)| = I(\mathcal{S})$ nodes. Every label lies in $U = [1..u]$ by construction. An induction on depth shows that the labels on the root-to-$v(S)$ path accumulate exactly the elements of $p'(S) \cup (S \setminus p'(S)) = S$: the induction base is the root, which represents $\emptyset$, and each step adds $S \setminus p'(S)$ to $p'(S)$. □

Figure 5.1 shows the insertion graph for a nine-set collection and the insertion tree obtained from it by chain expansion. The edge weights on the left add up to $I(\mathcal{S}) = 20$, and the tree on the right has 21 nodes in total. The labels on the root-to-$v(S_i)$ path reproduce the content of S_i.

The insertion tree carries labels from $[1..u]$ and no deletion marks. Compared to an encoding that lists for each S_i the elements of $S_i \setminus p'(S_i)$ as an unordered collection, the tree is only a structural reorganisation: no additional information is stored, and the total label count is $I(\mathcal{S})$. The structural statement above is all that is needed for the combinatorial picture. Its translation into a queryable data structure, via the tree extraction framework of Chapter 4, is deferred to the next chapter.

5.3 Symmetric-difference compressibility

The subset and superset primitives of Subsection 5.2.1 both require a directional relationship between S and its reference. Neither applies when S and R are incomparable. The symmetric-difference primitive is the smallest natural extension that covers this case.

Containment and insertion, developed in Section 5.2, both pick a directional reference. The containment hierarchy assigns each $S \in \mathcal{S}$ a parent $p(S) \supseteq S$ and pays $\lg \binom{|p(S)|}{|S|}$ bits; the insertion hierarchy assigns a parent $p'(S) \subseteq S$ and pays $|S \setminus p'(S)| \lg u$ bits. When two sets $A, B \in \mathcal{S}$ are incomparable, neither $A \subseteq B$ nor $B \subseteq A$ holds, so neither primitive gives any saving over the per-set baseline $H_{wc}(\mathcal{S})$.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

The symmetric difference $|A \triangle B| = |A \setminus B| + |B \setminus A|$ is the symmetric alternative. It measures proximity without requiring a containment. Gagie, He, and Navarro [9] turn this quantity into a compressibility measure $\Delta(\mathcal{S})$, and give the first representation of size $O(\Delta(\mathcal{S}))$ words that answers the five operations of Definition 1.2.2 directly on the compressed form. This section develops $\Delta(\mathcal{S})$ from its prehistory in information retrieval, matrix multiplication, and graph compression, constructs the

minimum-weight symdiff graph by an adapted Prim algorithm, and introduces the indel tree as the structural object that will carry queries in the next chapter.

5.3.1 Historical context

The idea of encoding a set as the symmetric difference with a close neighbour predates the compressibility measure by three decades. It was rediscovered in four distinct communities, each time motivated by a specific application and each time without full awareness of the previous occurrences.

The earliest systematic treatment is by Bookstein and Klein [8], motivated by inverted indexes in information retrieval. For each term of a text collection, a posting bitvector records the segments in which the term occurs; semantically correlated terms, such as the pair *Security* and *Council*, produce posting bitvectors with heavy overlap. Bookstein and Klein form the complete weighted graph on the posting bitvectors, with edge weight equal to the Hamming distance, introduce an artificial zero vector as an extra node, and take a minimum spanning tree of the augmented graph. Every posting bitvector is then encoded as the XOR with its parent in the tree (for non-root bitvectors) or directly (for root bitvectors). They report that this preprocessing nearly doubles the compression saving on the Hebrew Bible corpus when combined with sparse-bitvector coders.

Bookstein and Klein [8] treat each bitvector as a set and identify the Hamming distance between two bitvectors with the cardinality of the symmetric difference between the corresponding sets.

[8]: Bookstein et al. (1991), *Compression of correlated bit-vectors*

The same broader idea of encoding each object via its difference from a neighbour reappears a decade later in two different communities. Buchsbaum, Caldwell, Church, Fowler, and Muthukrishnan [50] tackle the compression of massive tables in engineering contexts through differential dependencies, storing each column as a delta against a greedily chosen predecessor column. In a parallel development, Björklund and Lingas [51] study fast Boolean matrix multiplication for highly clustered data. Gagie, He, and Navarro [9] credit Björklund and Lingas with the first observation that matrix multiplication can be carried out in time bounded by the total weight of a spanning forest of the row similarity graph, since the symmetric difference between two rows carries all the information needed to update the product incrementally. Exact MST construction was costly in their setting, and they used approximate MSTs instead.

The Hamming distance between two binary row vectors equals $|A \triangle B|$ when the rows are read as characteristic vectors over the column universe. The MST argument is Bookstein and Klein's; the application to matrix multiplication is Björklund and Lingas's.

[50]: Buchsbaum et al. (2000), *Engineering the compression of massive tables: an experimental approach*

[51]: Björklund et al. (2001), *Fast Boolean Matrix Multiplication for Highly Clustered Data*

The web-graph compression literature gives the most celebrated application. Boldi and Vigna [4] observe that when the rows of the adjacency matrix of a web graph are sorted by URL, consecutive rows share almost all of their outgoing edges. The WebGraph framework stores each adjacency list against a chosen reference list drawn from a bounded backreference window of size W, recording by a copy list which successors of the reference to retain and by a list of extra nodes which successors to add. The backreference choice is a heuristic approximation of a minimum-weight spanning forest of the row similarity graph, tuned to keep the decoding depth bounded. Gagie, He, and Navarro [9] note in retrospect that Boldi and Vigna were not aware of Bookstein and Klein's prior construction. Boldi and Vigna report compression ratios on the order of

Boldi and Vigna's [4] reference compression restricts candidate references to URL-sorted neighbours within a window W, so the resulting structure is an engineering approximation of a minimum-weight spanning forest rather than the true minimum. The gain in compression ratio is dramatic in practice.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

three to four bits per edge on real web crawls, an order-of-magnitude improvement over adjacency-list representations.

In Mantis [6], each k-mer in a de Bruijn graph carries a colour set recording the genomes in which the k-mer occurs. Colour sets of neighbouring k-mers differ by one or two colours, so the MST encoding yields several orders of magnitude improvement over per-k-mer storage.

[6]: Almodaresi et al. (2020), *An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search*

[52]: Alves et al. (2025), *Accelerating Graph Neural Networks Using a Novel Computation-Friendly Matrix Compression Format*

The elevation to a compressibility measure is what separates Gagie, He, and Navarro's treatment [9] from its predecessors. A thirty-year sequence of independent rediscoveries becomes a single combinatorial quantity $\Delta(\mathcal{S})$ with a clean definition.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

The technique reaches bioinformatics with Almodaresi, Pandey, Ferdman, Johnson, and Patro [6], who compress the colour sets of coloured de Bruijn graphs in the Mantis tool. They restrict the similarity graph to edges between colour sets of neighbouring vertices in the de Bruijn graph, take a minimum spanning tree of the resulting graph, and encode every colour set on that MST. A more recent instance is due to Alves, Moustafa, Benkner, Francisco, Gansterer, and Russo [52], who apply the same idea to compress binary adjacency matrices for graph neural network acceleration. Gagie, He, and Navarro [9] take Alves et al. as their direct predecessor, and the two-rooted symdiff graph of Definition 5.3.1 is a variant of the one-rooted model used there.

Over thirty years of independent rediscoveries, the MST-on-Hamming construction remained a compression heuristic. Gagie, He, and Navarro [9] elevate it to a compressibility measure $\Delta(\mathcal{S})$ tracking exactly the weight of the minimum spanning tree, prove that $\Delta(\mathcal{S}) \leq I(\mathcal{S})$ holds for every collection, give a construction algorithm whose running time depends on the MST weight rather than on the total pairwise symdiff volume, and exhibit the first representation of size $O(\Delta(\mathcal{S}))$ words that answers member, access, rank, predecessor, and successor directly on the compressed form. Prior work only compressed; no prior work offered a queryable compressed form.

The rest of this section develops the combinatorial content of $\Delta(\mathcal{S})$. The construction algorithm of Gagie et al. is presented in condensed form, and the indel tree, which carries the structural content into a labelled ordinal tree suitable for the tree extraction framework, closes the section.

5.3.2 Symdiff graph

The special nodes $\emptyset$ and U are not elements of $\mathcal{S}$; they are auxiliary roots that anchor the two trees of the symdiff graph. The choice to allow *two* roots is what enables both insertion-like and deletion-like reductions in a single structure.

The object underlying the symmetric-difference primitive is a weighted directed graph on $\mathcal{S}$ augmented with two auxiliary nodes, $\emptyset$ and U. Every set of $\mathcal{S}$ points to exactly one neighbour, and the weight of the outgoing edge records the symmetric-difference cost of describing the set from its neighbour.

Definition 5.3.1 (Symdiff graph [9]) *A symdiff graph on a collection $\mathcal{S}$ is a weighted directed graph on the node set $\mathcal{S} \cup \{\emptyset, U\}$, where $U = \bigcup_{S \in \mathcal{S}} S$ is the universe spanned by $\mathcal{S}$. Every $S \in \mathcal{S}$ has exactly one outgoing edge; its target $p(S) \in \mathcal{S} \cup \{\emptyset, U\}$ is the neighbour of S, and the weight of the edge $(S, p(S))$ is $|S \triangle p(S)|$. The graph is required to satisfy the reachability condition that from every $S \in \mathcal{S}$ one of $\emptyset$ or U is reachable by following outgoing edges. The weight of the graph is the sum of the edge weights.*

Given a symdiff graph, each $S \in \mathcal{S}$ is reconstructed from its neighbour $p(S)$ by inserting the elements of $S \setminus p(S)$ and deleting those of $p(S) \setminus S$. The total number of labels stored across the whole collection equals the weight of the graph. The reachability condition ensures that the decoding terminates at one of the two canonical anchors, $\emptyset$ or U, whose content is known a priori.

Definition 5.3.2 (Symdiff compressibility [9]) *The symmetric-difference compressibility of $\mathcal{S}$ is*

$$\Delta(\mathcal{S}) \;=\; \min\{\, W(G) : G \textit{ is a symdiff graph on } \mathcal{S} \,\},$$

where $W(G)$ denotes the weight of G.

The minimisation problem admits a clean structural description. A symdiff graph has $|\mathcal{S}|$ outgoing edges and at most $|\mathcal{S}| + 2$ nodes, with the two roots $\emptyset$ and U having no outgoing edges. The reachability condition forces the graph to consist of two rooted trees, one anchored at $\emptyset$ and one at U; every $S \in \mathcal{S}$ sits in exactly one of them. Minimising the total weight over all such two-tree configurations is captured by a single spanning-tree problem on an augmented graph.

Lemma 5.3.1 (Minimum-weight symdiff graph[9]) *Let K be the complete undirected graph on $\mathcal{S} \cup \{\emptyset, U\}$ with edge weights $w(A, B) = |A \triangle B|$ for $A, B \in \mathcal{S} \cup \{\emptyset, U\}$, except for the edge between $\emptyset$ and U, which is given weight 0. The weight of a minimum spanning tree of K equals $\Delta(\mathcal{S})$, and the two trees achieving $\Delta(\mathcal{S})$ are obtained by removing the zero-weight edge between $\emptyset$ and U from the MST.*

Proof. Every minimum spanning tree of K contains the edge between $\emptyset$ and U because it is the lightest in K. Removing this zero-weight edge disconnects the MST into two trees, one containing $\emptyset$ and the other containing U; the two trees span $\mathcal{S} \cup \{\emptyset, U\}$ and their total weight equals that of the MST. Conversely, any pair of trees rooted at $\emptyset$ and U together spanning $\mathcal{S} \cup \{\emptyset, U\}$, plus the zero-weight edge between $\emptyset$ and U, gives a spanning tree of K of the same weight. The minimum over pairs of rooted trees therefore coincides with the MST weight, and the minimum-weight two-tree configuration is recovered by deleting the zero-weight edge. □

The relation to insertion compressibility is immediate. Every insertion tree of Definition 5.2.5, read as a functional parent map with the extra node $\emptyset$ playing the role of its own root, gives a valid symdiff graph in which every set S points to its largest strict subset $p'(S)$.

Proposition 5.3.2 (Symdiff-insertion inequality[9]) *For every collection $\mathcal{S}$,*

$$\Delta(\mathcal{S}) \;\leq\; I(\mathcal{S}).$$

Proof. The insertion graph of Subsection 5.2.3 has one outgoing edge from each $S \in \mathcal{S}$ to its largest strict subset $p'(S) \in \mathcal{S} \cup \{\emptyset\}$. Every S reaches $\emptyset$ along this chain, so the graph satisfies Definition 5.3.1 on the node set $\mathcal{S} \cup \{\emptyset, U\}$, with U an auxiliary node carrying no incident edges. Because $p'(S) \subseteq S$, no deletions contribute and $|S \triangle p'(S)| = |S \setminus p'(S)|$. Summing gives a symdiff graph of total weight $I(\mathcal{S})$, so $\Delta(\mathcal{S}) \leq I(\mathcal{S})$. □

The inequality can be strict. Collections with many incomparable pairs whose symmetric difference is small are where $\Delta(\mathcal{S})$ outperforms $I(\mathcal{S})$. When no containment is available, the insertion hierarchy is forced to treat the set as a fresh root, paying $|S|$ in full, whereas the symdiff graph connects the set to its nearest neighbour at cost $|S \triangle \cdot|$.

5.3.3 Adapted Prim construction

A naive computation materialises all $\binom{m+2}{2}$ pairwise symdiff weights for the classical Prim on the dense graph K. This is wasteful when $\Delta(\mathcal{S}) \ll m^2$. The algorithm of Gagie et al. [9] discovers the weights incrementally from smallest to largest, and stops iterating on a pair as soon as its weight exceeds the current MST threshold.

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

The MST characterisation of Lemma 5.3.1 reduces the computation of $\Delta(\mathcal{S})$ to a minimum spanning tree on the augmented complete graph K with $m+2$ nodes. Evaluating all pairwise symmetric differences explicitly costs $O(m^2u)$ in the worst case, which is extravagant when $\Delta(\mathcal{S})$ is much smaller than m^2. Gagie, He, and Navarro [9] show that the MST can be computed by an adapted Prim algorithm that evaluates each pairwise symmetric difference only partially, stopping as soon as the comparison exceeds the current MST frontier threshold. The running time is a function of how much pairwise symdiff volume actually lies below the heaviest MST edge, not of the total volume.

Suspended Prim. The adaptation rests on a simple invariant of Prim's algorithm. At any point of Prim's execution, let k be one more than the maximum edge weight included in the MST so far. Prim will never add an edge of weight at least k while any edge of weight strictly less than k remains available from the current MST frontier. Consequently, Prim is correct when it is allowed to operate on the subset of edges of weight strictly less than k, suspending whenever that subset is exhausted and resuming after new edges of weight up to k become known.

Proximity iterator. The weight discovery is driven by a proximity iterator built on a generalised suffix tree of the concatenation of all sets. The sets are sorted in increasing order of their elements in time $O(n \lg u)$; each set S_i is appended a unique terminator $\$_i$; the concatenation $T(\mathcal{S}) = S_1\$_1 \cdots S_m\$_m$ has length $O(n)$. Its generalised suffix tree [53] is built in $O(n)$ time [54] with constant-time lowest-common-ancestor queries [33]. The LCA of two leaves returns the longest common prefix of the corresponding suffixes, which translates into a merge step that identifies the next element of $S_i \triangle S_j$ in constant time. The guarantee is a single-step iterator that advances by one element of the symmetric difference at each call.

[53]: Apostolico (1985), *The myriad virtues of subword trees*

[54]: Farach-Colton et al. (2000), *On the sorting-complexity of suffix tree construction*

[33]: Bender et al. (2000), *The LCA Problem Revisited*

Lemma 5.3.3 (Proximity iterator[9]) *After an $O(n \lg u)$-time preprocessing of $\mathcal{S}$, there is a data structure supporting the following operation. Given two pointers p, q into a pair of sorted sets S, S', each initialised to the first element, a single call advances p or q by one position and either reports one new element of $S \triangle S'$ or detects that both pointers have reached their terminators. After k successful calls on the pair (S, S'), either both pointers point to terminators, in which case $|S \triangle S'| < k$, or k distinct elements of $S \triangle S'$ have been reported, in which case $|S \triangle S'| \geq k$. Each call runs in $O(1)$ time.*

With the iterator in place, Prim is run in phases indexed by an integer threshold k. Phase k advances every still-active iterator by one step; pairs that exhaust both pointers have their final weight revealed and are fed to Prim, which consumes all available edges of weight strictly less than k before the next phase begins. The pseudocode appears in algorithm 1.

Theorem 5.3.4 (Construction of $\Delta(\mathcal{S})$[9]) *Let $\mathcal{S}$ be a collection of m sets over a universe of size u, with total size n. Let ℓ be the maximum edge weight in the*

Algorithm 1: Adapted Prim for $\Delta(\mathcal{S})$ and the minimum-weight symdiff graph, after Gagie, He, and Navarro [9].

Input: A collection $\mathcal{S}$ (sorted) with the proximity iterator of Lemma 5.3.3 preprocessed.
Output: The minimum-weight symdiff graph on $\mathcal{S} \cup \{\emptyset, U\}$.

```
V ← {∅, U} with the zero-weight edge (∅, U) in the MST; V′ ← S;
Bag B ← all pairs of nodes in V ∪ V′ except (∅, U), with weight
  marked unknown; each pair carries an iterator at p = q = 1;
for k = 1, 2, 3, ... do
    foreach edge (A, B) ∈ B still in play do
        advance the proximity iterator of (A, B) by one step;
        if both pointers of (A, B) reached their terminators then
            record w(A, B) := k − 1 in Prim's structures and remove
              (A, B) from B;
    while Prim's frontier contains an edge of weight < k into V′ do
        extract the lightest such edge (u, v), move v from V′ to V,
          add (u, v) to the MST;
    if V′ = ∅ then
        return the MST;
```

minimum-weight symdiff graph on $\mathcal{S}$. The adapted Prim algorithm computes $\Delta(\mathcal{S})$ and the minimum-weight symdiff graph in time

$$O\left(n \lg u + \sum_{S,S' \in \mathcal{S}} \min(\ell, |S \triangle S'|)\right) \subseteq O\big(n \lg u + \min(m^2\ell, mn)\big).$$

Proof. The $O(n \lg u)$ term is the preprocessing cost of Lemma 5.3.3. For the second term, every pair $(S, S') \in \mathcal{S}^2$ has its proximity iterator advanced $\min(\ell, |S \triangle S'|)$ times. At most ℓ times because the outer loop suspends Prim once the threshold reaches ℓ and the MST is complete, and at most $|S \triangle S'|$ times because once the iterator exhausts the symmetric difference both pointers reach their terminators. The total iterator work therefore equals $\sum_{S,S'} \min(\ell, |S \triangle S'|)$. Prim's own bookkeeping is dominated by this cost. The two worst-case bounds arise from $\min(\ell, |S \triangle S'|) \le \ell$ and from $|S \triangle S'| \le |S| + |S'|$. □

The adapted Prim improves strictly on the naive $O(m^2 u)$ whenever $\ell \ll u$, that is, whenever the minimum-weight symdiff graph is dominated by short edges. For the collections where $\Delta(\mathcal{S})$ is interesting in practice, such as adjacency lists of web graphs or colour sets of de Bruijn graphs, the MST edges have weight far below u, and the construction cost is governed by $m\ell$ rather than by mu.

5.3.4 Indel trees

The minimum-weight symdiff graph splits into two trees anchored at $\emptyset$ and U, and each edge $(p(S), S)$ carries a weight that decomposes as $|S \triangle p(S)| = |S \setminus p(S)| + |p(S) \setminus S|$ insertions and deletions. Unlike the insertion tree of Definition 5.2.5, whose edges carry only insertions, the indel tree of [9] carries both kinds of edit. Translating a weighted edge

Where the insertion tree of Definition 5.2.5 carries only positive labels drawn from $U = [1..u]$, the indel tree carries labels from $[-u..-1] \cup [1..u]$. Doubling the alphabet is the price of allowing deletions.

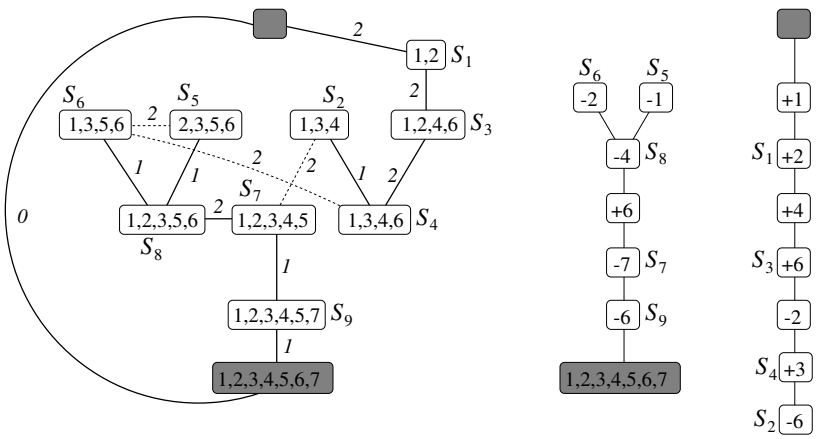

Figure 5.2: Reproduced from Gagie, He, and Navarro [9]. Left: a minimum-weight symdiff graph on $\mathcal{S} = \{S_1, \ldots, S_9\}$ with $\Delta(\mathcal{S}) = 13$. Dashed edges are available in the complete graph K but not used by the MST; they are the edges whose weight exceeds the largest MST edge weight $\ell = 2$. Right: the two indel trees rooted at U (drawn upside down) and at $\emptyset$. Chain labels $+x$ are insertions, $-x$ are deletions relative to the parent.

into a chain of labelled edges therefore requires two label classes, one for insertions and one for deletions.

Definition 5.3.3 (Indel trees [9]) *Let $\mathcal{S}$ be a collection, with $p(S)$ the neighbour of S in a minimum-weight symdiff graph of $\mathcal{S}$. The indel trees of $\mathcal{S}$ are two rooted labelled ordinal trees. The root of one tree is $v(\emptyset)$, the root of the other is $v(U)$. For every $S \in \mathcal{S}$ there is a node $v(S)$, lying in the same tree as $v(p(S))$. The edge connecting $v(p(S))$ to $v(S)$ is replaced by a chain of $|S \triangle p(S)|$ edges, each labelled with a distinct element of the insertion set $S \setminus p(S)$ or the deletion set $p(S) \setminus S$, in arbitrary order. Labels from the insertion set are written $+x$ for $x \in S \setminus p(S)$; labels from the deletion set are written $-y$ for $y \in p(S) \setminus S$. The label alphabet is $[-u..-1] \cup [1..u]$, of size $2u$ (Gagie et al. write this as $[-u..u]$; the label 0 is unused since $0 \notin U$).*

Proposition 5.3.5 (Indel-tree structure[9]) *The two indel trees of $\mathcal{S}$ together contain $\Delta(\mathcal{S}) + 2$ nodes. For every $S \in \mathcal{S}$, the content of S is recovered from the root of its tree by walking the path from the root to $v(S)$ and applying each label in order. An insertion label $+x$ adds x to the running set, and a deletion label $-y$ removes y.*

Proof. The two roots contribute two nodes. Each $S \in \mathcal{S}$ contributes a chain of $|S \triangle p(S)|$ edges, hence $|S \triangle p(S)|$ internal nodes. Summing over $\mathcal{S}$ adds $\sum_S |S \triangle p(S)| = \Delta(\mathcal{S})$ nodes. The decoding semantics follows by induction on depth. At the root the running set is $\emptyset$ or U as given. An insertion label $+x$ increments the running set by x; a deletion label $-y$ decrements it by y. By Definition 5.3.1, the labels on the chain from $v(p(S))$ to $v(S)$ realise exactly the symmetric difference between $p(S)$ and S, so the running set at $v(S)$ equals $p(S) \cup (S \setminus p(S)) \setminus (p(S) \setminus S) = S$. □

Figure 5.2 shows the minimum-weight symdiff graph of a nine-set collection, together with the two indel trees obtained by chain expansion. The MST edges have total weight $\Delta(\mathcal{S}) = 13$, and the two trees together have 15 nodes.

The indel trees realise $\Delta(\mathcal{S})$ as a pair of labelled ordinal trees over an alphabet of size $2u$. Every set of $\mathcal{S}$ is recovered as a root-to-$v(S)$ path, and the total label count is $\Delta(\mathcal{S})$. Turning this structural object into a data structure that answers member, access, rank, predecessor, and successor within $O(\Delta(\mathcal{S}))$ words of space is taken up in the next chapter,

where the tree extraction framework of Chapter 4 supplies the technical machinery.

5.4 Atom-based information-theoretic bound

The three compressibility measures introduced so far, the containment entropy $L(\mathcal{S})$, the insertion compressibility $I(\mathcal{S})$, and the symdiff compressibility $\Delta(\mathcal{S})$, each satisfy $L, I, \Delta \leq H_{wc}(\mathcal{S})$ and improve on the baseline whenever $\mathcal{S}$ has the corresponding flavour of structure. Each of them, however, is still an upper bound on the encoding cost, obtained by exhibiting an encoding of $\mathcal{S}$. None of them speaks to how many bits every encoding of $\mathcal{S}$ must use.

Sections Section 5.2 and Section 5.3 exhibited three measures L, I, Δ, each strictly below H_{wc} whenever $\mathcal{S}$ has containment, insertion, or symmetric-difference structure. None of them addresses whether H_{wc} itself is tight against the raw information content of $\mathcal{S}$.

We now pose the dual question. Fix a collection $\mathcal{S}$ and ask how many bits are required, in the worst case, to distinguish $\mathcal{S}$ from every other collection that shares the same coarse structural data. The answer is a counting bound on a Venn-diagram partition of U induced by $\mathcal{S}$, and it yields a lower bound $L^*(\mathcal{S})$ against which every representation of $\mathcal{S}$ must be compared. The bound is simultaneously an achievable upper bound, via enumerative coding of the sequence of membership patterns, and this pair of opposite inequalities sets up the framework under which the polynomial-time construction of the next section is analysed.

5.4.1 Atoms and the worst-case counting bound

Every element of the universe can be tagged by the list of sets that contain it.

The membership pattern $\sigma(x)$ records which of the m sets contain x. Under the convention of Section 5.1, no universe element belongs to zero sets, so $\sigma(x) \neq \emptyset$ for every $x \in U$.

Definition 5.4.1 (Membership pattern) *The membership pattern of an element $x \in U$ relative to $\mathcal{S} = \{S_1, \ldots, S_m\}$ is the set*

$$\sigma(x) \;=\; \{\, i \in [m] : x \in S_i \,\}.$$

For every nonempty $\tau \subseteq [m]$, the atom of pattern τ is

$$A_\tau \;=\; \{\, x \in U : \sigma(x) = \tau \,\}.$$

Under the convention $U = \bigcup_i S_i$ fixed in Section 5.1, every $x \in U$ belongs to at least one S_i, so $\sigma(x) \neq \emptyset$ and the set of possible patterns is $2^{[m]} \setminus \{\emptyset\}$, of size $2^m - 1$. The atoms form a partition of U into at most $2^m - 1$ nonempty parts, with $\sum_\tau |A_\tau| = u$ where the sum ranges over patterns τ with $A_\tau \neq \emptyset$.

Let $R(\mathcal{S}) = \{\, \tau \subseteq [m] \setminus \{\emptyset\} : A_\tau \neq \emptyset \,\}$ denote the set of *realised* patterns and write $k = |R(\mathcal{S})|$. Enumerate $R(\mathcal{S}) = \{\tau_1, \ldots, \tau_k\}$ and set $c_{\tau_j} = |A_{\tau_j}|$. The pair $(R(\mathcal{S}), (c_{\tau_j})_{j=1}^k)$ determines the row sizes $|S_i| = \sum_{\tau \ni i} c_\tau$ and the coverage size $u = \sum_j c_{\tau_j}$, but not the collection itself, since many placements of universe elements into atoms are consistent with the same counts.

The number k of realised patterns is the single instance-specific quantity that controls the atom-based bound. It can be as small as 1 (all sets equal) and as large as $2^m - 1$ (all nonempty Venn regions occupied).

A worst-case counting bound follows by ignoring the count vector entirely. Each element of U carries one of $2^m - 1$ possible nonempty patterns, and the patterns are chosen independently across elements. The number of

The worst-case counting bound is the information content of $\mathcal{S}$ with only coverage as side information. It improves to $L^*(\mathcal{S})$ in the next subsection, which takes the atom counts themselves as side information.

collections with universe coverage U is therefore at most $(2^m - 1)^u$, so any representation that distinguishes all such collections uses at least

$$\lceil \lg(2^m - 1)^u \rceil \;=\; \lceil u \lg(2^m - 1) \rceil$$

bits in the worst case. This is the weakest of the lower bounds we will derive. Its only merit is that the side information it assumes is trivial.

5.4.2 The atom-based bound

The atom counts (c_τ) are strictly more refined side information than the row sizes $(|S_i|)$. Two collections can have the same row sizes but different atom counts, and vice versa.

Fix the realised-pattern alphabet $R(\mathcal{S})$ and the atom counts $(c_\tau)_{\tau \in R(\mathcal{S})}$, and count the collections consistent with this data. Each such collection is a labelling of the u elements of U by patterns in $R(\mathcal{S})$ such that exactly c_τ elements receive label τ. The number of such labellings is the multinomial coefficient

$$\binom{u}{(c_\tau)_{\tau \in R(\mathcal{S})}} \;=\; \frac{u!}{\prod_{\tau \in R(\mathcal{S})} c_\tau!}.$$

Proposition 5.4.1 (Atom-based bound) *Let $R(\mathcal{S}) = \{\tau_1, \ldots, \tau_k\}$ be the realised patterns of $\mathcal{S}$ and $c_{\tau_j} = |A_{\tau_j}|$ the atom counts. Set*

$$L^*(\mathcal{S}) \;=\; \lg \binom{u}{c_{\tau_1}, \ldots, c_{\tau_k}}.$$

Any encoding that uses $(R(\mathcal{S}), (c_{\tau_j})_{j=1}^k)$ as side information and distinguishes every indexed collection sharing these parameters uses at least $\lceil L^*(\mathcal{S}) \rceil$ bits in the worst case.

Proof. Let $\mathcal{C}(R, (c_\tau))$ denote the class of indexed collections $\mathcal{S}' = \{S'_1, \ldots, S'_m\}$ over U whose realised-pattern set is R and whose atom counts match (c_τ). We exhibit a bijection between $\mathcal{C}(R, (c_\tau))$ and the set of pattern labellings $\ell \colon U \to R$ in which label τ occurs exactly c_τ times.

Forward map. Given $\mathcal{S}' \in \mathcal{C}(R, (c_\tau))$, define $\ell_{\mathcal{S}'}(x) = \sigma_{\mathcal{S}'}(x)$ where $\sigma_{\mathcal{S}'}(x) = \{i \in [m] : x \in S'_i\}$. The codomain is R by definition of the class, and the frequency of each τ in $\ell_{\mathcal{S}'}$ is $|A_\tau(\mathcal{S}')| = c_\tau$.

Inverse map. Given a labelling $\ell \colon U \to R$ with frequencies (c_τ), define $S'_i = \{x \in U : i \in \ell(x)\}$ for each $i \in [m]$. The resulting collection $\mathcal{S}'$ has $\sigma_{\mathcal{S}'}(x) = \ell(x)$, so its realised-pattern set is the image of ℓ, which equals R because every $\tau \in R$ has $c_\tau \geq 1$. The atom counts are (c_τ).

The two maps are inverses, hence $|\mathcal{C}(R, (c_\tau))| = \binom{u}{(c_\tau)}$. Any prefix-free code that assigns a distinct codeword to every member of $\mathcal{C}(R, (c_\tau))$ has a codeword of length at least $\lceil \lg |\mathcal{C}(R, (c_\tau))| \rceil = \lceil L^*(\mathcal{S}) \rceil$ bits, and an enumerative index over the class achieves this bound up to the ceiling, by the method of types [55, Ch. 11, § 11.1]. □

[55]: Cover et al. (2006), *Elements of Information Theory*

The specific application of the method of types to indexed set collections, yielding Proposition 5.4.1, is the contribution of this chapter. The underlying technique is classical and due to Csiszár; the enumerative coding of type classes is treated in Cover and Thomas [55, Ch. 11, § 11.1].

The enumerative encoding of the membership-pattern sequence over the realised-pattern alphabet is a direct application of the method of types [55, Ch. 11, § 11.1]. The specific application of this technique to indexed set collections, yielding the atom-based bound of Proposition 5.4.1, is the contribution of this chapter; we have not found a prior statement of it in the literature on compressed set representations.

5.4.3 Relative position

We place $L^*(\mathcal{S})$ between the combinatorial lower bound on $\mathcal{S}$ and the two standard counting upper bounds.

The atom-based bound $L^*(\mathcal{S})$ is dominated separately by two upper bounds, $H_{wc}(\mathcal{S})$ and $u \lg(2^m - 1)$. The two upper bounds differ in the side information each assumes and are incomparable.

Proposition 5.4.2 (Atom bound against per-set entropy) *For every collection $\mathcal{S}$ with row sizes $(|S_i|)_{i=1}^m$,*

$$L^*(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. Let R be the realised-pattern set and (c_τ) the atom counts of $\mathcal{S}$. Every element of the atom-count type class $\mathcal{C}(R,(c_\tau))$ determines an m-tuple of sets $(S'_1,\dots,S'_m)$ by $S'_i = \bigcup_{\tau\ni i} A'_\tau$, where (A'_τ) is the atom partition realising the given counts. The row sizes are $|S'_i| = \sum_{\tau\ni i} c_\tau = |S_i|$, so the image lies in the class of m-tuples with the prescribed row sizes $(|S_i|)$.

The map is injective. Given the m-tuple $(S'_1,\dots,S'_m)$, the atom A'_τ is recovered as $\bigcap_{i\in\tau} S'_i \cap \bigcap_{i\notin\tau}(U \setminus S'_i)$, so the labelled partition $(A'_\tau)_{\tau\in R}$ is determined by the m-tuple. Distinct labelled partitions map to distinct m-tuples.

Source and target cardinalities give

$$\binom{u}{(c_\tau)} \;=\; |\mathcal{C}(R,(c_\tau))| \;\le\; \prod_{i=1}^{m}\binom{u}{|S_i|},$$

and taking base-2 logarithms yields $L^*(\mathcal{S}) \le H_{wc}(\mathcal{S})$. □

Proposition 5.4.3 (Atom bound against coverage count) *If $\mathcal{S}$ realises k patterns, with $1 \le k \le 2^m - 1$, then*

$$L^*(\mathcal{S}) \;\le\; u \lg k \;\le\; u \lg(2^m - 1).$$

Proof. The multinomial coefficient satisfies $\binom{u}{(c_\tau)} = \frac{u!}{\prod_\tau c_\tau!} \le k^u$, since the left-hand side counts pattern sequences of length u over an alphabet of size k with prescribed frequencies, and k^u counts all pattern sequences of length u over the same alphabet. Taking logarithms gives $L^*(\mathcal{S}) \le u \lg k$. Under the convention of Section 5.1, the empty pattern is excluded, so $k \le 2^m - 1$ and the second inequality follows. □

The two upper bounds are incomparable. Neither dominates the other for all collections.

The incomparability of H_{wc} and $u \lg(2^m - 1)$ is instance-dependent. Small row sizes favour H_{wc}, high row density favours $u \lg(2^m - 1)$, and there is no universal ordering between them.

Proposition 5.4.4 (Incomparability of upper bounds) *There exist collections on which $H_{wc}(\mathcal{S}) > u \lg(2^m - 1)$ and collections on which $H_{wc}(\mathcal{S}) < u \lg(2^m - 1)$. Therefore no universal inequality between H_{wc} and $u \lg(2^m - 1)$ holds.*

Proof. The two instances exhibited in the examples below witness strict inequalities in opposite directions. □

Example 5.4.1 (Disjoint balanced instance) Let $m = 2$, $u = 10$, and $S_1 = \{1, \ldots, 5\}$, $S_2 = \{6, \ldots, 10\}$. Then

$$H_{wc}(\mathcal{S}) = 2\lg\binom{10}{5} = 2\lg 252 \approx 15.95, \qquad u\lg(2^m - 1) = 10\lg 3 \approx 15.85,$$

so $H_{wc}(\mathcal{S}) > u\lg(2^m - 1)$.

Example 5.4.2 (Singleton instance) Let $m = 2$, $u = 2$, and $S_1 = \{1\}$, $S_2 = \{2\}$. Then

$$H_{wc}(\mathcal{S}) = 2\lg\binom{2}{1} = 2, \qquad u\lg(2^m - 1) = 2\lg 3 \approx 3.17,$$

so $H_{wc}(\mathcal{S}) < u\lg(2^m - 1)$.

A sharper intermediate upper bound counts binary matrices with prescribed row sums and no zero column. Its inclusion-exclusion expression squeezes between L^* and the minimum of the two standard bounds, and the treatment is deferred to the appendix.

The picture is then that $L^*(\mathcal{S})$ is bounded above by $H_{wc}(\mathcal{S})$ and by $u\lg(2^m - 1)$ separately, with each upper bound derived from different side information, row sizes in one case and coverage in the other, and with no ordering between the two upper bounds. A sharper upper bound $L^+_{\text{rows}}(\mathbf{s})$ counts binary matrices with row sums $\mathbf{s} = (|S_i|)$ and no zero column via inclusion-exclusion, and satisfies $L^*(\mathcal{S}) \le L^+_{\text{rows}}(\mathbf{s}) \le \min(H_{wc}(\mathcal{S}), u\lg(2^m - 1))$; we defer the derivation to Chapter A.

5.4.4 Convergence and motivation

The atom-based bound $L^*(\mathcal{S})$ is simultaneously a lower bound and an achievable upper bound in bit-space. The encoding that meets it is a flat string over the realised-pattern alphabet and provides no structure from which queries could be supported efficiently.

The atom-based bound is tight up to the ceiling.

Proposition 5.4.5 (Achievability of L^*) *For every collection $\mathcal{S}$ under the convention $U = \bigcup_i S_i$, there is an encoding of $\mathcal{S}$ of length exactly $\lceil L^*(\mathcal{S})\rceil$ bits given the side information $(R(\mathcal{S}), (c_\tau)_{\tau \in R(\mathcal{S})})$.*

Proof. Order the universe as $U = \{x_1, \ldots, x_u\}$ and form the pattern sequence $\pi(\mathcal{S}) = (\sigma(x_1), \ldots, \sigma(x_u))$ over the alphabet $R(\mathcal{S})$. By the bijection in the proof of Proposition 5.4.1, the map $\mathcal{S} \mapsto \pi(\mathcal{S})$ is a bijection between indexed collections in the atom-count type class and sequences of length u over $R(\mathcal{S})$ with prescribed frequencies (c_τ). Enumerative coding of the type class assigns a distinct codeword of length $\lceil \lg \binom{u}{(c_\tau)} \rceil = \lceil L^*(\mathcal{S})\rceil$ bits to each sequence [55, Ch. 11, § 11.1].

[55]: Cover et al. (2006), *Elements of Information Theory*

The decoder, given the side information and the codeword, reconstructs $\pi(\mathcal{S})$ and then sets $S_i = \{x_j : i \in \sigma(x_j)\}$ for each $i \in [m]$, recovering $\mathcal{S}$. □

Meeting L^* with a flat pattern-sequence code is algorithmically trivial but structurally useless. None of the five operations of Definition 1.2.2 can be answered on the codeword without decompressing most of it.

The encoding that achieves Proposition 5.4.5 is a flat string over $R(\mathcal{S})$. It has no hierarchical organisation, no parent-child relations among the sets of $\mathcal{S}$, and no local structure from which rank, select, or predecessor queries can be answered without scanning the entire codeword. Its existence shows that the atom-based bound is tight in bit-space but says nothing about queryable representations. Any queryable representation of $\mathcal{S}$ must therefore be judged not against H_{wc} alone but against L^*.

The polynomial-time construction of Section 5.5 is judged against L^* in bits and against the tree-extraction primitives of Chapter 4 in query time. The gap measures the price of query support, not combinatorial inefficiency.

Read in the opposite direction, Proposition 5.4.5 says that the gap between the cost of any queryable representation of $\mathcal{S}$ and the atom-based bound $L^*(\mathcal{S})$ is the price of query support, not a residual combinatorial

inefficiency. The polynomial-time construction of the next section yields a queryable hierarchy of cost $L_{\mathrm{SUM}}(\mathcal{S})$, and the gap $L_{\mathrm{SUM}}(\mathcal{S}) - L^*(\mathcal{S})$ is exactly the overhead needed to make the representation support the five operations of Definition 1.2.2 through the tree-extraction primitives of Chapter 4. The full development of the convergence of Proposition 5.4.5 with the strongest one-level encoding in the lattice closure framework is carried out in Chapter A.

5.5 Set-Union Matching

The hierarchies of Section 5.2 and Section 5.3 share one structural constraint. Their internal nodes are sets already in $\mathcal{S}$, plus at most the anchors $\emptyset$ and U. The constraint shapes their limits. When $A, B \in \mathcal{S}$ are incomparable, no set of $\mathcal{S}$ serves as a common ancestor smaller than U; the containment hierarchy routes both through U, and the symdiff framework falls back on the mixed insertion and deletion marks of the indel tree (Definition 5.3.3). Alanko et al. [7] observe that augmenting with synthetic sets such as $S_i \cap S_j$ could reduce space, but note that the resulting optimisation may be intractable. Gagie et al. [9] leave optimal augmentation as an explicit open problem.

Containment, insertion, and symdiff hierarchies all draw their internal nodes from the collection itself, augmented at most by $\emptyset$ or U. Synthetic nodes such as $S_i \cap S_j$ or $S_i \cup S_j$ are absent from all three frameworks.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[9]: Gagie et al. (2026), *Compressed Set Representations based on Set Difference*

We introduce one synthetic construct, the union. For incomparable A and B the set $M = A \cup B$ is the smallest set containing both, so encoding A and B against M costs strictly less than encoding them against U whenever $|M| < u$. We quantify the saving by a two-bitvector description of the pair, define a cost functional on the resulting forest, specify a greedy construction, and prove that the output never exceeds the per-set baseline $H_{wc}(\mathcal{S})$.

Fix two sets $A, B \subseteq U$ and write $M = A \cup B$, $k = |A \cap B|$, $l = |A \setminus B|$, $r = |B \setminus A|$, so $|M| = k + l + r$. Given M, the pair (A, B) is recovered from two bitvectors. A bitvector of length $|M|$ marks the k elements of the intersection, selecting one of $\binom{|M|}{k}$ possibilities. A second bitvector of length $l+r$ separates the symmetric-difference elements into those in $A \setminus B$ and those in $B \setminus A$, selecting one of $\binom{l+r}{l}$ possibilities. The product of the two counts is the multinomial coefficient $\binom{|M|}{k,l,r}$. A short self-delimiting header records k and l, after which $r = |M| - k - l$ follows.

Two bitvectors describe A and B jointly given M. The first records which of the $|M|$ elements lie in the intersection; the second, among the symmetric-difference elements, records which belong to A and which to B. Every bit is one decision on one element of M.

Definition 5.5.1 (Merge cost) *For $A, B \subseteq U$ with $M = A \cup B$, $k = |A \cap B|$, $l = |A \setminus B|$, $r = |B \setminus A|$, the merge cost of the pair (A, B) is*

$$w(A, B) \;=\; \lg \binom{|M|}{k, l, r} \;+\; c(|M|),$$

where $c(|M|) = \lceil \lg(|M| + 1) \rceil + \lceil \lg(|M| - k + 1) \rceil$ encodes k and l. When $A \cap B = \emptyset$, the multinomial reduces to $\binom{l+r}{l}$ and the overhead to $\lceil \lg(|M| + 1) \rceil$.

The cost $w(A, B)$ is measured against M, not U. When M comes from a prior merge and is already available to the decoder, only $w(A, B)$ is spent to recover the pair; no further support cost is charged for M itself. Describing A and B jointly through M costs less than describing them independently as subsets of M. The two-bitvector decomposition treats the three Venn regions $A \cap B$, $A \setminus B$, $B \setminus A$ as a single multinomial

choice, and $\lg \binom{|M|}{k,l,r} \le \lg \binom{|M|}{|A|} + \lg \binom{|M|}{|B|}$ whenever $A \cap B \neq \emptyset$. A general inequality covering any number of children is proved in Chapter A; the two-child case used here is its immediate specialisation.

Iterating merges over the whole collection yields a binary forest.

Definition 5.5.2 (SUM forest and cost) *A SUM forest on $\mathcal{S}$ is a binary rooted forest F with m leaves, one per $S_i \in \mathcal{S}$, and every internal node v labelled by $M_v = A_v \cup B_v$ where A_v, B_v are its children. Its cost is*

$$\Phi(F) = \sum_{R \in \mathrm{roots}(F)} \lg \binom{u}{|R|} + \sum_{v \in \mathrm{internal}(F)} w(A_v, B_v).$$

The first sum is the support cost, with each root encoded against U; the second is the structural cost, with each internal node contributing the merge cost of its decomposition.

The trivial forest F_0 has m isolated roots and no internal nodes, so $\Phi(F_0) = \sum_i \lg \binom{u}{|S_i|} = H_{wc}(\mathcal{S})$. Every other SUM forest is measured against this baseline.

Greedy minimum-weight matching extracts, at each level, the pair with the smallest merge cost, removes its endpoints, and repeats. When the number of remaining roots is odd, one root is left unmatched and passes unchanged to the next level.

The procedure builds a SUM forest level by level. At each level, evaluate the merge cost $w(R_i, R_j)$ on every pair of current roots, then extract a greedy minimum-weight matching: repeatedly choose the cheapest remaining pair and remove its endpoints. Each matched pair (A, B) becomes a new root $M = A \cup B$ with children A and B. Each level halves the root count, so the procedure terminates after at most $\lceil \lg m \rceil$ levels. algorithm 2 states this, together with the book-keeping that the optimal-forest variant below requires.

Algorithm 2: Set-Union Matching with optimal-forest stopping.

Input: Collection $\mathcal{S} = \{S_1, \ldots, S_m\}$ over universe U.
Output: A SUM forest F^* achieving the smallest value of Φ across all levels $0, 1, \ldots, \lceil \lg m \rceil$.

```
F ← trivial forest on S (no internal nodes);
F* ← F; Φ* ← Φ(F);
for t = 1, 2, ..., ⌈lg m⌉ do
    compute w(R_i, R_j) for every pair of current roots R_i, R_j in F;
    while at least two roots of F remain unmatched at level t do
        pick an unmatched pair (A, B) minimising w(A, B);
        replace A, B in F by a new root M = A ∪ B with children A, B;
    if Φ(F) < Φ* then
        F* ← F; Φ* ← Φ(F);
return F*;
```

The level-0 pass evaluates $\binom{m}{2}$ merge costs, each in time $O(|A| + |B|)$ from sorted representations of the sets. Later levels evaluate fewer pairs but on larger unions; across $\lceil \lg m \rceil$ levels the worst-case total is $O(m^2 n \lg m)$, where $n = \sum_i |S_i|$. The algorithm is polynomial in the input size.

A unary synthetic node would describe the decomposition of a parent M into a single child $B \subsetneq M$, adding one encoding step without producing a second branch that shares M. The binomial identity $\binom{|C|}{|B|}\binom{|B|}{|A|} = \binom{|C|}{|A|}\binom{|C|-|A|}{|B|-|A|}$ forces $\lg \binom{|C|}{|B|} + \lg \binom{|B|}{|A|} \ge \lg \binom{|C|}{|A|}$ whenever $A \subseteq B \subseteq C$; the stacked encoding never saves bits.

SUM is binary by construction. Every internal node has exactly two children, and no synthetic node of outdegree one appears. The restriction is not arbitrary. The unary-node lemma of Chapter A shows that stacking a synthetic intermediate with a single child never reduces the encoding cost, so excluding unary configurations forfeits nothing.

The greedy choice at each level is a local decision. It minimises the sum of merge costs at the current level but cannot anticipate the configuration the next level inherits. The cost Φ may therefore rise between successive levels.

Greedy matching minimises the sum of merge costs at a single level but cannot look ahead. A cheap matching at level t can lock the remaining roots into an expensive merge at level $t+1$.

Proposition 5.5.1 (Non-monotonicity) *There exist collections $\mathcal{S}$ for which $\Phi(F_{t+1}) > \Phi(F_t)$ at some level t.*

Proof. Let $\mathcal{S}$ contain four pairwise-disjoint sets of size $u/4$. Level-0 matching merges them into two roots of size $u/2$. Each merge adds $\lg \binom{u/2}{u/4} \approx u/2$ bits of structural cost and reduces support cost by $2\lg\binom{u}{u/4} - \lg\binom{u}{u/2}$; the net change is a decrease. Level-1 matching merges the two size-$u/2$ roots into U. The structural cost gains $\lg\binom{u}{u/2} \approx u$ bits and the support cost drops from $\lg\binom{u}{u/2}$ to zero. The gain exceeds the saving. □

Figure 5.3 traces the cost across the three snapshots of this construction. The minimum of Φ is attained at the intermediate snapshot, not at the final one.

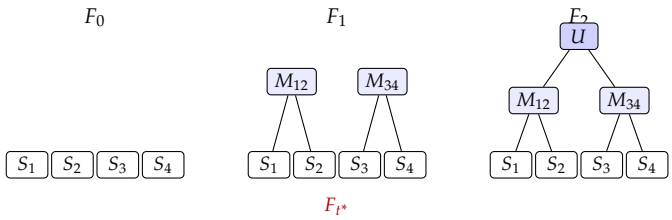

Figure 5.3: SUM on four pairwise-disjoint sets of size $u/4$. From F_0 to F_1, two merges pair (S_1, S_2) and (S_3, S_4) into synthetic unions of size $u/2$; the cost drops from $4\lg\binom{u}{u/4}$ to $2\lg\binom{u}{u/2} + 2\lg\binom{u/2}{u/4}$. From F_1 to F_2, a single merge produces U and the cost changes to $\lg\binom{u}{u/2} + 2\lg\binom{u/2}{u/4}$; the structural gain $\lg\binom{u}{u/2}$ of the new merge outweighs the support saving $\lg\binom{u}{u/2}$ it removes, as proved in Proposition 5.5.1. The minimum is $F_{t^*} = F_1$.

Recording the cost of every snapshot during execution and returning the best one turns a local procedure into a globally bounded one. The book-keeping is already present in algorithm 2.

Definition 5.5.3 (Optimal-forest variant) *The optimal-forest variant of SUM records $\Phi(F_t)$ after each level and returns the snapshot F^* that minimises Φ across $t = 0, 1, \ldots, \lceil \lg m \rceil$. We write $L_{\mathrm{SUM}}(\mathcal{S}) = \Phi(F^*)$.*

Including F_0 among the candidate snapshots yields the universal guarantee against the baseline.

Theorem 5.5.2 (Universal upper bound) *For every collection $\mathcal{S}$,*

$$L_{\mathrm{SUM}}(\mathcal{S}) \;\le\; H_{wc}(\mathcal{S}).$$

Proof. F_0 is a valid snapshot and $\Phi(F_0) = H_{wc}(\mathcal{S})$, so the minimum across snapshots is at most $H_{wc}(\mathcal{S})$. □

The union construction endows a SUM forest with a pure-containment structure that sets it apart from the insertion and indel trees of the previous sections.

Three label conventions across the chapter: an insertion tree (Definition 5.2.5) carries positive labels that accumulate from $\emptyset$; an indel tree (Definition 5.3.3) carries signed labels in $[-u..-1] \cup [1..u]$ and records both additions and deletions; a SUM tree carries positive labels only, but reads them as elements removed from a root set R. Three frameworks, three label alphabets, three directions of reduction.

Proposition 5.5.3 (Deletion tree structure) *For every internal node v with children A_v, B_v, both inclusions $A_v \subseteq M_v$ and $B_v \subseteq M_v$ hold, so every parent-child edge in a SUM forest is a containment.*

Proof. Immediate from $M_v = A_v \cup B_v$. □

Expanding each edge (M_v, A_v) into a chain of $|M_v \setminus A_v|$ edges, one per element of $M_v \setminus A_v$, and similarly for (M_v, B_v), produces a labelled ordinal tree with labels in $[1..u]$. A label on the path from a root R down to a leaf S marks an element present in R but absent from S. Once an element is absent at some node, the containment $A \subseteq M$ along every descendant edge keeps it absent throughout the subtree, so no label repeats on a single root-to-leaf path. We call such a tree a *deletion tree*. It is complementary to the insertion tree of Definition 5.2.5, whose labels accumulate from $\emptyset$, and to the indel tree of Definition 5.3.3, whose signed labels record both directions of change.

When $A \subset B$ already holds in $\mathcal{S}$, the merge (A, B) gives $M = B$, $k = |A|$, $l = 0$, and $w(A, B) = \lg \binom{|B|}{|A|} + O(\lg |B|)$. SUM recovers the containment-entropy contribution of A given B (Definition 5.2.2) up to the self-delimiting overhead. On incomparable pairs, where the containment hierarchy has no option but U, SUM routes through $M = A \cup B$ and pays $\lg \binom{u}{|M|} + w(A, B)$ instead of the combined $\lg \binom{u}{|A|} + \lg \binom{u}{|B|}$; the former is strictly smaller whenever $|M| < u$.

The output of SUM sits inside a broader optimisation problem developed in Chapter A, where the natural ambient space for reference encoding is the distributive lattice [56] of realised membership-pattern upsets and the objective is a Directed Steiner Arborescence [57] in that lattice. SUM corresponds to one variant of the appendix's landscape, restricted to unions for the synthetic nodes, to a binary multi-level hierarchy for the forest structure, to the two-child multinomial encoding at every merge, and to level-by-level greedy construction. The gap

$$L_{\mathrm{SUM}}(\mathcal{S}) \;-\; L^*(\mathcal{S})$$

between the SUM cost and the atom-based bound L^* of Proposition 5.4.1 measures the overhead of supporting the five operations of Definition 1.2.2. The flat encoding that meets L^* (Proposition 5.4.5) supports none of them. The next chapter turns the deletion tree of Proposition 5.5.3 into a queryable data structure by applying the tree-extraction framework of Chapter 4 to its labelled ordinal form.

[56]: Birkhoff (1937), *Rings of sets*

[57]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

Appendix

A Lattice closure framework for set collection encoding

A.1 Formulation and structure 78
A.2 Variant landscape, adjacent literature, and open problems 83

The chapter collects theoretical material that belongs with Chapter 5 but sits outside its narrative: a lattice-theoretic description of the candidate reference family, a combinatorial optimisation problem on that family, two structural lemmas that justify design choices made in Section 5.5, and a convergence theorem that matches Proposition 5.4.5.

The purpose of this appendix is to describe, in one place, the combinatorial optimisation problem within which the constructions of Chapter 5 live. The atom-based bound $L^*(\mathcal{S})$ of Section 5.4 establishes a lower bound that every representation of $\mathcal{S}$ must meet, and Section 5.5 exhibits a polynomial-time greedy construction whose encoding cost $L_{\mathrm{SUM}}(\mathcal{S})$ sits above it. We ask what broader optimisation problem contains both, and what its structure says about the gap between them. The answer is a Directed Steiner Arborescence problem on the lattice closure of $\mathcal{S}$, augmented with binomial arc costs.

The material that follows is theoretical groundwork. No new data structure is constructed, and no existing bound of the thesis is sharpened. Section A.1 develops the lattice closure, describes it via the poset of realised membership patterns, formulates the Directed Steiner problem, proves two structural lemmas on synthetic nodes and on multi-child encoding, and closes with a convergence theorem linking the flat atom encoding to the lower bound L^* of Section 5.4. Section A.2 taxonomises seven variants of the optimisation, places the greedy construction of Section 5.5 within the taxonomy, surveys adjacent literature, and records five open problems.

A.1 Formulation and structure

The lower bound L^* of Proposition 5.4.1 and the upper bound L_{SUM} of Theorem 5.5.2 are two extremes of a single optimisation problem whose candidate reference family is the lattice closure of $\mathcal{S}$ under binary intersection and union.

Given the atom-based bound L^* of Section 5.4 and the SUM cost L_{SUM} of Section 5.5, this appendix develops the optimisation problem that contains both. The candidate family of reference sets is the lattice closure of $\mathcal{S}$ under binary intersection and union. Each S_i is encoded as a bitvector relative to a strict superset drawn from the closure, and the total cost sums the per-edge contributions along the resulting parent-pointer tree. Choosing parents to minimise this sum is the problem formulated below.

Definition A.1.1 (Lattice closure) *Let $\mathcal{S} = \{S_1, \ldots, S_m\}$ be a family of finite subsets of a universe U. The lattice closure $\mathcal{C}(\mathcal{S})$ is the smallest family of subsets of U containing $\mathcal{S}$ and closed under binary intersection and binary union.*

[56]: Birkhoff (1937), *Rings of sets*

Set intersection and union distribute over one another on a family of subsets of a fixed ground set, so $(\mathcal{C}(\mathcal{S}), \cap, \cup)$ is a finite distributive lattice [56, Theorem I.6]. Under the convention $U = \bigcup_i S_i$ of Section 5.1, the universe is the join of $\mathcal{S}$ and belongs to $\mathcal{C}(\mathcal{S})$ unless $\mathcal{S}$ already contains a covering set. We write $\mathcal{L} = (\mathcal{C}(\mathcal{S}) \cup \{U\}) \setminus \{\emptyset\}$ for the ambient family of useful reference candidates; the empty set, which may belong to $\mathcal{C}(\mathcal{S})$ when some iterated intersection of generators vanishes, is excluded because no S_i satisfies $S_i \subsetneq \emptyset$ and it therefore cannot serve as a reference.

The encoding model and the per-edge counting cost are those used throughout Chapter 5. The $O(|S_i|)$ bitvector payload is absorbed into the standard $O(n + m \lg n)$ overhead of Theorem 2.2.2.

The encoding model follows Chapter 5. Each S_i is represented as a

bitvector of length $|P_i|$ against a chosen $P_i \in \mathscr{L}$ with $S_i \subsetneq P_i$, at a cost of $\lg \binom{|P_i|}{|S_i|}$ bits plus an $O(|S_i|)$ payload. A tree-shaped encoding of $\mathcal{S}$ specifies, for each $i \in [m]$, a parent $P_i \in \mathscr{L}$, and a rooted tree on the synthetic references inside $\mathscr{L} \setminus \mathcal{S}$ that makes every P_i reconstructible. The cost to minimise is the sum of $\lg \binom{|P|}{|Q|}$ along all parent-child edges of this tree, including edges internal to $\mathscr{L} \setminus \mathcal{S}$.

The lattice $\mathscr{L}$ admits a compact description through the realised patterns of $\mathcal{S}$. Using the membership pattern $\sigma(x)$ and atom $A_\tau = \sigma^{-1}(\tau)$ of Definition 5.4.1, set

$$R = R(\mathcal{S}) = \{\, \sigma(x) : x \in U \,\} \subseteq 2^{[m]} \setminus \{\emptyset\}$$

and order R by subset inclusion. The atoms $\{A_\tau : \tau \in R\}$ partition U into $|R|$ nonempty parts. An *antichain* of $(R, \subseteq)$ is a set of pairwise incomparable elements of R, and an *upset* of $(R, \subseteq)$ is a set $Q \subseteq R$ such that $\tau \in Q$ and $\tau \subseteq \tau' \in R$ imply $\tau' \in Q$. Every upset Q is determined by its minimal elements, an antichain of R; every antichain of R generates a unique upset as the collection of all elements of R above it. For a nonempty upset Q, write $X_Q = \bigcup_{\tau \in Q} A_\tau$. Let $J(R)$ denote the number of upsets of R, counting both the empty upset and R itself.

Theorem A.1.1 (Closure representation) *The map $Q \mapsto X_Q$ is a lattice isomorphism from the nonempty upsets of $(R, \subseteq)$, ordered by set inclusion and equipped with pointwise union and intersection, onto $(\mathscr{L}, \subseteq, \cup, \cap)$. In particular $|\mathscr{L}| = J(R) - 1$.*

Proof. Every generator S_i equals X_{Q_i} for the upset $Q_i = \{\tau \in R : i \in \tau\}$, since $x \in S_i$ iff $i \in \sigma(x)$ iff $\sigma(x) \in Q_i$ iff $x \in A_{\sigma(x)} \subseteq X_{Q_i}$. The universe U equals X_R because the atoms partition U. Atoms of distinct patterns are pairwise disjoint, so for any two subsets $Q_1, Q_2 \subseteq R$,

$$X_{Q_1} \cap X_{Q_2} = X_{Q_1 \cap Q_2}, \qquad X_{Q_1} \cup X_{Q_2} = X_{Q_1 \cup Q_2},$$

and the pointwise intersection and union of two upsets are again upsets. Every element of $\mathscr{C}(\mathcal{S}) \cup \{U\}$ is obtained from the S_i and U by iterated $\cap, \cup$, so by these identities every such element is X_Q for some upset Q of R. Restricting to nonempty upsets excludes exactly the image $X_\emptyset = \emptyset$, which is outside $\mathscr{L}$ by construction. The map is therefore surjective onto $\mathscr{L}$.

For injectivity, suppose $X_{Q_1} = X_{Q_2}$ for nonempty upsets Q_1, Q_2. Then for every $\tau \in R$, the atom A_τ is a nonempty subset of X_{Q_1} iff $\tau \in Q_1$, and the same holds for Q_2; hence $Q_1 = Q_2$. The map is thus bijective, commutes with $\cap$ and $\cup$ by the identities above, and preserves and reflects the order by the same atom-by-atom argument. It is therefore a lattice isomorphism, and $|\mathscr{L}| = J(R) - 1$. □

The theorem is Birkhoff's representation of finite distributive lattices, specialised to $\mathscr{L}$ via the realised-pattern poset. The size $|\mathscr{L}|$ is controlled by the combinatorics of R rather than by $|U|$ or $|\mathcal{S}|$ directly. A coarse bound invokes the *Dedekind number* $D(m)$ [58, pp. 103–148], defined as the number of antichains in the Boolean lattice $2^{[m]}$; its asymptotic growth $\log_2 D(m) \sim \binom{m}{\lfloor m/2 \rfloor}$ is due to Kleitman [59]. Every nonempty upset of R has a unique minimal antichain inside R, and antichains of R inject

Birkhoff's representation theorem for finite distributive lattices [56] identifies every such lattice with the lattice of upsets of its poset of join-irreducibles; here the join-irreducibles of $\mathscr{L}$ are the principal upsets $\uparrow \tau = \bigcap_{i \in \tau} S_i$ for $\tau \in R$, so the poset of join-irreducibles is $(R, \subseteq)$ itself.

[58]: Dedekind (1897), *Über Zerlegungen von Zahlen durch ihre grössten gemeinsamen Teiler*

[59]: Kleitman (1969), *On Dedekind's problem: The number of monotone Boolean functions*

into antichains of $2^{[m]}$; hence $|\mathscr{C}(\mathcal{S})| \leq D(m)$. Equality occurs only in the degenerate regime $R = 2^{[m]} \setminus \{\emptyset\}$, where every nonempty Venn region is realised. For realistic collections $|R| \ll 2^m - 1$ and $J(R)$ is many orders of magnitude below $D(m)$. A sharper bound uses the width w of $(R, \subseteq)$: every upset of R has a minimal antichain of size at most w, and antichains of size at most w in a poset of size $|R|$ number at most $\sum_{j=0}^{w} \binom{|R|}{j}$, so

$$|\mathscr{L}| \;=\; J(R) - 1 \;\leq\; \sum_{j=0}^{w} \binom{|R|}{j}.$$

The DAG $G_{\mathscr{L}}$ inherits the strict-containment order on $\mathscr{L}$ as its arc set, with costs drawn from the counting bound of Section 5.4.

With the structure of $\mathscr{L}$ established and its size bounded in terms of R, the reference-encoding optimisation admits a standard graph-theoretic formulation. Build an arc-weighted directed acyclic graph $G_{\mathscr{L}}$ on the vertex set $\mathscr{L}$ by inserting an arc $P \to Q$ whenever $Q \subsetneq P$, with cost

$$c(P, Q) \;=\; \lg \binom{|P|}{|Q|}.$$

The root U has no incoming arc; every other vertex has at least one.

[57]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

Theorem A.1.2 (Reference encoding as a Steiner arborescence) *The minimum cost of a tree-shaped reference encoding of $\mathcal{S}$ equals the weight of a minimum-cost directed Steiner arborescence [57, p. 74] in $G_{\mathscr{L}}$ rooted at U and covering the terminals $\{S_1, \ldots, S_m\}$:*

$$\mathrm{OPT}_{\mathrm{tree}}(\mathcal{S}) \;=\; \min_{T} \sum_{(P,Q)\in T} \lg \binom{|P|}{|Q|}.$$

Proof. Every tree-shaped encoding assigns to each S_i a directed path from U to S_i through strict-containment arcs of $G_{\mathscr{L}}$; the union of these paths is an arborescence in $G_{\mathscr{L}}$ rooted at U with the S_i as terminals, and each arc contributes $\lg \binom{|P|}{|Q|}$ to the cost exactly once. Conversely, an arborescence in $G_{\mathscr{L}}$ rooted at U and covering $\{S_1, \ldots, S_m\}$ is a tree-shaped encoding of equal cost. □

Restricting the candidate family to $\mathcal{S} \cup \{U\}$ removes every synthetic node. Each S_i is then referenced directly from a strictly larger S_j or from U; assigning each S_i its smallest containing parent is optimal and polynomial, and the total cost is Alanko's containment entropy $L(\mathcal{S})$ of Definition 5.2.2.

Restricting the candidate family to $\mathcal{S} \cup \{U\}$ removes every synthetic node from $\mathscr{L}$. Each S_i is then referenced directly from a strictly larger S_j, or from U when no such S_j exists. Assigning each S_i its smallest containing parent is optimal and polynomial, giving total cost

$$L(\mathcal{S}) \;=\; \sum_{i=1}^{m} \lg \binom{|p(S_i)|}{|S_i|},$$

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

[57]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

Alanko's containment entropy of Definition 5.2.2 [7]. The general problem on an arbitrary candidate family $\mathscr{F}$ with nonnegative arc costs is Directed Steiner Tree, hence NP-hard [57]. The closure-specific variant of Theorem A.1.2, with arc costs $\lg \binom{|P|}{|Q|}$ on $\mathscr{L}$, is not reduced from any standard hard problem in this thesis and its complexity status remains open.

Two elementary inequalities constrain every cost-minimising arborescence in $G_{\mathscr{L}}$, independently of the complexity status above. The first rules out synthetic vertices of outdegree one; the second bounds the cost of encoding sibling children jointly.

Lemma A.1.3 (Chain superadditivity) *For every triple of finite sets $A \subsetneq B \subsetneq C$,*

$$\lg\binom{|C|}{|B|} + \lg\binom{|B|}{|A|} \;>\; \lg\binom{|C|}{|A|}.$$

Proof. Double counting the triples (Z, Y, C) with $Z \subseteq Y \subseteq C$, $|Z| = |A|$, $|Y| = |B|$ gives the identity

$$\binom{|C|}{|B|}\binom{|B|}{|A|} \;=\; \binom{|C|}{|A|}\binom{|C| - |A|}{|B| - |A|}.$$

Strict inclusions $A \subsetneq B \subsetneq C$ force $|C| - |A| \geq 2$ and $0 < |B| - |A| < |C| - |A|$, hence $\binom{|C|-|A|}{|B|-|A|} \geq 2 > 1$. Taking lg yields the inequality. □

A consequence is structural. If P is a synthetic vertex of outdegree one in an arborescence T, with incoming arc $P' \to P$ and unique outgoing arc $P \to Q$, the chain $P' \to P \to Q$ contributes strictly more than a direct $P' \to Q$ edge; deleting P and reattaching Q under P' strictly lowers the cost. Every cost-minimising arborescence therefore excludes synthetic vertices of outdegree one, and the SUM algorithm of Section 5.5 constructs its output as a binary forest whose internal nodes have outdegree exactly two.

Contracting a unary synthetic node in an arborescence strictly reduces the total cost by Lemma A.1.3. Every cost-minimising arborescence omits such vertices, so the SUM algorithm of Section 5.5 loses nothing by excluding them at design time.

Lemma A.1.4 (Joint sibling encoding) *Let P be a finite set and $Q_1, \ldots, Q_d \subseteq P$. For $b \in \{0,1\}^d$, let*

$$a_b \;=\; \big|\{\, x \in P : [x \in Q_j] = b_j \text{ for all } j \in [d] \,\}\big|.$$

Then

$$\lg\binom{|P|}{(a_b)_{b\in\{0,1\}^d}} \;\leq\; \sum_{j=1}^{d} \lg\binom{|P|}{|Q_j|}.$$

The inequality is strict whenever the marginals $|Q_j|$ admit more than one compatible joint-count vector.

Proof. The multinomial $\binom{|P|}{(a_b)}$ counts the labellings $\ell\colon P \to \{0,1\}^d$ whose joint counts are exactly (a_b). Each such ℓ projects to d binary labellings $\ell_j\colon P \to \{0,1\}$, with ℓ_j carrying $|Q_j| = \sum_{b:b_j=1} a_b$ ones. The map $\ell \mapsto (\ell_1, \ldots, \ell_d)$ is injective because the coordinate projections determine ℓ. The codomain is the set of tuples of binary labellings whose j-th coordinate has marginal $|Q_j|$, which has cardinality $\prod_j \binom{|P|}{|Q_j|}$; hence

$$\binom{|P|}{(a_b)} \;\leq\; \prod_{j=1}^{d} \binom{|P|}{|Q_j|}.$$

Equality holds iff the image of the projection map is the whole codomain, i.e. iff every tuple of coordinate labellings with the given marginals comes from a joint labelling with joint counts (a_b). That condition reduces to asking whether (a_b) is the only joint-count vector compatible with the marginals. When multiple compatible joint-count vectors exist the inequality is strict. □

The two-child case of Lemma A.1.4 is the merge cost of Definition 5.5.1. A parent $M = A \cup B$ has children A, B with joint counts $k = |A \cap B|$,

The two-child case of Lemma A.1.4, instantiated at $P = M$, $Q_1 = A$, $Q_2 = B$ with $A \cup B = M$, is the merge cost $w(A, B) = \lg\binom{|M|}{k,l,r}$ of Definition 5.5.1. It lies strictly below the independent-child cost for every nontrivial merge, regardless of whether A and B overlap.

$l = |A \setminus B|$, $r = |B \setminus A|$ and implicit count $a_{(0,0)} = 0$ in M (since $A \cup B$ covers M). Under the constraint $a_{(0,0)} = 0$, the marginals $|A|, |B|$ leave k with range $[\max(0, |A| + |B| - |M|), \min(|A|, |B|)]$; this range is a single value only when $|A| \in \{0, |M|\}$ or $|B| \in \{0, |M|\}$, which are the trivial merges where one child equals $\emptyset$ or M. For every nontrivial merge, with both A and B proper nonempty subsets of M, the joint cost $\lg \binom{|M|}{k,l,r}$ is strictly below the independent cost $\lg \binom{|M|}{|A|} + \lg \binom{|M|}{|B|}$, whether or not $A \cap B$ is empty. The general d-child inequality places the same saving at every branching point of the arborescence.

Applying Lemma A.1.4 at the root of the arborescence yields a special case of independent interest. A tree of depth one places every S_i as a direct child of U and charges Lemma A.1.4 with $d = m$ across all siblings. The joint-count vector indexed by $\{0,1\}^m$ specialises to the atom-count vector of Definition 5.4.1: each $b \in \{0,1\}^m$ corresponds to a pattern $\tau = \{j : b_j = 1\} \subseteq [m]$, and the joint count a_b equals c_τ, the number of universe elements in the atom of pattern τ. Under the coverage convention $U = \bigcup_i S_i$, the pattern $\tau = \emptyset$ has $c_\emptyset = 0$, so only patterns $\tau \in R$ contribute to the multinomial.

Theorem A.1.5 (One-level encoding matches L^*) *Under the convention $U = \bigcup_i S_i$, the one-level tree-shaped encoding that places every S_i as a direct child of U and charges the joint cost of Lemma A.1.4 across the m children has total cost*

$$\lg \binom{u}{(c_\tau)_{\tau \in R}} = L^*(\mathcal{S}),$$

matching the atom-based bound of Proposition 5.4.1.

Proof. Instantiate Lemma A.1.4 with $P = U$ and $Q_j = S_j$ for $j \in [m]$. The coverage convention forces $c_\emptyset = 0$, so the multinomial collapses to $\binom{u}{(c_\tau)_{\tau \in R}}$, matching the definition of $L^*(\mathcal{S})$ in Proposition 5.4.1. □

Theorem A.1.5 identifies the flat encoding of Proposition 5.4.5 with the one-level tree-shaped encoding in the lattice framework. Both cost exactly L^* and support no queries beyond full decoding.

The flat encoding that attains L^* in Proposition 5.4.5 coincides with the one-level tree-shaped encoding inside $\mathcal{L}$. Neither direction of $L^*(\mathcal{S}) = \lg \binom{u}{(c_\tau)}$ is slack. The gap between the SUM cost $L_{\text{SUM}}(\mathcal{S})$ of Theorem 5.5.2 and the bound $L^*(\mathcal{S})$ is therefore not a combinatorial inefficiency. The one-level encoding meeting L^* supports no queries; SUM pays a strictly positive surplus in exchange for the deletion-tree structure of Proposition 5.5.3 and the tree-extraction primitives of Chapter 4. Within the lattice framework, that surplus is the cost of query support, not the cost of compression.

Figure A.1 gives a small concrete example on $m = 3$ sets. A synthetic union $M = S_2 \cup S_3$ sits between the root U and the two incomparable sets S_2, S_3, while S_1 attaches directly to U. The arborescence cost is $2 \lg \binom{8}{5} + \lg \binom{5}{4} + \lg \binom{5}{3} \approx 17.26$ bits, strictly below the flat $\lg \binom{8}{5} + \lg \binom{8}{4} + \lg \binom{8}{3} \approx 17.74$ bits that a one-level encoding pays on the same collection. The saving of about 0.49 bits comes from the joint sibling encoding of S_2, S_3 inside a universe of size five, cheaper than treating them as independent subsets of the universe of size eight.

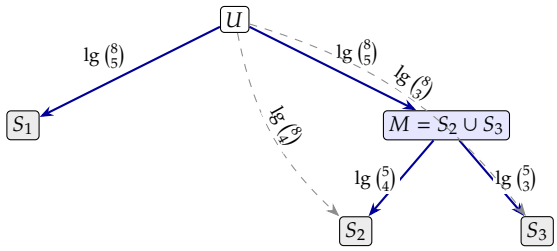

Figure A.1: A minimum-cost directed Steiner arborescence in $G_{\mathcal{L}}$ for $\mathcal{S} = \{S_1, S_2, S_3\}$ over $U = \{1, \ldots, 8\}$, with $S_1 = \{1, 2, 3, 4, 5\}$, $S_2 = \{4, 5, 6, 7\}$, $S_3 = \{5, 6, 8\}$. The synthetic node $M = S_2 \cup S_3 = \{4, 5, 6, 7, 8\}$ has size 5. Solid arcs form the arborescence (cost ≈ 17.26 bits); dashed arcs show the two direct $U \to S_2$ and $U \to S_3$ edges that a flat one-level encoding would use instead (cost ≈ 17.74 bits on this collection).

A.2 Variant landscape, adjacent literature, and open problems

The Directed Steiner formulation of Section A.1 admits many variants depending on the admissible candidate family, on the structural constraints on the arborescence, and on how per-parent costs aggregate across children. We record seven regimes that recur in the compressed-set-representation literature and identify where the construction of Section 5.5 sits. Five open problems follow, together with pointers to adjacent work on concept lattices, the set-basis problem, and space-efficient data structures for finite lattices.

The Steiner formulation of Theorem A.1.2 covers a range of optimisation problems depending on which subfamily of $\mathcal{L}$ is admitted as a reference, which structural restrictions are placed on the arborescence, and how the arc cost is aggregated across children. The greedy algorithm of Section 5.5 sits in one such regime.

Seven variants. *Variant 1 (existing-set references).* The candidate family is $\mathcal{S} \cup \{U\}$ and no synthetic nodes are admitted. Every S_i references either the smallest $S_j \in \mathcal{S}$ strictly containing it, or U when no such S_j exists. The Steiner problem reduces to an assignment problem that is solvable in polynomial time by scanning $\mathcal{S}$ in order of decreasing size and assigning each S_i to its smallest container [7]. The resulting optimum equals Alanko's containment entropy $L(\mathcal{S})$ of Section 5.2, and the assignment-tree construction of Alanko, Bille, Gørtz, Navarro, and Puglisi matches it up to lower-order terms.

[7]: Alanko et al. (2025), *Compact data structures for collections of sets*

Variant 2 (explicit candidate family with arbitrary costs). The candidate family $\mathcal{F} \supseteq \mathcal{S} \cup \{U\}$ is given explicitly as part of the input, with arbitrary non-negative arc costs. This is Directed Steiner Tree in its classical form, whose NP-hardness is classical [57] and which admits an $O(n^{\varepsilon})$-approximation for every constant $\varepsilon > 0$ via the Charikar, Chekuri, Cheung, Dai, Goel, Guha, and Li scheme.

[57]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

Variant 3 (closure via upset antichains). The candidate family is the full lattice closure $\mathcal{L}$, represented compactly through the upset antichains of $(R, \subseteq)$ as in Theorem A.1.1. Enumerating candidates through antichains of R rather than through subsets of U saves space whenever $|R| \ll u$. The optimisation retains the Steiner-hardness of Variant 2 in the worst case but becomes tractable when R has additional structure, for example when it is a chain, a two-dimensional product poset, or a forest.

Variant 4 (parameterised regimes). When $|R|$ or the width $w = \mathrm{width}(R)$ is small, the size of $\mathcal{L}$ is bounded by $\sum_{j \le w} \binom{|R|}{j}$ (Remark **??**), and the Steiner problem on $\mathcal{L}$ admits polynomial-time exact or approximation algorithms. This is the parametric analogue of Variant 3. It is the most plausible regime for datasets in which the collection has a shallow containment hierarchy, or in which the atoms fall into a bounded number of equivalence classes under the subset order.

Variant 5 (two-level union-only). Each S_i is either a direct child of U or a child of a single synthetic union of two siblings. The arborescence has depth at most two. Every materialised union pays its reference cost $\lg \binom{|U|}{|M|}$ once and shares that cost among its two children at joint-Venn cost $\lg \binom{|M|}{k,l,r}$. This is a facility-location-like restriction: the $\binom{m}{2}$ potential synthetic parents are the *facilities*, and the children choose which to open. The resulting optimisation is related to metric facility location on the collection of pairs but with a nonmetric cost structure driven by the binomial coefficients.

Variant 6 (multi-level union-only with joint-Venn per parent). Every parent-child edge is a containment, every internal node is the union of its children, and merge costs use the joint multinomial from Lemma A.1.4. The SUM algorithm of Section 5.5 inhabits this variant as a greedy level-by-level heuristic, merging at each step the pair of candidates whose joint-Venn merge cost of Definition 5.5.1 is smallest. Proposition 5.5.3 shows that the output is a pure deletion tree, so it remains amenable to the tree-extraction machinery of Chapter 4. The open question inside Variant 6 is whether SUM is approximately optimal, and under what structural conditions it is exactly optimal.

Variant 7 (flat atom encoding). Every original set is a direct child of U, and the m children are encoded jointly through the joint-Venn cost across all of them. By Theorem A.1.5, the cost is exactly $L^*(\mathcal{S})$, matching the atom-based bound of Proposition 5.4.1 and the flat encoding of Proposition 5.4.5. The arborescence exposes no hierarchy and supports no efficient queries, but its cost is the information-theoretic limit over the closure family.

The seven variants can be read as a sequence of relaxations. Variant 1 is Alanko's tractable core; Variant 7 is the information-theoretic limit. SUM (Section 5.5) sits in Variant 6 and approximates the ideal of Variant 3 from above.

SUM occupies Variant 6 as a greedy heuristic. Variants 3 and 4 are the tractable refinements most worth pursuing further, since they retain access to the full closure while exploiting the structure of R. Variant 7 is the information-theoretic limit case; its match with Proposition 5.4.5 closes the circle between the lower bound of Section 5.4 and the lattice framework of this appendix.

Formal Concept Analysis [60] builds a lattice from an incidence matrix, with concepts as the closed rectangles. The closure operator is $(\cdot)''$ on objects or attributes; it differs from closure under $(\cap, \cup)$, so concept lattices and $\mathcal{C}(\mathcal{S})$ need not agree even for the same incidence matrix.

[60]: Ganter et al. (1999), *Formal Concept Analysis: Mathematical Foundations*

Adjacent literature. The closure $\mathcal{C}(\mathcal{S})$ is related to, but distinct from, the concept lattice of Formal Concept Analysis introduced by Ganter and Wille [60]. Concept lattices take a binary object-attribute incidence relation and build the lattice of *formal concepts*, defined as pairs (A, B) with $A = B'$ and $B = A'$ under the Galois connection induced by the incidence. Closure under the concept operators differs from closure under set intersection and union, so the concept lattice of an incidence $\mathcal{S}$ and the lattice $\mathcal{C}(\mathcal{S})$ are generically different lattices even though they are built from the same data. The relation between them, and whether concept-lattice algorithms can be transplanted to compute or approximate an optimal arborescence on $\mathcal{L}$, is open.

Stockmeyer [61] proved that deciding whether a collection of subsets admits a basis of size at most k under union is NP-complete. This is a warning sign for any variant that admits arbitrary synthetic unions as candidate parents.

[61]: Stockmeyer (1975), *The set basis problem is NP-complete*

A second adjacency is the Set Basis Problem of Stockmeyer [61]. Given a set collection $\mathcal{S}$ and a target k, the problem asks whether $\mathcal{S}$ admits a basis of size k under union, meaning a family $\mathcal{B}$ with $|\mathcal{B}| \leq k$ such that every S_i is a union of members of $\mathcal{B}$. Stockmeyer proved the decision problem NP-complete. The minimum-union-basis problem is a drastically simplified sibling of the closure-specific Steiner problem, and its hardness

signals that synthetic-set optimisation over arbitrary families is unlikely to be tractable without structural assumptions on $\mathcal{S}$.

Munro, Sandlund, and Sinnamon gave the first space-efficient data structures for arbitrary finite lattices [62], within a $\Theta(\log n)$ factor of the information-theoretic lower bound $\Omega(n^{3/2})$ on representing a lattice of n elements. Their first structure uses $O(n^{3/2} \log n)$ bits, tests partial order in $O(1)$ time, and computes meets and joins in $O(n^{3/4})$ time; their second, parameterised by the maximum degree d in the transitive reduction, uses $O(n^{3/2} \log n)$ bits and supports meet and join in $O(d \log n / \log d)$ time. For a distributive lattice such as $\mathcal{L}$, these bounds are valid but coarse. The upset characterisation of Theorem A.1.1 permits a more compact representation through R, and whether its explicit materialisation in the Munro, Sandlund, Sinnamon style can be combined with SUM-level approximation is an experimental direction we do not pursue here.

Munro, Sandlund, and Sinnamon [62] match the $\Omega(n^{3/2})$ lower bound on representing an arbitrary finite lattice within a $\Theta(\log n)$ factor. Distributivity (which $\mathcal{L}$ has) permits a more compact representation via the poset R.

[62]: Munro et al. (2020), *Space-Efficient Data Structures for Lattices*

A fourth adjacency is the hierarchical-encoding problem in programming languages with multiple inheritance, studied by Aït-Kaci, Boyer, Lincoln, and Nasr [63] and by Caseau. Their objective is to encode each element of a partially ordered set by a short bitstring so that the partial order relation becomes bitwise, and lattice operations become bitwise AND/OR. The objective and the cost model differ from ours: hierarchical encoding pays by the length of the bitstring, whereas the Steiner formulation of Section A.1 pays by the sum of binomial log-costs along the arborescence. The two directions share the underlying lattice but optimise incomparable quantities.

Hierarchical encoding for multiple inheritance (Aït-Kaci, Boyer, Lincoln, Nasr [63]) assigns each lattice element a short bitstring so that lattice operations become bitwise. The objective is different from ours; we pay by the logarithm of the number of bitvectors consistent with a containment, they pay by the length of the bitstring.

[63]: Aït-Kaci et al. (1989), *Efficient implementation of lattice operations*

The technical engine behind both the lower bound L^* and the convergence theorem of this appendix is Csiszár's method of types, treated in Cover and Thomas [55, Ch. 11, § 11.1]. The method of types gives tight enumerative codes for empirical-distribution-constrained sequences and underlies the counting arguments in Proposition 5.4.1 and Theorem A.1.5. Its role here is the standard one: partition the space of collections into type classes indexed by the realised-pattern frequencies, and use the size of each type class as a bit-counting primitive. The specific application to indexed set collections through the realised-pattern alphabet is the contribution of Chapter Chapter 5 and of this appendix.

The method of types [55, Ch. 11, § 11.1] is the source of the multinomial bounds of Section 5.4 and Section A.1. Its application to indexed set collections, specifically the realised-pattern type class, is the technique underlying Proposition 5.4.1 and Theorem A.1.5.

Open problems. *OP1 (complexity of the closure-specific problem).* Is the Directed Steiner Arborescence problem on $G_{\mathcal{L}}$ with binomial arc costs $c(P, Q) = \lg \binom{|P|}{|Q|}$ NP-hard? Variant 2 is NP-hard on arbitrary candidate families with arbitrary nonnegative costs. Variant 1 is polynomial-time solvable. The intermediate regime of Variant 3, in which the candidate family is $\mathcal{L}$ and the costs are closure-specific, has no known NP-hardness reduction. A reduction from Set Basis [61] or from some restricted Directed Steiner Tree is the natural place to look.

[61]: Stockmeyer (1975), *The set basis problem is NP-complete*

OP2 (approximation). For Variants 3 and 5, is there a polynomial-time approximation algorithm whose ratio depends on $|R|$ or on width(R) rather than on u? The Charikar et al. scheme [57] gives $O(n^{\varepsilon})$ for every constant $\varepsilon > 0$ on general Directed Steiner Tree; a closure-specific version that exploits the distributive structure of $\mathcal{L}$ could plausibly achieve a constant-factor approximation in the small-$|R|$ regime.

[57]: Charikar et al. (1999), *Approximation algorithms for directed Steiner problems*

OP3 (query-supporting atom encoding). Is there a data structure that achieves $L^*(\mathcal{S})$ bit-space (Variant 7) while supporting membership, access, and rank in time sublinear in $|R| \cdot u$? The flat encoding of Proposition 5.4.5 is information-theoretically optimal but decodes to a pattern sequence that supports no direct queries; the SUM deletion tree of Theorem 5.5.2 is queryable but pays $L_{\mathrm{SUM}}(\mathcal{S}) \geq L^*(\mathcal{S})$. Closing the gap is the central query-complexity question left by Chapter Chapter 5.

OP4 (parameterised complexity). Is the closure-specific Steiner problem fixed-parameter tractable in $|R|$, in width(R), or in other structural parameters of the realised-pattern poset? The lattice size $|\mathcal{L}| = J(R) - 1$ is already singly exponential in $|R|$, so FPT in $|R|$ would be a strong positive result. FPT in width(R) is weaker but potentially more robust in practice.

OP5 (experimental evaluation). An empirical comparison of the greedy SUM encoding of Variant 6 against the flat L^* encoding of Variant 7 and against the binary-matrix bound L^+_{rows} of Subsection 5.4.3 on real datasets is an experimental question that Chapter Chapter 5 does not address. Natural candidates are web-graph adjacency lists, which drove the Boldi-Vigna compression framework [4], the colour sets of coloured de Bruijn graphs [5], and inverted indexes. A study of the instance-dependent gap $L_{\mathrm{SUM}} - L^*$ across these three families would map the regime in which SUM is near-optimal and the regime in which the surplus cost is substantial, and would direct future work toward the variants for which SUM leaves the largest room for improvement.

[4]: Boldi et al. (2004), *The webgraph framework I: compression techniques*

[5]: Alanko et al. (2023), *Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes*

Bibliography

[1] Ming Li, Paul Vitányi, et al. *An introduction to Kolmogorov complexity and its applications*. Vol. 3. Springer, 2008 (cited on page 1).

[2] Gonzalo Navarro. *Compact data structures: A practical approach*. Cambridge University Press, 2016 (cited on page 1).

[3] Guy Jacobson. 'Space-efficient static trees and graphs'. In: *30th annual symposium on foundations of computer science*. IEEE. 1989, pp. 549–554 (cited on pages 1, 8, 27–29, 33).

[4] Paolo Boldi and Sebastiano Vigna. 'The webgraph framework I: compression techniques'. In: *Proceedings of the 13th international conference on World Wide Web*. 2004, pp. 595–602 (cited on pages 2, 4, 63, 86).

[5] Jarno N Alanko et al. 'Themisto: a scalable colored k-mer index for sensitive pseudoalignment against hundreds of thousands of bacterial genomes'. In: *Bioinformatics* 39.Supplement_1 (2023), pp. i260–i269 (cited on pages 2, 3, 86).

[6] Fatemeh Almodaresi et al. 'An efficient, scalable, and exact representation of high-dimensional color information enabled using de Bruijn graph search'. In: *Journal of Computational Biology* 27.4 (2020), pp. 485–499 (cited on pages 3, 64).

[7] Jarno N Alanko et al. 'Compact data structures for collections of sets'. In: *23rd Symposium on Experimental Algorithms*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik. 2025, p. 6 (cited on pages 3–6, 58–60, 73, 80, 83).

[8] Abraham Bookstein and Shmuel T. Klein. 'Compression of correlated bit-vectors'. In: *Information Systems* 16.4 (1991), pp. 387–400 (cited on pages 3, 18, 63).

[9] Travis Gagie, Meng He, and Gonzalo Navarro. 'Compressed Set Representations based on Set Difference'. In: *arXiv preprint arXiv:2601.23240* (2026) (cited on pages 4–6, 44, 48, 50, 51, 58, 61–68, 73).

[10] Guy Jacobson. 'Succinct Static Data Structures'. PhD thesis. Pittsburgh, PA: Carnegie Mellon University, 1988 (cited on pages 8, 11, 27–29, 33).

[11] V. L. Arlazarov et al. 'On economical construction of the transitive closure of a directed graph'. In: *Doklady Akademii Nauk SSSR* 194 (1970). English translation in Soviet Math. Dokl. 11 (1970), pp. 1209–1210, pp. 487–488 (cited on pages 9, 33).

[12] David Richard Clark. 'Compact Pat Trees'. PhD thesis. Waterloo, Ontario, Canada: University of Waterloo, 1996 (cited on pages 10, 11).

[13] Peter Bro Miltersen. 'Lower Bounds on the Size of Selection and Rank Indexes'. In: *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2005, Vancouver, British Columbia, Canada, January 23–25, 2005*. SIAM. 2005, pp. 11–12 (cited on page 12).

[14] Alexander Golynski. 'Optimal Lower Bounds for Rank and Select Indexes'. In: *Automata, Languages and Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10–14, 2006, Proceedings, Part I*. Vol. 4051. Lecture Notes in Computer Science. Journal version: Theoretical Computer Science 387(3), 2007, pp. 348–359. Springer, 2006, pp. 370–381 (cited on pages 12, 13).

[15] Paul Beame and Faith E. Fich. 'Optimal Bounds for the Predecessor Problem and Related Problems'. In: *Journal of Computer and System Sciences* 65.1 (2002), pp. 38–72 (cited on page 13).

[16] Mihai Pătraşcu. 'Succincter'. In: *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, Philadelphia, PA, USA, October 25–28, 2008*. IEEE. 2008, pp. 305–313 (cited on page 13).

[17] Roberto Grossi et al. 'More Haste, Less Waste: Lowering the Redundancy in Fully Indexable Dictionaries'. In: *26th International Symposium on Theoretical Aspects of Computer Science, STACS 2009, Freiburg, Germany, February 26–28, 2009, Proceedings*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2009, pp. 517–528 (cited on page 13).

[18] Venkatesh Raman and S. Srinivasa Rao. 'Static Dictionaries Supporting Rank'. In: *Algorithms and Computation, 10th International Symposium, ISAAC '99, Chennai, India, December 16–18, 1999, Proceedings*. Vol. 1741. Lecture Notes in Computer Science. Springer, 1999, pp. 18–26 (cited on page 14).

[19] Rajeev Raman, Venkatesh Raman, and Srinivasa Rao Satti. 'Succinct indexable dictionaries with applications to encoding k-ary trees, prefix sums and multisets'. In: *ACM Transactions on Algorithms (TALG)* 3.4 (2007), 43–es (cited on pages 14, 15).

[20] Michael L. Fredman, János Komlós, and Endre Szemerédi. 'Storing a Sparse Table with $O(1)$ Worst Case Access Time'. In: *Journal of the ACM* 31.3 (1984), pp. 538–544 (cited on page 14).

[21] Peter Elias. 'Efficient Storage and Retrieval by Content and Address of Static Files'. In: *Journal of the ACM* 21.2 (1974), pp. 246–260 (cited on page 16).

[22] Robert M. Fano. *On the number of bits required to implement an associative memory*. Tech. rep. Memorandum 61. Cambridge, Massachusetts: Computer Structures Group, Project MAC, Massachusetts Institute of Technology, 1971 (cited on page 16).

[23] Roberto Grossi and Jeffrey Scott Vitter. 'Compressed Suffix Arrays and Suffix Trees with Applications to Text Indexing and String Matching'. In: *SIAM Journal on Computing* 35.2 (2005), pp. 378–407 (cited on pages 16, 17).

[24] Daisuke Okanohara and Kunihiko Sadakane. 'Practical entropy-compressed rank/select dictionary'. In: *2007 Proceedings of the Ninth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM. 2007, pp. 60–70 (cited on pages 18, 57).

[25] Sebastiano Vigna. 'Quasi-Succinct Indices'. In: *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining, WSDM 2013, Rome, Italy, February 4–8, 2013*. ACM. 2013, pp. 83–92 (cited on page 18).

[26] Kunihiko Sadakane and Roberto Grossi. 'Squeezing Succinct Data Structures into Entropy Bounds'. In: *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22–26, 2006*. SIAM. 2006, pp. 1230–1239 (cited on page 19).

[27] Roberto Grossi, Ankur Gupta, Jeffrey Scott Vitter, et al. 'High-order entropy-compressed text indexes'. In: *SODA*. Vol. 3. 2003, pp. 841–850 (cited on pages 20, 22, 23, 49).

[28] Gonzalo Navarro. 'Wavelet Trees for All'. In: *Journal of Discrete Algorithms* 25 (2014), pp. 2–20 (cited on pages 21, 23, 24).

[29] Paolo Ferragina et al. 'Compressed Representations of Sequences and Full-Text Indexes'. In: *ACM Transactions on Algorithms* 3.2 (2007), p. 20 (cited on pages 23, 24, 52).

[30] Bernard Chazelle. 'A Functional Approach to Data Structures and Its Use in Multidimensional Searching'. In: *SIAM Journal on Computing* 17.3 (1988), pp. 427–462 (cited on page 24).

[31] J Ian Munro and Venkatesh Raman. 'Succinct representation of balanced parentheses and static trees'. In: *SIAM Journal on Computing* 31.3 (2001), pp. 762–776 (cited on pages 26–28, 32).

[32] J. Ian Munro. 'Tables'. In: *Foundations of Software Technology and Theoretical Computer Science, 16th Conference, FSTTCS 1996, Hyderabad, India, December 18–20, 1996, Proceedings*. Vol. 1180. Lecture Notes in Computer Science. Springer, 1996, pp. 37–42 (cited on page 33).

[33] Michael A. Bender and Martín Farach-Colton. 'The LCA Problem Revisited'. In: *LATIN 2000: Theoretical Informatics, 4th Latin American Symposium, Punta del Este, Uruguay, April 10–14, 2000, Proceedings*. Vol. 1776. Lecture Notes in Computer Science. Springer, 2000, pp. 88–94 (cited on pages 34, 38, 66).

[34] David Benoit et al. 'Representing Trees of Higher Degree'. In: *Algorithmica* 43.4 (2005), pp. 275–292 (cited on pages 34, 35).

[35] Richard F. Geary, Rajeev Raman, and Venkatesh Raman. 'Succinct Ordinal Trees with Level-Ancestor Queries'. In: *ACM Transactions on Algorithms* 2.4 (2006), pp. 510–534 (cited on pages 36–39, 41, 55).

[36] Meng He, J. Ian Munro, and S. Srinivasa Rao. 'Succinct Ordinal Trees Based on Tree Covering'. In: *Automata, Languages and Programming, 34th International Colloquium, ICALP 2007, Wroclaw, Poland, July 9-13, 2007, Proceedings*. Vol. 4596. Lecture Notes in Computer Science. Journal version: ACM Transactions on Algorithms 8(4), 2012, Article 42. Springer, 2007, pp. 509–520 (cited on pages 36, 38).

[37] Arash Farzan and J. Ian Munro. 'A Uniform Approach Towards Succinct Representation of Trees'. In: *Algorithm Theory – SWAT 2008, 11th Scandinavian Workshop on Algorithm Theory, Gothenburg, Sweden, July 2–4, 2008, Proceedings*. Lecture Notes in Computer Science. Journal version in Algorithmica 2014. Springer, 2008, pp. 173–184 (cited on pages 39, 55).

[38] Gonzalo Navarro and Kunihiko Sadakane. 'Fully Functional Static and Dynamic Succinct Trees'. In: *ACM Transactions on Algorithms* 10.3 (2014), 16:1–16:39 (cited on pages 39, 40).

[39] Meng He, J Ian Munro, and Gelin Zhou. 'A framework for succinct labeled ordinal trees over large alphabets'. In: *Algorithmica* 70.4 (2014), pp. 696–717 (cited on pages 40–42, 46–48).

[40] Kuo-Chung Tai. 'The tree-to-tree correction problem'. In: *Journal of the ACM* 26.3 (1979), pp. 422–433. DOI: 10.1145/322139.322143 (cited on page 43).

[41] Philip Bille. 'A survey on tree edit distance and related problems'. In: *Theoretical Computer Science* 337.1–3 (2005), pp. 217–239. DOI: 10.1016/j.tcs.2004.12.030 (cited on page 43).

[42] Meng He, J. Ian Munro, and Gelin Zhou. 'Path queries in weighted trees'. In: *Algorithms and Computation (ISAAC 2011)*. Ed. by Takao Asano et al. Vol. 7074. Lecture Notes in Computer Science. Springer, 2011, pp. 140–149. DOI: 10.1007/978-3-642-25591-5_16 (cited on page 43).

[43] Meng He, J Ian Munro, and Gelin Zhou. 'Data structures for path queries'. In: *ACM Transactions on Algorithms (TALG)* 12.4 (2016), pp. 1–32 (cited on pages 43–45, 49–51, 53, 54).

[44] Djamal Belazzougui and Gonzalo Navarro. 'Optimal lower and upper bounds for representing sequences'. In: *ACM Transactions on Algorithms* 11.4 (2015), 31:1–31:21. DOI: 10.1145/2629339 (cited on page 47).

[45] Jérémy Barbay et al. 'Succinct indexes for strings, binary relations and multi-labeled trees'. In: *ACM Transactions on Algorithms* 7.4 (2011), 52:1–52:27. DOI: 10.1145/2000807.2000820 (cited on page 47).

[46] Alexander Golynski, J. Ian Munro, and S. Srinivasa Rao. 'Rank/select operations on large alphabets: a tool for text indexing'. In: *Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2006)*. SIAM. 2006, pp. 368–373 (cited on page 47).

[47] David R. Clark and J. Ian Munro. 'Efficient suffix trees on secondary storage (extended abstract)'. In: *Proceedings of the Seventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 1996)*. Distinct from Clark's 1996 PhD thesis "Compact Pat Trees" already in this bib. SIAM. 1996, pp. 383–391 (cited on page 48).

[48] Prosenjit Bose et al. 'Succinct orthogonal range search structures on a grid with applications to text indexing'. In: *Algorithms and Data Structures Symposium (WADS 2009)*. Vol. 5664. Lecture Notes in Computer Science. Springer, 2009, pp. 98–109. DOI: 10.1007/978-3-642-03367-4_9 (cited on page 53).

[49] Timothy M. Chan, Kasper Green Larsen, and Mihai Pătraşcu. 'Orthogonal range searching on the RAM, revisited'. In: *Proceedings of the 27th Annual Symposium on Computational Geometry (SoCG 2011)*. ACM, 2011, pp. 1–10. DOI: 10.1145/1998196.1998198 (cited on page 54).

[50] Adam L. Buchsbaum et al. 'Engineering the compression of massive tables: an experimental approach'. In: *Proceedings of the Eleventh Annual ACM-SIAM Symposium on Discrete Algorithms (SODA 2000)*. ACM/SIAM, 2000, pp. 175–184 (cited on page 63).

[51] Andreas Björklund and Andrzej Lingas. 'Fast Boolean Matrix Multiplication for Highly Clustered Data'. In: *Algorithms and Data Structures, 7th International Workshop (WADS 2001), Providence, RI, USA, August 8–10, 2001, Proceedings*. Vol. 2125. Lecture Notes in Computer Science. Springer, 2001, pp. 258–263 (cited on page 63).

[52] José N. F. Alves et al. 'Accelerating Graph Neural Networks Using a Novel Computation-Friendly Matrix Compression Format'. In: *IEEE International Parallel and Distributed Processing Symposium (IPDPS 2025)*. IEEE, 2025, pp. 1091–1103 (cited on page 64).

[53] Alberto Apostolico. 'The myriad virtues of subword trees'. In: *Combinatorial Algorithms on Words*. Vol. 12. NATO ASI Series F. Springer-Verlag, 1985, pp. 85–96 (cited on page 66).

[54] Martín Farach-Colton, Paolo Ferragina, and S. Muthukrishnan. 'On the sorting-complexity of suffix tree construction'. In: *Journal of the ACM* 47.6 (2000), pp. 987–1011 (cited on page 66).

[55] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley-Interscience, 2006 (cited on pages 70, 72, 85).

[56] Garrett Birkhoff. 'Rings of sets'. In: *Duke Mathematical Journal* 3.3 (1937), pp. 443–454 (cited on pages 76, 78, 79).

[57] Moses Charikar et al. 'Approximation algorithms for directed Steiner problems'. In: *Journal of Algorithms* 33.1 (1999). Preliminary version in SODA 1998, pp. 192–200, pp. 73–91 (cited on pages 76, 80, 83, 85).

[58] Richard Dedekind. 'Über Zerlegungen von Zahlen durch ihre grössten gemeinsamen Teiler'. In: *Festschrift der Technischen Hochschule zu Braunschweig*. Reproduced in *Gesammelte mathematische Werke*, Vol. II. 1897, pp. 103–148 (cited on page 79).

[59] Daniel Kleitman. 'On Dedekind's problem: The number of monotone Boolean functions'. In: *Proceedings of the American Mathematical Society* 21.3 (1969), pp. 677–682 (cited on page 79).

[60] Bernhard Ganter and Rudolf Wille. *Formal Concept Analysis: Mathematical Foundations*. Berlin: Springer, 1999 (cited on page 84).

[61] Larry J. Stockmeyer. 'The set basis problem is NP-complete'. In: *IBM Research Report RC-5431* (1975). Published as: Larry J. Stockmeyer, "The Set Basis Problem is NP-Complete", IBM Watson Research Center Technical Report RC-5431, 1975; also cited in SIAM J. Comput. literature (cited on pages 84, 85).

[62] J. Ian Munro, Bryce Sandlund, and Corwin Sinnamon. 'Space-Efficient Data Structures for Lattices'. In: *17th Scandinavian Symposium and Workshops on Algorithm Theory (SWAT 2020)*. Vol. 162. LIPIcs. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, 31:1–31:22. DOI: 10.4230/LIPIcs.SWAT.2020.31 (cited on page 85).

[63] Hassan Aït-Kaci et al. 'Efficient implementation of lattice operations'. In: *ACM Transactions on Programming Languages and Systems* 11.1 (1989), pp. 115–146 (cited on page 85).